ASSIGNMENT COVERSHEET

UTS: ENGINEERING & INFORMATION TECHNOLOGY

SUBJECT NUMBER & NAME	NAME OF STUDENT(s) (PRINT CLEARLY)	STUDENT ID(s)
41026 Advanced Software Development	*Sangyun Kim* *Kartikay Singh* *Kris Ngo* *Ethan Schweinsberg* *Brian Tran*	13317324 25020443 14488643 25267321 25491679
	SURNAME *FIRST NAME*	

STUDENT EMAIL	STUDENT CONTACT NUMBER
sangyun.kim@student.uts.edu.au kartikay.singh@student.uts.edu.au ethan.schweinsberg@student.uts.edu.au brian.tran-1@student.uts.edu.au mai.t.ngo@student.uts.edu.au	0413601151 0493038173 0477684060 0426476015

NAME OF TUTOR	TUTORIAL GROUP	DUE DATE
Grace Billiris	Group 36	09/09/2026

ASSESSMENT ITEM NUMBER & TITLE

Assignment
Assessment 1 – Release 0: Agentic AI Foundations, Microservices & DevOps

- ☑ I acknowledge that if AI or another nonrecoverable source was used to generate materials for background research and self-study in producing this assignment, I have checked and verified the accuracy and integrity of the information used.
- ☑ I confirm that I have read, understood and followed the guidelines for assignment submission and presentation on page 2 of this cover sheet.
- ☑ I confirm that I have read, understood and followed the advice in the Subject Outline about assessment requirements.
- ☑ I understand that if this assignment is submitted after the due date it may incur a penalty for lateness unless I have previously had an extension of time approved and have attached the written confirmation of this extension.

Declaration of originality: The work contained in this assignment, other than that specifically attributed to another source, is that of the author(s) and has not been previously submitted for assessment. I have rewritten any material provided by AI or other nonrecoverable sources and where appropriate acknowledged their contribution. I understand that, should this declaration be found to be false, disciplinary action could be taken and penalties imposed in accordance with University policy and rules. In the statement below, I have indicated the extent to which I have collaborated with others, whom I have named.

No content generated by AI technologies or other sources has been presented as my own work and I have rewritten any text provided by AI or other sources in my own words.

Statement of collaboration:

Signature of student(s) ____  **__KN _____KS_____Sangyun Kim_____BT_____ Date: 06/09/2026_____**

ASSIGNMENT RECEIPT

To be completed by the student if a receipt is required

SUBJECT NUMBER & NAME	NAME OF TUTOR
SIGNATURE OF TUTOR	**RECEIVED DATE**

Release 0 - Technical Report (Assessment 1)

Team

Name	Student ID	Email
Kartikay Singh	25020443	kartikay.singh@student.uts.edu.au
Ethan Schweinsberg	25267321	ethan.schweinsberg@student.uts.edu.au
Sangyun Kim	13317324	sangyun.kim@student.uts.edu.au
Kris Ngo	14488643	mai.t.ngo@student.uts.edu.au
Brian Tran	25491679	brian.tran-1@student.uts.edu.au

1. Project Overview

OmniTech Marketplace is an Agentic AI-powered e-commerce platform for consumer electronics. The project is designed to make it easier for customers to look through products, compare technical specifications, receive product recommendations, manage their cart and orders, and submit warranty claims or return requests in one system.

For Release 0, our team focused on building the main technical foundation of the application. Each team member was responsible for one feature, which currently runs in its own container and includes the frontend, backend/API, and seeded SQLite data needed for CRUD operations. All features support CRUD operations and are connected through a shared home page. At this stage, each feature has its own interface design with a shared CSS theme, and a fully consistent shared visual style will be improved in later releases.

The application also includes AI-Mode through a shared Ollama service using approved open-source LLMs. The AI helps with feature-specific tasks, such as summarising product specifications, generating customer preference tags, recommending suitable products, checking cart compatibility, and assisting with warranty requests. Each AI feature follows the Plan → Act → Observe → Adapt workflow to plan a response, use the relevant data, check the result, and adjust it if needed.

The five features in the integrated system are Product Catalog and Specifications, Customer Profiles and Preferences, AI Product Consultant, Cart and Order Processing, and Warranty and Return. Docker Compose is used to run the whole system together, while GitHub Actions is used to build and validate each student's services.

2. Individual Feature Allocation

Student 1

Student	Kris Ngo
Feature Name	Product Catalog and Specs
Feature Description	Manage product listings, categories, prices, stock, and technical specifications for customers and the AI consultant.

Student 2

Student	Ethan Schweinsberg
Feature Name	Customer Profiles & Preferences
Feature Description	Manage user accounts, ecosystem preferences and their shipping profiles, using AI to create personalised preference tags.

Student 3

Student	Kartikay Singh
Feature Name	AI Product Consultant
Feature Description	Provides customers with an intelligent, conversational tech advisory interface that recommends consumer electronics based on their specific needs and the store's inventory, utilizing the Agentic AI workflow.

Student 4

Student	Sangyun Kim
Feature Name	Cart & Order Processing
Feature Description	Manages customer shopping carts, seamless checkout processes, and order history, while utilizing an AI agent to validate hardware compatibility and suggest relevant accessories based on the cart's contents.

Student 5

Student	Brian Tran
Feature Name	Warranty, Returns & RMA Logistics
Feature Description	Manage warranty claims, product returns, ticket status and remove cancelled claims.

3. Agile Team Project Plan

3.1 Methodology

The team followed an Agile Scrum methodology for our project. Work was organized into 5 different sprints based and aligned with the three release milestones. Each sprint is designed to target a specific deliverable set and the progress is tracked through GitHub issues, pull requests and weekly workshop discussions.

3.2 Sprint Structure

Sprint	Duration	Release	Focus
Sprint 1	Weeks 1–4	-	Team formation, project approval, repo setup, feature selection
Sprint 2	Weeks 5–6	Release 0	Individual microservice development, database setup, CI/CD pipelines
Sprint 3	Weeks 7–8	Release 0	Integration, Docker Compose, initial shared CSS styling, testing, report and video
Sprint 4	Weeks 9–10	Release 1	MCP server, RAG server, grounded AI responses
Sprint 5	Weeks 11–12	Release 2	Multi-agent system, cloud deployment, final testing

3.3 Team Ceremonies

Ceremony	Frequency	Purpose
Sprint Planning	Start of each sprint	Define sprint backlog items and assign owners
Stand-up	Weekly (workshop session)	Sync progress, surface blockers
Sprint Review	End of sprint	Demo working software to the team
Retrospective	End of sprint	Identify what went well and what to improve

3.4 Communications & Collaborations

1. We handle source control through a shared GitHub repository (Pizzalilla/OmiTech) with protection on main.
2. Each team member developed their feature on their own branch and merged into main via Pull Request. Each pull request needs to be confirmed by other members before merging into main.
3. Discord Server to chat for daily coordination, problems or meetings for synchronization planning.
4. We track all our tasks through the GitHub issues and Pull requests labelled by the feature area it worked on

3.5 Definition Of Done

A feature is considered to be finished for a given release when it has:

1. All functional requirements for that release are implemented
2. The automated tests pass locally but also in the Ci on GitHub
3. The feature starts successfully through the shared docker-compose.yml configuration without build errors.
4. The feature is accessible from the shared index.html page
5. The pull request for the code has been reviewed and merged into main.
6. Pre-testing and post-testing documentation is complete

4. Sprint Backlog

How we worked

We split the work so that each student owns one feature and writes their own user stories. Stories with the ID US-0.x are shared team work. Stories with the ID US-1.x to US-5.x belong to the matching student number.

We tracked every story on a GitHub Projects board with four columns, which were To Do, In Progress, Review and Done. We estimated in story points using 1, 2, 3, 5 and 8. We checked progress at the Wednesday workshop each week.

Definition of Done

- The code is merged into main after another student reviewed it.
- The feature runs from docker compose up with no extra steps.
- The database table has at least ten records and all CRUD operations work.
- The feature is linked from the shared home page and uses the agreed shared CSS styling theme where applicable.
- The AI mode returns a response from the local Ollama model.
- The student's GitHub Actions workflow passes.

Sprint backlog

Team

ID	User story	Owner	Est	Status
US-0.1	Set up the shared GitHub repository with the standard folder structure.	Student 1	3	Done
US-0.2	Build one home page that links to all five features.	Student 2	5	Done
US-0.3	Apply one shared CSS theme across every page.	Student 5	3	Done
US-0.4	Write docker-compose.yml so the whole application starts with one command.	Student 4	8	Done
US-0.5	Run Ollama as a shared service with the approved models pulled.	Student 3	5	Done
US-0.6	Build a shared Plan, Act, Observe, Adapt helper for all AI modes.	All five	8	Done
US-0.7	Add student-1.yml to student-5.yml GitHub Actions workflows.	All five	5	Done
US-0.8	Run a full integration test before the showcase.	All five	3	Done
US-0.9	Record and publish the showcase video.	All five	3	Done
US-0.10	Put the Release 0 technical report together.	All five	5	Done

Individual feature stories

Each student has three stories, which are the frontend page, the backend and database with CRUD, and the AI mode.

ID	User story	Owner	Est	Status
US-1.1	Browse, search, and filter the product catalogue by category, brand, price, and specifications.	Student 1	5	Done
US-1.2	Add, edit and delete catalogue items, seeded with ten records.	Student 1	8	Done
US-1.3	AI mode generates and reviews a shopper-friendly summary for an existing catalogue product using its stored specifications.	Student 1	5	Done
US-2.1	Create an account and view a profile page.	Student 2	5	Done
US-2.2	View, update and delete profiles, seeded with ten users.	Student 2	8	Done
US-2.3	AI mode generates preference tags based on a customer’s profile and stored preferences.	Student 2	5	Done
US-3.1	Start and view AI product consultation sessions based on a customer’s budget, preferences, and intended use.	Student 3	5	Done
US-3.2	Create, rename, and delete consultation sessions, with seeded session, message, and recommendation data..	Student 3	8	Done
US-3.3	AI mode recommends suitable products from the available seeded product catalogue data.	Student 3	5	Done
US-4.1	Add items to a cart and change or remove them.	Student 4	5	Done
US-4.2	View seeded order history and create an order through the cart checkout process.	Student 4	8	Done
US-4.3	AI mode checks cart items for compatibility and suggests relevant accessories before checkout.	Student 4	8	Done
US-5.1	Create a warranty or return ticket using a customer order.	Student 5	5	Done
US-5.2	View, update, and delete warranty or return tickets.	Student 5	8	Done
US-5.3	AI mode evaluates a warranty claim and provides a decision with a reason.	Student 5	5	Done

5. Overall Project Plan

The OmniTech Marketplace application will be built as microservice architecture, which will comprise of five isolated services, a static entry gateway and a shared local AI service. The five microservices will be on ports (5001-5005), the entry gateway will be on port 8080, and the shared AI service will be on port 11434.

Each student will create a stack consisting of a Flask REST API backend (not yet implemented), HTML fronted, and an isolated SQLite database with a minimum of 10 seed records per table. A shared Ngux web server running on port 8080 will serve as a static gateway that can route users to each microservice endpoint. There will be a shared CSS stylesheet that is applied across all microservice templates so there can be a unified typography across microservices. The system will be managed by a shared .yml configuration file, which will be able to handle service startup and port allocation. All the microservices will communicate with a Ollama LLM instance which will be accessible on port 11434.

Each microservice will implement an agentic execution loop through Plan -> Act -> Observe -> Adapt where it will retrieve domain context for database tables, issue a request to the shared Ollama service for a structured response, validate the AI output, and then commit validated date or re-prompt Ollama if it failed validation for any reason.

6. Functional and Non-Functional Requirements

6.1 Functional Requirements

ID	Requirement	Priority	Acceptance criteria
FR-1.1	The system shall allow users to view the product catalogue.	Must Have	The catalogue page displays the product records, including product name, category, price, stock status, and key specifications.
FR-1.2	The system shall allow users to search for products by name or specification keyword.	Must Have	Entering a valid keyword updates the displayed products to show matching items.
FR-1.3	The system shall allow users to filter products by brand, price range, and specification criteria.	Should Have	Applying one or more filters shows only products that match the selected criteria.
FR-1.4	The system shall allow users to view detailed product specifications.	Must Have	Selecting a product displays its full specifications, price, availability, and product description.
FR-1.5	The system shall allow an administrator to create a product record.	Must Have	Submitting valid product details adds the product to the catalogue and it remains visible after refreshing the page.
FR-1.6	The system shall allow an administrator to update an existing product record.	Must Have	Editing a product’s details saves the changes and the updated information appears in the catalogue.
FR-1.7	The system shall allow an administrator to delete a product record.	Must Have	Deleting a product removes it from the catalogue and it no longer appears in search or filter results.
FR-1.8	The system shall use AI-Mode to summarise a product’s technical specifications.	Must Have	Selecting the AI summary option returns a short, readable summary based on the selected product’s seeded specifications.
FR-1.9	The system shall validate the AI summary before displaying it to the user.	Should Have	The summary is only displayed when it refers to the selected product and does not include unrelated product details; otherwise, the system retries or shows an error message.
FR-1.10	The product catalogue shall be seeded with at least ten product records.	Must Have	When the application first starts, the catalogue contains at least ten products without requiring manual data entry.
FR-2.1	The system shall allow users to create a	Must Have	Submitting valid customer details creates a new profile

	customer profile.		and displays it in the customer list.
FR-2.2	The system shall allow users to view all customer profiles.	Must Have	The customer page displays seeded and newly created customer profiles.
FR-2.3	The system shall allow users to update all customer profiles.	Must Have	The
FR-2.4	The system shall allow users to delete all customer profiles.	Must Have	
FR-2.5	The system shall validate when creating a customer profile.	Should Have	
FR-2.6	The system shall allow users to lookup customer profiles.	Should Have	
FR-2.7	The system shall validate when updating a customer profile.	Should Have	
FR-2.8	The system shall confirm when deleting a customer profile.	Should Have	
FR-2.9	The system shall execute an agnetic loop to extract preference tags from customer notes	Must Have	
FR-2.10	The system shall allow users to update preference tags on customer profiles	Must Have	
FR-2.11	The system shall allow users to delete preference tags from customer profiles.	Must Have	
FR-2.12	The system shall validate the AI preference tags are valid	Must Have	
FR-2.13	The system shall save preference tags to user profiles and have them persist between sessions	Must Have	
FR-3.1	The system shall allow users to create new consultation sessions with a custom title.	Must Have	POST /api/sessions returns 201 with the session object containing the provided title and a generated ID.
FR-3.2	The system shall allow users to view a list of all past consultation sessions, ordered by most recent.	Must Have	GET /api/sessions returns a JSON array sorted by updated_at descending. The list contains all seeded and user-created sessions.
FR-3.3	The system shall allow users to rename an existing consultation session.	Should Have	PUT /api/sessions/<id> with a new title returns 200 and the updated session. Subsequent GET reflects the change. Returns 400 if the title is empty.
FR-3.4	The system shall allow users to delete a consultation session, cascading to all associated messages and recommendations.	Must Have	DELETE /api/sessions/<id> returns 200. A follow-up GET for the same session returns 404. All child ChatLogs and SavedRecommendations rows are also deleted (CASCADE).
FR-3.5	The system shall allow users to send a natural-language message within a consultation session and receive an AI-generated product recommendation.	Must Have	POST /api/chat with session_id and message returns 200 with a non-empty reply field. Both the user message and the AI reply are persisted in ChatLogs.
FR-3.6	The AI response shall recommend only products that exist in the OmniTech product catalog, validated against known product IDs.	Must Have	Every ID in recommended_product_ids exists in catalog.VALID_IDS. IDs not in the catalog shortlist are rejected during the Observe stage.

FR-3.7	The system shall implement the Plan → Act → Observe → Adapt agentic AI workflow for each consultation turn.	Must Have	The meta field in the chat response includes stage, attempts, reprompts, and used_fallback, confirming all four stages executed. Agent logs show PLAN, ACT, OBSERVE, and DONE/ADAPT entries.
FR-3.8	The system shall automatically save product recommendations returned by the AI into the SavedRecommendations table.	Must Have	When the AI response contains recommended_product_ids, a new row appears in SavedRecommendations with matching product_ids and summary. The saved_recommendation field in the JSON response is non-null.
FR-3.9	The system shall allow users to view saved recommendations for a session, including resolved product details (name, price, specs).	Must Have	GET /api/sessions/<id>/recommendations returns a JSON array where each entry includes a products array with full catalog details (name, price, specs, category).
FR-3.10	The system shall allow users to add custom preference tags to saved recommendations (e.g., "budget", "travel").	Should Have	POST /api/recommendations/<id>/tags with tag="Great Value" returns 200 with great-value in the tags array. Tags are slugified (lowercase, hyphens). Duplicate tags are not added.
FR-3.11	The system shall allow users to delete individual saved recommendations.	Should Have	DELETE /api/recommendations/<id> returns 200. The recommendation no longer appears in the session's recommendation list.
FR-3.12	The system shall allow users to clear the entire chat history of a session while preserving the session itself.	Should Have	DELETE /api/sessions/<id>/messages returns 200. A follow-up GET for messages returns an empty array. The session itself still exists via GET /api/sessions/<id>.
FR-3.13	The system shall provide a dashboard view showing all past consultation sessions with message counts and recommendation counts.	Should Have	GET /dashboard returns 200 with HTML containing both "Past consultations" and "Saved product recommendations" headings. Each session card displays message and recommendation counts.
FR-3.14	The system shall gracefully degrade to a deterministic keyword-based catalog search when the Ollama LLM is unreachable.	Must Have	When Ollama is offline, POST /api/chat still returns 200 with a non-empty reply containing real catalog products. meta.used_fallback is true. No error is shown to the user.
FR-3.15	The system shall filter consultation sessions by user ID via query parameter.	Could Have	GET /api/sessions?user_id=u-1001 returns only sessions where user_id matches. All returned rows have user_id == "u-1001".
FR-3.16	The frontend shall update dynamically using HTMX partial rendering without full page reloads.	Must Have	Sending a message appends chat bubbles via hx-swap="beforeend". The recommendations dock updates via out-of-band swap (hx-swap-oob). No full page reload occurs during normal chat interaction.
FR-3.17	Each database table shall be pre-populated with a minimum of 10 seed records on first run.	Must Have	On first startup, ConsultationSessions has 10 rows, ChatLogs has 20 rows (2 per session), and SavedRecommendations has 10 rows. Verified by SELECT COUNT(*) queries.
FR-4.1	The system shall allow users to view the items in their shopping cart.	Must Have	The cart page displays each item with product name, category, unit price, quantity and line total.
FR-4.2	The system shall show whether each cart item is in stock.	Must Have	Each item displays an "In Stock" or "Out of Stock" indicator.
FR-4.3	The system shall allow users to increase the quantity of a cart item.	Must Have	Selecting the increase button raises the quantity by one and updates the line total.
FR-4.4	The system shall allow users to decrease the quantity of a cart item.	Must Have	Selecting the decrease button lowers the quantity by one; reducing it to zero removes the item.

FR-4.5	The system shall allow users to remove an item from the cart.	Must Have	Selecting remove deletes the item and the cart no longer displays it.
FR-4.6	The system shall update the cart without reloading the page.	Should Have	Cart and summary areas refresh in place after any cart action.
FR-4.7	The system shall allow users to choose between standard delivery and store pick-up.	Must Have	Selecting a fulfilment mode updates the summary to show the chosen mode.
FR-4.8	The system shall display an order summary for the cart.	Must Have	The summary shows subtotal, item count, fulfilment charge, estimated GST and grand total.
FR-4.8	The system shall display all prices in Australian dollars.	Should Have	Every price is formatted as $#####.##
FR-4.10	The system shall provide a health-check endpoint for the database microservice.	Must Have	GET / returns service name, status and port with HTTP 200.
FR-4.11	The system shall return all orders as a list.	Must Have	GET /orders returns HTTP 200 and a JSON array sorted by order number.
FR-4.12	The system shall return a single order with its line items.	Must Have	GET /orders/{id} returns the order and a list of its line items.
FR-4.13	The system shall return a single cart with its items.	Should Have	GET /carts/{id} returns the cart and a list of its items.
FR-4.14	The database shall be populated with at least ten records per table.	Must Have	The seed script loads 10 orders, 13 line items and 2 carts.
FR-4.15	The system shall provide AI advice on the appliances in the user's cart.	Must Have	Requesting an audit returns two points covering power draw and installation clearance.
FR-4.16	The system shall allow users to ask the AI a free-text question about their cart.	Must Have	Submitting a question returns a relevant answer in the AI panel.
FR-4.17	The system shall follow the Plan → Act → Observe → Adapt agentic workflow.	Must Have	The backend builds a prompt from the cart (Plan), calls the LLM (Act), displays the result (Observe), and re-runs against an updated cart (Adapt).
FR-4.18	The system shall remain usable when the AI service is unavailable.	Must Have	An AI failure displays an "AI Helper Offline" message while cart, checkout and history keep working.
FR-4.19	The feature shall be reachable from the unified team home page.	Must Have	The Student 4 card on the home page opens the cart page.
FR-4.20	The feature shall use the team's shared CSS theme.	Must Have	The page loads the shared stylesheet and uses the team's colour and spacing tokens.
FR-4.21	The feature shall provide navigation to the other students' microservices.	Should Have	The navigation bar links to the other four services and marks the current one active.
FR-4.22	The frontend shall communicate with the backend through REST APIs only.	Must Have	All interactions are HTTP requests to /api/... routes; the browser never reaches the database directly.
FR-5.1	The system shall allow users to create a warranty return ticket with a claim	Must	Selecting an order will display a form where the user can enter a claim and create the ticket.
FR-5.2	The system shall allow a users to view all tickets	Must	Entering the view tickets page should display all the current warranty tickets with its id, product category, claim and status.
FR-5.3	The system shall allow a user to evaluate a ticket	Must	Clicking the evaluate button on the ticket will prompt the AI to evaluate the ticket’s claim against policy rules.

FR-5.4	The AI response shall generate a decision	Must	The AI will display its decision inside the decisions box after pressing the evaluate button.
FR-5.5	The AI response shall generate reasonings to the decision	Must	Underneath the decisions box, the AI will display the reasons as to why they made that decision.
FR-5.6	The system shall allow a user to update a ticket with the AI's response	Must	When pressing update, the AI's decision and reasoning will be saved to the ticket as well as its status changing, this will be updated on the screen in the tickets table above it.
FR-5.7	The system shall automatically determine the product's category when creating a ticket from an order	Must	Selecting an order to create a ticket for will display the product's name and category from the order.
FR-5.8	The frontend shall update the ticket table after updating a ticket	Should	Pressing update after the AI's evaluation will instantly update the ticket table displaying its new status as the AI's decision.
FR-5.9	Each database table shall be initialised with at least 10 seed data	Must	Starting the application will initialise the database and pre-populate the database tables with 10 tickets, products and orders.

6.2 Non-functional requirements

ID	Type	Requirement	How it was verified
NFR-1.1	Data integrity	Product records must not accept missing names or brands, invalid category IDs, negative prices, or negative stock values.	Automated product-admin tests submit invalid values and confirm that the request returns an error without creating or updating the record.
NFR-1.2	Reliability	The AI product-summary feature must not display a summary containing unsupported product claims.	**test_agentic_loop.py** checks that unsupported claims are rejected, the agent retries when needed, and no summary is shown after the maximum number of attempts.
NFR-1.3	Usability	Administrative updates to categories, products, and specifications should update only the relevant page area rather than requiring a full browser reload.	Demonstrated through the HTMX partial pages and covered by the admin and catalogue tests.
NFR-1.4	Maintainability	Catalogue, database, and AI-agent logic must remain separated into backend, database, and test folders so changes can be tested independently.	Repository structure review confirms that **backend/**, **database/**, and **tests/** are separated within **student-1/**.
NFR-1.5	Testability	The Product Catalogue feature must provide automated tests for product CRUD, categories, specifications, AI summaries, and the agentic workflow.	The **student-1/tests/** folder contains separate pytest files for these functions, and student-1.yml runs the suite in GitHub Actions.
NFR-1.6	Portability	The Student 1 service must build and start in Docker, with a reachable health endpoint.	**student-1.yml** builds the Docker image, starts a container, checks **/health**, and verifies the seeded category and product records.
NFR-2.1	Performance	The HTMX frontend must update dynamic components	Using browser developer tools to confirm API calls return HTML partials in >150ms
NFR-2.2	Reliability	The AI agent should have a rule based fallback if Ollama LLM is inaccessible	Verified by stopping the ollama-service container and confirming fallback_plan() executed automatically

		for any reason	without issue
NFR-2.3	Portability	The microservice must run as an isolated container environment on port 5002 without database dependencies on other services	Deploy the standalone container using docker-compose up student-2 and verifying API communication
NFR-2.4	Reliability	Deleting a customer record must remove all associated customer preferences and tags from the database	Executing DELETE /customers/<id> and querying foreign key child tables
NFR-2.5	Maintainability	Backend routes and database utilities must complete automated testing without failing or warnings	Running pytest tests/test_app.py and passing 100% of the tests
NFR-2.6	Maintainability	The agentic loop must output structured logs in a Plan->Act->Observe->Adapt format	Verified using docker logs -f student-2 while going through the tag generation process to show the steps in the terminal
NFR-3.1	Constraint	The system shall respond to API requests within 2 seconds when Ollama is offline (fallback path).	Timed POST /api/chat with Ollama stopped , response returned in <0.1s. The fallback uses in-memory keyword matching with no network calls.
NFR-3.2	Constraint	The system shall respond to API requests within the Ollama timeout (120s default) when the LLM is available.	Timed POST /api/chat with Ollama running Llama 3.2 , typical response in 2–5s. Agent logs confirmed via ACT ollama replied in X.Xs. Timeout is configurable via OLLAMA_TIMEOUT env var.
NFR-3.3	Constraint	The AI consultant shall attempt at most one corrective re-prompt before falling back, to bound total latency.	MAX_REPROMPTS = 1 is enforced in agent.py. Test test_chat_persists_both_turns confirms the fallback activates after failure. Agent logs show at most 2 ACT calls per turn.
NFR-3.4	Quality	The application shall run inside a Docker container deployable via docker-compose up.	docker build -t student-3 ./student-3 succeeds in CI (GitHub Actions). docker-compose up starts the container on port 5003. Verified by accessing http://localhost:5003 in the browser.
NFR-3.5	Quality	The frontend shall use the shared OmniTech CSS design system tokens for consistent styling across the integrated application.	The shared palette tokens (Slate Blue #81A6C6, Warm Cream #F3E3D0, Sandstone #D2C4B4) are vendored into frontend/static/style.css. Visual inspection confirms matching colours across the home page and the consultant UI.
NFR-3.6	Quality	The SQLite database schema shall self-apply on every connection (CREATE TABLE IF NOT EXISTS), so the service never raises a "no such table" error.	Deleted consultant.db and restarted the service , tables and seed data were recreated automatically. get_db() runs the full schema DDL on every connection.
NFR-3.7	Quality	All API endpoints shall return appropriate HTTP status codes (200, 201, 400, 404) and JSON error messages.	Automated tests verify: 201 for creation, 400 for validation errors (missing title, invalid sender, unknown product IDs), 404 for non-existent resources. Error handler returns {"error": "..."} JSON for /api/ routes.
NFR-3.8	Quality	Automated tests shall run in isolation using temporary databases, with no dependency on a running Ollama instance.	Each test uses tmp_path for an isolated SQLite file. agent.act is monkeypatched , no network calls to Ollama. All 44 tests pass in CI (Ubuntu, no Ollama installed).
NFR-3.9	Quality	The CI/CD pipeline shall validate syntax, run tests, and build the Docker image on every push or PR to main.	.github/workflows/student-3.yml triggers on push/PR to main and student3-AIConsultation. Pipeline runs py_compile (4 files), pytest (44 tests), and docker build. All steps green on the merged PR #3.

NFR-3.10	Quality	The frontend shall be responsive and usable on screens 360px and wider.	Tested in browser at 360px, 768px, and 1440px widths. The sidebar collapses on narrow screens (<760px), the composer and chat bubbles scale with the viewport, and no horizontal overflow occurs.
NFR-4.1	Performance	Cart actions shall complete within 1 second.	Timed viewing, quantity change and fulfilment toggle during local testing; all pytest cases pass well inside their 3-second timeout.
NFR-4.2	Performance	An AI consultation shall return within 15 seconds and stay short enough to read at a glance.	test_ai_helper_custom_question passes with a 15-second timeout, code review of the 160-token cap and the two-point prompt.
NFR-4.3	Reliability	A failure in the AI or database service shall not crash the microservice, and the error shall be shown in plain language.	Stopped Ollama and the database service in turn; the page displayed "AI Helper Offline" and "Failed to load past orders" while both processes stayed running.
NFR-4.4	Reliability	Calls between microservices shall time out rather than hang the page.	Code review of the 2\-second timeout, confirmed an error message appeared within about two seconds when the database service was stopped.
NFR-4.5	Reliability	Seed data shall be reproducible on every rebuild.	Re-ran `init_db.py` several times; `GET /orders` returned the same 10 records each time, confirmed by `test_get_all_orders_count`
NFR-4.6	Portability	All three microservices shall run in Docker containers started from the shared Docker Compose file.	‘docker-compose up --build` started the Student 4 container; both ports 5004 and 5014 responded.
NFR-4.7	Portability	Services shall be reachable from other containers and from the host on the team's assigned ports.	Reproduced the connection-refused failure with a loopback bind, then confirmed teammates could open the page after switching to `0.0.0.0
NFR-4.8	Portability	Service addresses and the AI model shall be configurable by environment variable.	Code review; ran the container with `OLLAMA_HOST` and `OLLAMA_MODEL` supplied by Docker Compose and confirmed the AI helper connected.
NFR-4.9	Maintainability	Frontend, backend and database code shall be kept in separate folders and communicate only over HTTP.	Repository structure review of `student-4/`,no cross-imports between the three services.
NFR-4.10	Maintainability	All Python dependencies shall be pinned to exact versions.	Review of `requirements.txt`, every package specifies a version.
NFR-4.11	Testability	The feature shall have an automated test suite covering the database, cart and AI endpoints, run automatically in CI.	10 pytest cases in `tests/test_orders_api.py` pass locally the `student-4-ci.yml` GitHub Actions run completed green on push and pull request.
NFR-4.12	Usability	The interface shall match the team's shared CSS theme and be localised for Australian users.	Visual comparison with the other services from the unified home page; manual inspection of AUD prices, 10% GST, metric units and 240 V references.
NFR-4.13	Security	Customer and order data shall not leave the local environment for AI processing.	Code review the only configured AI endpoint is the local Ollama container, and the model is the approved open-source `llama3.2`
NFR-5.1	Testability	The warranty support feature must provide automated tests for tickets CRUD.	student-5/tests/test_tickts.py has pytest and functions for automated testing as well as the student-5.yml on Github Actions
NFR-5.2	Portibility	The application must run inside a docker container and be built and	Building docker and docker compose up successfully builds and runs the container on port 5005 which is

		deployable using docker compose up.	verified as typing http://localhost:5003 in the browser takes you to the warranty support page.
NFR-5.3	Quality	The pull requests must run CI/CD pipeline to validate syntax, tests and build the docker image.	When a pull request is made, .githubs/workflows/student-5.yml runs and the syntax, pytest (5 tests) and docker build all pass.
NFR-5.4	Usability	The user shall be able to identify the ticket's status without needing to access the database.	http://localhost:5005/tickets displays all tickets in a table with the status of every ticket in the status column.
NFR-5.5	Maintainability	The database shall initialise every time the application starts without a manual setup.	init_db() is called every time the application starts and inside init_db(), each table has CREATE TABLE IF NOT EXISTS and will always delete all the data inside the 3 database tables and initialise with 10 seed data for each table.
NFR-5.6	Portability	The application must run inside a docker container on port 5005 and operate without any dependencies on external database servers.	Docker build and compose up is successful, accessing http://localhost:5005 and entering the create ticket or view tickets page displays all orders or tickets without starting an external database server. All database operations function without an external database.

7. Feature Plan

Student 1

Feature Name: Product Catalog and Specifications

Description:
The Product Catalog and Specifications feature allows customers to browse consumer electronics products, compare their prices and technical details, and search or filter the catalogue based on their needs. Administrators can manage categories, products, and product specifications through CRUD functions. The feature also uses Ollama to generate a short product summary and review it against the stored product data before showing it to the user.

Microservice Breakdown

Layer	Technology	Responsibility
Frontend	HTML, CSS, HTMX	Displays the product catalogue, filters, product details, administrator pages, and AI summary results. HTMX updates catalogue and admin areas without a full page reload.
Backend	Python, Flask	Handles catalogue pages, CRUD operations, product filtering, input validation, JSON API endpoints, AI requests, and the agentic workflow.
Database	SQLite	Stores categories, products, and product specifications. The database is seeded when it is empty.
AI Integration	Ollama with **llama3.2**	Generates product summaries and performs the Plan → Act → Observe → Adapt review process.

For Release 0, these components are organised into separate folders inside **student-1/** but run together in the **student-1** Docker container.

Database Design
Categories

Column	Type	Description
id	INTEGER PRIMARY KEY	Unique category ID
name	TEXT	Category name
description	TEXT	Short category description

Products

Column	Type	Description
id	INTEGER PRIMARY KEY	Unique product ID
name	TEXT	Product name
brand	TEXT	Product brand
category_id	INTEGER FOREIGN KEY	Links the product to a category from **Categories** table
price	REAL	Product price
stock	INTEGER	Available stock quantity
description	TEXT	Product description
image_url	TEXT	URL for the product image

Product Specifications

Column	Type	Description
id	INTEGER PRIMARY KEY	Unique specification ID
product_id	INTEGER FOREIGN KEY	Links the specification to a product
spec_name	TEXT	Specification name, such as capacity or energy rating,...
spec_value	TEXT	Value of the specification

API Endpoints

Method	Endpoint	Description
GET	/health	Returns the service health status
GET, POST	/api/categories	Lists categories or creates a new category
GET, PUT, DELETE	/api/categories/<id>	Retrieves, updates, or deletes a category
GET, POST	/api/products	Lists filtered products or creates a new product
GET, PUT, DELETE	/api/products/<id>	Retrieves, updates, or deletes a product
GET, POST	/api/products/<id>/specifications	Lists or creates product specifications
GET, PUT, DELETE	/api/products/<id>/specifications/<spec_id>	Retrieves, updates, or deletes a specification
POST	/api/products/<id>/ai-summary	Generates a product summary using Ollama
POST	/api/products/<id>/ai-review	Runs the full Plan → Act → Observe → Adapt review workflow

Release 0 Scope

The Product Catalog and Specifications feature delivers:

- A seeded SQLite catalogue containing AT LEAST 10 categories, 12 products, and product specifications.
- Product browsing, searching, and filtering by category, brand, price range, search text, and specification keyword.
- Product detail pages displaying price, stock, descriptions, and technical specifications.
- Administrator CRUD operations for categories, products, and product specifications.
- Input validation for required fields, valid categories, and non-negative price and stock values.
- JSON API endpoints for catalogue data and future service integration.
- AI-generated product summaries using the shared Ollama service and **llama3.2**.
- A separate agentic workflow Plan → Act → Observe → Adapt that checks generated claims against stored product data and retries once if necessary.
- Pytest coverage and a GitHub Actions workflow for testing, Docker build, health checking, and seed-data validation.

Student 2

Feature Name
Customer Profiles & Preferences

Description
It is an account management layer that gives the ability to manage user accounts, ecosystem preferences and their shipping profiles. It has CRUD controls for user accounts allowing for the creation, reading, updating and deletion of them. It integrates the agentic AI loop using a local Ollama LLM which uses the Plan-Act-Abserve-Adapt loop to analyse user preferences and generate preference tags for users.

Microservice Breakdown

Layer	Technology	Responsibility
Frontend	HTML, CSS, HTMX, JavaScript	Inline profile editing, customer lookup, preference tag removal, validation and dynamic HTMX partial rendering
Backend	Python, Flask	REST API for customer profiles, ecosystem preferences, and preference tags, HTMX partial endpoints, agentic AI loop
Database	SQLite	Persistent storage for customer record, ecosystem preferences, and preference tags
AI Integration	Ollama	Local LLM analyses customer profiles and ecosystem preferences to generate preference tags

Database Design

Customers

Column	Type	Description
id	INTEGER PRIMARY KEY	ID that uniquely identifies each customer
first_name	TEXT	Customers first name
last_name	TEXT	Customers last name
email	TEXT	Customers email
phone	TEXT	Customers phone number
shipping_street	TEXT	Customers street address line
shipping_city	TEXT	Customers city
shipping_state	TEXT	Customers state
shipping_postcode	TEXT	Customers postcode
created_at	TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	Record update timestamp

Preferences

Column	Type	Description
id	INTEGER PRIMARY KEY	ID that uniquely identifies each preference
customer_id	INTEGER FOREIGN KEY	References customer_id
ecosystem	TEXT	Hardware ecosystem (Apple, Google, Android, Windows)
budget_tier	TEXT	Spending classifications (budget, mid-tier, premium)
notes	TEXT	Description of the profile used for AI
created_at	TIMESTAMP	Record creation timestamp

Preference Tags

Column	Type	Description
id	INTEGER PRIMARY KEY	ID that uniquely identifies each preference tag
customer_id	INTEGER FOREIGN KEY	References customer_id
tag_name	TEXT	Name of preference tag
source	TEXT	Tag origin
created_at	TIMESTAMP	Record creation timestamp

API Endpoints

Method	Endpoint	Description
GET	/	Main customer dashboard
GET	/customers	Fetch all customer records
POST	/customers	Create new customer profile
GET	/customers/by-id	Find customer by ID
PUT	/customers/<id>	Update customer details
DELETE	/customers/<id>	Delete customer profile
GET	/customers/<id>/edit	Edit customer table
POST	/generate-ai-suggestions	Start agentic AI loop
POST	/apply-tags	Add preference tags to customer profile
DELETE	/customers/<cid>/tags/<tid>	Remove preference tag from customer profile

Release 0 Scope
Release 0 for customer profiles and preferences delivers

- Core account management with CRUD functionality for customer profiles and input validation for them
- Inline editing and HTMX dynamic UI that changes real-time
- Customer lookup and tag management
- Agentic AI loop for tagging customers
- Testing using pytest that tests profile CRUD, tag management and AI output
- Docker containerisation on port 5002, which linked to the shared group n port 8080

Student 3

Feature Name: AI Product Consultant

Description: It is a conversational assistant that will assist you based on the needs and specifications of the product you are trying to find based on the available products in the catalog of the store. It uses the local Ollama LLM, the customer can speak in natural language and specify their needs like (cheap, certain amount of RAM, use case) and through the AI workflow the best match product will be recommended in a natural human response.

Microservice Breakdown:

Layer	Technology	Responsibility
Frontend	HTML, CSS, JavaScript, HTMX	Chat interface, session management, dashboard, dynamic partial rendering
Backend/API	Python, Flask	REST API for sessions, messages, recommendations; agentic AI orchestration; HTMX partial endpoints
Database	SQLite	Persistent storage for consultation sessions, chat logs, and saved recommendations
AI Integration	Ollama (Llama 3.2)	Local LLM inference for generating product recommendations

Database Design:

Consultation Sessions - Stores all the individual consultation sessions:

Column	Type	Description
id	INTEGER PK	Auto-incrementing session ID
user_id	TEXT	Owner of the session (default: 'guest')
title	TEXT	Editable session title
created_at	TIMESTAMP	Creation timestamp
updated_at	TIMESTAMP	Last activity timestamp

Chat Logs - Stores the individual messages for each chat log within a session:

Column	Type	Description
id	INTEGER PK	Auto-incrementing message ID
session_id	INTEGER FK	References ConsultationSessions.id (CASCADE delete)
sender	TEXT	user' or 'ai' (CHECK constraint)
message_text	TEXT	The message content
created_at	TIMESTAMP	Message timestamp

Saved Recommendations - Stores the recommendations from the AI, that the user has decided to save:

Column	Type	Description
id	INTEGER PK	Auto-incrementing recommendation ID
session_id	INTEGER FK	References ConsultationSessions.id (CASCADE delete)
product_ids	TEXT	Comma-separated catalog product IDs
summary	TEXT	AI-generated summary of why the products fit
tags	TEXT	Comma-separated user preference tags
created_at	TIMESTAMP	Recommendation timestamp

API Endpoints:

Method	Endpoint	Description
GET	/	Main chat interface (index page)
GET	/dashboard	Dashboard with past sessions and recommendations
GET	/api/sessions	List all sessions (optional ?user_id= filter)
POST	/api/sessions	Create a new session
GET	/api/sessions/<id>	Get session bundle (session + messages + recommendations)
PUT	/api/sessions/<id>	Update session title
DELETE	/api/sessions/<id>	Delete session (cascades)
GET	/api/sessions/<id>/messages	List messages for a session
POST	/api/sessions/<id>/messages	Add a message to a session
DELETE	/api/sessions/<id>/messages	Clear all messages for a session
DELETE	/api/messages/<id>	Delete a single message
GET	/api/sessions/<id>/recommendations	List recommendations for a session

POST	/api/recommendations	Create a recommendation
PUT	/api/recommendations/<id>	Update recommendation tags/summary
POST	/api/recommendations/<id>/tags	Add a single preference tag
DELETE	/api/recommendations/<id>	Delete a recommendation
POST	/api/chat	Agentic AI consultation turn (JSON or HTMX form)
GET	/partials/session-list	HTMX partial: session sidebar list
GET	/partials/chat/<id>	HTMX partial: chat panel for a session
GET	/partials/recommendations/<id>	HTMX partial: recommendations dock

Release 0 Scope:
The Ai Product Consultant delivers:

- Full CRUD operations across all three database tables via REST API.
- Agentic AI consultation with Plan → Act → Observe → Adapt workflow.
- HTMX-powered chat interface with session management.
- Dashboard for viewing past consultations and saved recommendations.
- Deterministic fallback when Ollama is offline.
- 44 automated tests covering all endpoints, agent logic, and catalog functions.
- GitHub Actions CI/CD pipeline.
- Docker containerisation integrated into the shared docker-compose.yml.

Student 4

Feature Name: Cart & Order Processing
Description: It is a shopping and order layer that gives the customer the ability to manage their shopping cart, choose how an order is fulfilled, place the order and review their order history. It has CRUD controls for cart items and orders, allowing for the creation, reading, updating and deletion of them. It integrates the agentic AI loop using a local Ollama LLM which uses the Plan\-Act\-Observe\-Adapt loop to analyse the appliances in the cart and generate Australian power, clearance and installation guidance before the customer buys.

Microservice Breakdown:

Layer	**Technology**	**Responsibility**
Frontend	HTML, CSS, JavaScript, HTMX	Cart item rows with stock indicators, quantity controls, delivery/pick\-up toggle, live order summary, checkout modal, order history modal, AI helper panel and dynamic HTMX partial rendering
Backend/API	Python, Flask	REST API for the cart, fulfilment mode, pricing and checkout, HTMX partial endpoints, order history retrieval, agentic AI loop
Database	SQLite	| Persistent storage for orders, order line items and active shopping carts
AI Integration	Ollama (Llama 3.2)	Local LLM analyses the cart contents and generates appliance safety and installation advice using Australian metric units and 240 V electrical standards

Database Design:

Column	Type	Description
line_item_id	INTEGER PK	ID that uniquely identifies each line item
order_id	INTEGER PK	References orders.order_id
cart_id	INTEGER PK	References carts.cart_id
product_id	INTEGER	Product identifier from the catalogue
product_name	TEXT	Name of the appliance
category	TEXT	Product category (Kitchen, Laundry, Cleaning, Climate Control)
quantity	INTEGER	Number of units of the product
unit_price	REAL	Price per unit in AUD

Carts

Column	Type	Description
cart_id	INTEGER PK	ID that uniquely identifies each shopping cart
user_id	INTEGER FK	Customer the cart belongs to
created_at	TIMESTAMP	Record creation timestamp

API Endpoints

Backend / API microservice (port 5004)

Method	Endpoint	Description
GET	/	Main cart and checkout page
GET	/api/cart/view	Fetch the cart items and order summary
GET	/api/cart/modify	Increase, decrease or remove a cart item
POST	/api/cart/fulfillment	Switch between standard delivery and store pick\-up
POST	/api/cart/checkout	Place the order and return the receipt
POST	/api/orders/history	Fetch past orders for the order history modal
GET	/api/orders/history	Fetch past orders for the order history modal
POST	/api/orders/ai-validate-cart	Start the agentic AI loop for appliance safety advice

Database microservice (port 5014)

Method	Endpoint	Description
GET	/	Service health check
GET	/orders	Fetch all order records
GET	/orders/<id>	Fetch a single order with its line items
GET	/carts/<id>	Fetch a single cart with its items
POST	/orders	Create a new order record (to be implemented)
PUT	/orders/<id>	Update the fulfilment status of an order (to be implemented)
DELETE	/orders/<id>	` | Delete an order record (to be implemented)

Release 0 Scope

Release 0 for cart, checkout and order management delivers:

- Core cart management with CRUD functionality for cart items and orders, and validation for empty carts and out\-of\-range items
- Inline quantity editing and HTMX dynamic UI that updates the cart and order summary in real time
- Delivery and store pick\-up fulfilment options with live pricing, including a delivery fee and 10% GST
- Checkout with a confirmation receipt, and order history lookup from the database microservice
- SQLite database seeded with 10 orders, 13 line items and 2 active carts
- Agentic AI loop that audits the cart for Australian power draw and installation clearance, and answers free\-text customer questions
- Testing using pytest that covers the database contract, cart operations, order history and AI output, run automatically by GitHub Actions
- Docker containerisation on ports 5004 (cart page and API) and 5014 (database API), linked to the shared group home page on port 8080

Student 5

Feature: Warranty Support
Description: It is a support system with CRUD operations allowing users to create, update and delete warranty tickets. It uses local Ollama LLM (qwen) to evaluate a ticket's claim against OmniTech's policy rules to make a decision with reasoning.

Microservice Breakdown

Layer	Technology	Responsibility
Frontend	HTML, CSS, HTMX, JavaScript	Displaying tickets, orders and the AI's decision and reasoning.
Backend	Python, Flask	REST API for tickets, orders, AI evaluation.
Database	SQLite	Persistent storage for tickets, products and orders.
AI Integration	Ollama (Qwen2.5:0.5b)	Local LLM evaluates ticket claim against policy rules.

Database Design
Tickets

Column	Type	Description
ticket_id	INTEGER PRIMARY KEY	Auto incrementing ID for the ticket
customer_id	INTEGER	ID of the customer creating the ticket
product_id	INTEGER	ID of the product the ticket is for
product_category	TEXT	The category the product falls under
ticket_claim	TEXT	The claim the AI will evaluate against policy rules
ticket_status	TEXT	Indicates whether the ticket has been evaluated or not
ticket_created_date	TIMESTAMP	Date the ticket was created
ai_decision	TEXT	The AI's decision is stored here
ai_reasoning	TEXT	The AI's reasoning for the decision is stored here
ai_reviewed_date	TIMESTAMP	The date the AI reviewed the ticket

Products

Column	Type	Description
product_id	INTEGER PRIMARY KEY	Auto incrementing ID for products
product_name	TEXT	Name of the product
product_category	TEXT	What category the product falls under
product_price	REAL	The price of the product
product_description	TEXT	What the product is and for
product_warranty_years	INTEGER	How many years warranty the product is under

Orders

Column	Type	Description
order_id	INTEGER PRIMARY KEY	Auto incrementing ID for orders
customer_id	INTEGER	ID of the customer making an order
product_id	INTEGER	ID of the product ordered
order_price	REAL	Total cost of the order
order_status	TEXT	Indicates if the order is completed or not
order_date	TIMESTAMP	Date the order was made

API Endpoints

Method	Endpoint	Description
GET	/	Main interface for warranty support
GET	/tickets	Tickets page that fetches all the tickets
GET	/create-ticket	Fetches all orders to create a ticket on
GET	/ai-evaluation	Loads a card to display the AI's response
POST	/api/tickets/<id>/evaluate	AI evaluates a ticket by its ID
POST	/tickets/<id>/update	Updates ticket after AI evaluates it
POST	/tickets/create	Creates a ticket with a claim
DELETE	/tickets/<id>/delete	Deletes a ticket

Release 0 Scope
The AI warranty support delivers:

- CRUD operations for the tickets database table via REST API
- Table UI that allows users to view tickets and select to evaluate.
- 5 automated tests for the tickets database.
- GitHub Actions CI/CD pipeline.
- Docker containerisation integrated into the shared docker-compose.yml

8. Risk Management Plan

Student 1

ID	Risk	Likelihood	Impact	Mitigation	Contingency
R-1.1	Ollama is unavailable or responds too slowly during testing or the showcase.	Medium	High	The application catches AI connection errors and uses a configured timeout.	Show the catalogue CRUD functions and explain that the AI endpoint returns an unavailable message when Ollama cannot be reached.
R-1.2	The AI summary includes product claims that are not supported by the stored specifications.	Medium	High	The Plan → Act → Observe → Adapt workflow checks generated claims against stored product facts and retries once.	The summary is withheld and warnings are returned if the second attempt still fails.
R-1.3	Invalid administrator input creates incorrect catalogue data.	Medium	Medium	The backend validates required fields, category IDs, and non-negative prices and stock values.	Show the validation message and correct the input before resubmitting.
R-1.4	Changes to categories or products affect related specifications.	Low	Medium	Foreign-key relationships and application checks prevent invalid category and product references.	Restore the affected seeded data or recreate the record through the administrator page.
R-1.5	The Student 1 Docker container does not start correctly in the integrated application.	Low	High	GitHub Actions compiles the code, runs tests, builds the image, and checks the **/health** endpoint.	Review container logs, fix the configuration, and rebuild the Student 1 image.
R-1.6	Changes made by other team members cause integration issues with the shared home page or Docker Compose file.	Medium	Medium	Keep Student 1 code inside its own folder and test it through the assigned port before merging changes.	Revert the affected integration change and use the last working Student 1 configuration while resolving the issue.

Student 2

ID	Risk	Likelihood	Impact	Mitigation	Contingency
R-2.1	Ollama is offline or unavailable or takes too long to respond	Medium	High	Set a request timeout to catch connection errors and timeouts	Execute to fallback_plan(), so users still receive valid preference tags
R-2.2	Ollama returns non-JSON text or recommends tag names outside the system categories	High	Medium	Use format="json" in the Ollama API body and run the response through the validation function observe()	Re-prompt Ollama with error feedback detailing the missing rules
R-2.3	Deleting a customer leaves orphaned preferences or AI tags in child table	Low	High	Enable foreign key constraints in SQLite and define DELETE CASCADE on child tables	Use automated database cleanup scripts in pytest to show zero orphaned rows are left after customer deletion tests
R-2.4	Users submit invalid data during inline table edits	Medium	Low	Perform server-side field validation in Flask before executing SQLite UPDATE queries	Send an HX-Trigger: showValidationAlert response header to get the validation error message
R-2.5	Python standard output buffering prevents agent execution steps from appearing in terminal logs	Medium	Low	Configure logging and set PYTHONUNBUFFERED =1 in docker-compose.yml	Buffer flushing (sys.stdout.flush()) inside agent execution to ensure logs reach docker logs -f student-2

Student 3

ID	Risk	Likelihood	Impact	Mitigation	Contingency
R1	Ollama is unreachable or too slow during the showcase demo	Medium	High	I tested the whole system with Ollama stopped and built a deterministic catalog fallback into the agent loop.	The fallback activates automatically , the consultant still recommends real products using keyword matching, so the demo keeps working.
R2	The LLM makes up product IDs that don't exist in our catalog	High	High	The Observe stage checks every recommended ID against the catalog shortlist. Anything that doesn't match gets flagged and triggers a corrective re-prompt.	If the re-prompt still fails, the fallback takes over and returns only verified catalog products.
R3	The LLM returns something that isn't valid JSON	Medium	Medium	I set "format": "json" in the Ollama request. The Observe stage also tries a regex-based JSON extraction as a backup parser.	If JSON still can't be extracted after the re-prompt, it falls back to the keyword matcher.
R4	The database file gets deleted or corrupted	Low	High	get_db() runs CREATE TABLE IF NOT EXISTS on every connection, so the schema re-creates itself. Seed data gets inserted whenever the tables are empty.	Even if someone deletes consultant.db, the service starts up fine , everything gets recreated automatically.
R5	Merge conflicts when integrating into main	Medium	Medium	I work on my own branch (student3-AIConsultation) and merge through PRs with CI checks.	My service is self-contained in student-3/ so conflicts with other students' code are unlikely. I resolve any that come up manually.
R6	Docker container won't build or run	Low	High	CI runs docker build on every push, so I catch build issues early. The Dockerfile is pretty simple , just a Python base image, pip install, and copy.	If something breaks, I can debug locally with docker build -t student-3 ./student-3 and check docker logs.
R7	Changes to the shared CSS break my UI	Medium	Low	I copied the shared design tokens into my own style.css rather than linking to the shared file (since the Docker container doesn't include the shared/ folder).	If the shared CSS changes, I update my vendored copy manually.
R8	Tests become flaky because of Ollama dependency	Medium	Medium	All tests monkeypatch agent.act so they never actually call Ollama. Each test gets its own temporary database file.	The test suite is completely offline and deterministic , it runs the same in CI as it does on my laptop.

Student 4

ID	Risk	Likelihood	Impact	Mitigation	Contingency
R-4.1	Ollama is unavailable or responds too slowly during testing or the showcase.	Medium	High	The backend catches AI connection errors, uses a configured timeout, and caps generation so answers return quickly.	Demonstrate the cart, checkout and order history, and explain that the AI panel returns an "AI Helper Offline" message when Ollama cannot be reached.
R-4.2	The AI consultant gives power or clearance advice that is not supported by the appliance's real specifications.	Medium	High	The Plan → Act → Observe → Adapt workflow builds the prompt from the actual cart items and constrains the model to Australian metric units and 240 V standards at a low temperature.	Observe → Adapt workflow builds the prompt from the actual cart items and constrains the model to Australian metric units and 240 V standards at a low temperature. Present the advice as guidance only, and check the figures against the manufacturer's specification before relying on them.
R-4.3	Checkout does not persist the order because the database microservice has no write endpoint, so order history shows seeded data only.	High	High	Add `POST`, `PUT` and `DELETE` routes for orders and cover each one with a pytest case before the showcase.	Demonstrate order history from the seeded records and record the limitation under known issues in the technical report.
R-4.4	The database microservice is unreachable, so checkout and order history fail.	Low	High	Requests to the database service use a 2-second timeout and caught exceptions, so the page shows an error message instead of crashing.	Restart the container, confirm the health check responds on port 5014, and demonstrate the cart while the service recovers.
R-4.5	Cart contents are held in backend memory, so they are lost when the container restarts.	Medium	Medium	The demonstration cart is seeded in code and reloads on startup, so the feature always begins from a known state.	Restart the backend service to restore the demonstration cart before recording the video or presenting.
R-4.6	The Student 4 containers are unreachable from other containers or from teammates' machines.	Medium	High	Both Flask services bind to `0.0.0.0`, ports are published in the shared Docker Compose file, and GitHub Actions starts both services and runs the test suite on every push.	Rebuild with `docker-compose up --build`, then verify both ports 5004 and 5014 respond before continuing.

R-4.7	Changes made by other team members cause integration issues with the shared home page or Docker Compose file.	Medium	Medium	Keep Student 4 code inside its own folder and test it through the assigned ports before merging changes.	Revert the affected integration change and use the last working Student 4 configuration while resolving the issue.
R-4.8	The committed `orders.db` file causes merge conflicts, because the container also rebuilds it during the image build.	Medium	Low	Decide whether the database file is a seeded fixture or a build output, and align `.gitignore` with that decision.	Delete the conflicting file, re-run `init_db.py` to rebuild the seeded data, and commit the agreed version.

Student 5

ID	Risk	Likelihood	Impact	Mitigation	Contingency
R1	The LLM returns makes a decision that isn't either Approved or Rejected	Medium	Medium	I've added Decision: <Approved or Rejected> to the task prompt to make sure the AI makes one of those 2 decisions.	
R2	The database file is corrupted or deleted	Low	High	init_db() runs CREATE TABLE IF NOT EXIST whenever it is called which is everytime the application starts. init_db() also deletes all the data inside these tables and inserts 10 seed data for each table every time the database is initialised.	Because the database is initialised upon starting the application, the database is rebuilt the same way.
R3	The seed data is duplicated during the database's initialisation	Low	Medium	init_db() deletes all the data inside the existing or newly created database tables when the application starts, and new seed data is added.	Starting the application means the database will delete its existing data regardless and add seed data.
R4	Docker container fails to build or start	Low	High	CI/CD pipeline runs docker build and is triggered by a pull request. This allows the docker issue to be resolved right after merging.	Rebuild docker locally using docker compose up --build and ensuring the logs show the build was successful.
R5	Merge conflicts when merging student5-warranty branch into main	Medium	Medium	Implementation was done inside student-5 folder before creating a pull request to merge into main.	Because my implementation was done inside student-5 folder, there should be almost impossible. If there are conflicts, they are solved manually.

R6	Shared CSS styles.css file breaks UI or returns 404	Medium	Low	The shared design tokens inside styles.css in the shared folder copied into my style.css in my frontend folder and I do not reference the shared CSS file in my html pages to mitigate risks.	Changes to the shared CSS file are added to my feature's CSS file.
R7	Missing required dependencies	Low	High	Dockerfile contains COPY requirements.txt . RUN pip install --no-cache-dir -r requirements.txt so the docker container will be built with these dependencies installed.	Any missing or new dependencies will be added to the requirements.txt and the docker container will be rebuilt.

9. Repository Structure

The OmmiTech repository is organised into shared project resources, individual student feature folders, AI service placeholders, documentation, and CI/CD workflows. Each student works mainly inside their own **student-x** folder, which keeps feature code, frontend files, database logic, tests, and Docker configuration separate from the other students' work.

For Release 0, Docker Compose runs seven services: the shared home page, five student feature services, and the shared Ollama service. Most student features currently package their frontend, backend/API logic, and seeded SQLite data within one Docker container. Student 4 includes separate backend and database code but both processes are currently started inside the same Compose container. The planned MCP, RAG, and multi-agent folders are included as placeholders for later releases.

```
OmniTech/
├── .github/
│   └── workflows/
│       ├── student-1.yml
│       ├── student-2.yml
│       ├── student-3.yml
│       ├── student-4.yml
│       └── student-5.yml
├── ai-services/
│   ├── ai-mode/
│   ├── mcp-server/
│   ├── multi-agent-server/
│   └── rag-server/
├── docs/
│   ├── architecture/
│   ├── release-0/
│   ├── release-1/
│   ├── release-2/
│   └── reports/
├── scripts/
│   ├── deploy/
│   └── test/
├── shared/
│   ├── configuration/
│   └── frontend/
│       ├── css/
│       ├── assets/
│       ├── js/
│       ├── index.html
│       └── ui-reference.html
├── student-1/
│   ├── backend/
│   ├── database/
│   ├── data/
│   ├── frontend/
│   ├── tests/
│   ├── Dockerfile
│   └── requirements.txt
├── student-2/
│   ├── backend/
│   ├── database/
│   ├── frontend/
│   ├── tests/
│   ├── Dockerfile
│   └── requirements.txt
```

```
├── student-3/
│   ├── backend/
│   ├── database/
│   ├── frontend/
│   ├── tests/
│   ├── Dockerfile
│   └── requirements.txt
├── student-4/
│   ├── backend/
│   ├── database/
│   ├── frontend/
│   ├── templates/
│   ├── tests/
│   └── Dockerfile
├── student-5/
│   ├── backend/
│   ├── database/
│   ├── frontend/
│   ├── prompts/
│   ├── tests/
│   ├── Dockerfile
│   └── requirements.txt
├── .gitignore
├── README.md
└── docker-compose.yml
```

Folder/file	Purpose
.github/workflows/	Contains one GitHub Actions workflow for each student feature. These workflows perform code checks, tests, and Docker image builds where configured.
ai-services/	Contains the Ollama model-pull helper. The MCP, RAG, and multi-agent folders are placeholders for future releases.
docs/	Stores project documentation, release folders, architecture material, and testing reports.
scripts/	Reserved for future deployment and testing scripts.
shared/frontend/	Contains the unified home page, shared CSS source files, assets, and UI reference page.
student-1/	Product Catalogue and Specifications feature, including Flask code, SQLite catalogue data, templates, CSS, and tests.
student-2/	Customer Profiles and Preferences feature, including profile CRUD, preference-tag AI logic, database code, frontend files, and tests.
student-3/	AI Product Consultant feature, including consultation logic, local product data, chat templates, and tests.
student-4/	Cart and Order Processing feature, including frontend files, backend code, database code, initialisation scripts, and tests.

student-5/	Warranty, Returns and RMA feature, including ticket logic, policy prompt files, database code, frontend templates, and tests.
docker-compose.yml	Defines the seven Release 0 services, shared network, Ollama volume, ports, and service dependencies.
README.md	Provides setup instructions, service URLs, technology details, and the project development workflow.

10. Individual Software Architecture Diagrams

Student 1: Product Catalog and Specifications

This feature currently runs as one Docker Compose service on port 5001. Within the container, Flask serves the catalog and administration pages, handles the API and AI request, and accesses the seeded catalogue database. The service then sends AI requests to Ollama container using **llama3.2**.

Diagram

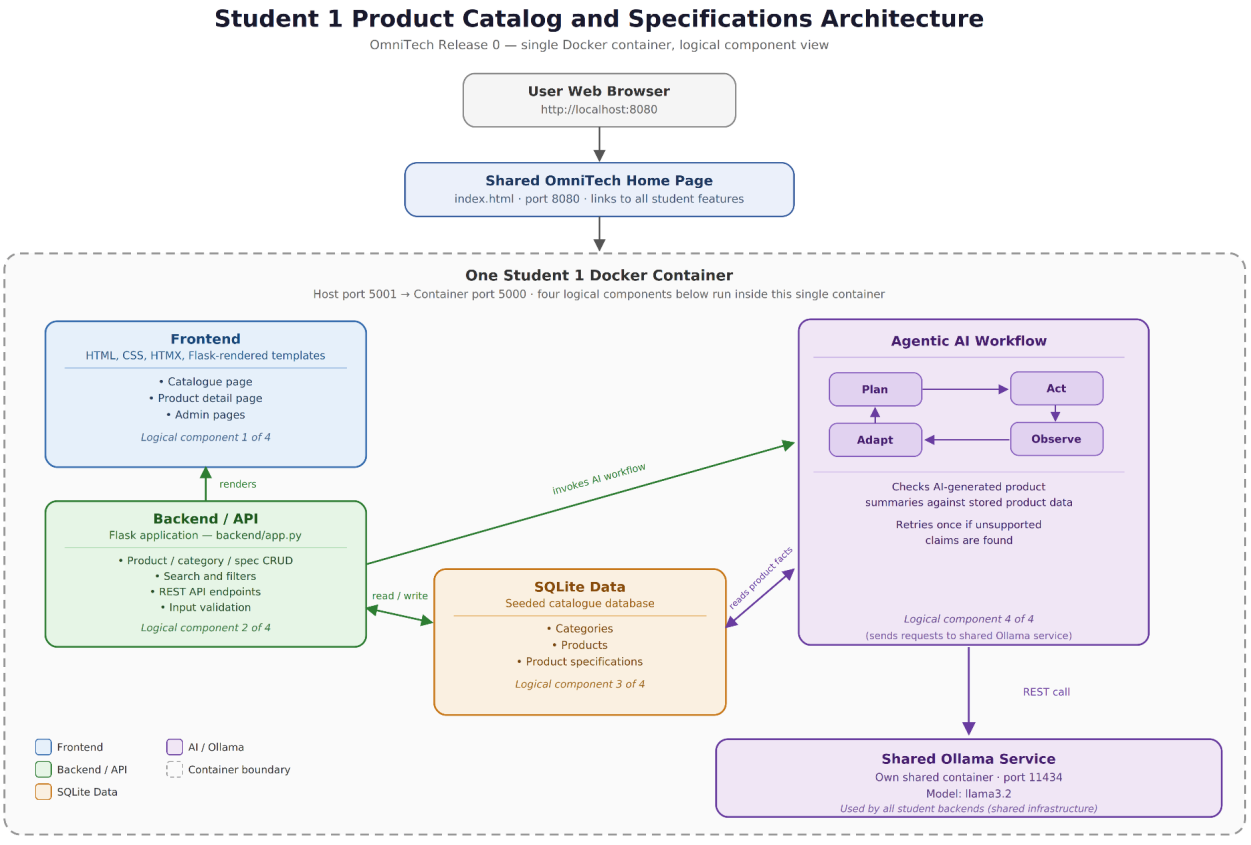


Figure 1. Student 1 Product Catalog and Specifications architecture.

Component Description

Components	Description
Frontend	Flask-rendered HTML templates, CSS, and HTMX provide the catalogue, product details, filters, and administrator pages.
Backend/API	**backend/app.py** handles page requests, product filtering, CRUD operations, input validation, REST API endpoints, and AI requests.
Database	SQLite stores categories, products, and product specifications. The data is seeded when the database is empty.
AI workflow	**backend/agent.py** and **backend/ai.py** connect to the shared Ollama service. The workflow generates and reviews product summaries.
Shared Ollama service	Runs **llama3.2** on port 11434 and is used by Student 1 for AI summary and review requests.

Data Flow

1. The user opens the shared OmniTech home page on port 8080 and selects the Product Catalog feature.
2. The browser sends the request to the Student 1 Flask service on port 5001.
3. The Flask backend retrieves product, category, and specification data from the local SQLite database.
4. The backend returns a rendered HTML page or HTMX fragment to update the relevant part of the interface.
5. For administrator actions, the backend validates the submitted data before creating, updating, or deleting the SQLite record.
6. For an AI review, the backend collects the selected product's stored facts and sends them to Ollama.
7. The Agentic AI workflow checks the generated summary against the stored data. If unsupported claims are found, it requests one corrected response; if the result is still unsupported, the summary is not displayed.

CRUD Coverage

Data entity	Create	Read	Update	Delete
Categories	Administrators can create product categories.	Users and administrators can view category data.	Administrators can edit category names and descriptions.	Administrators can delete a category when it is not used by products.
Products	Administrators can add products with a category, price, stock level, description, and image URL.	Users can browse, search, filter, and view product details.	Administrators can edit product information.	Administrators can remove products from the catalogue.
Product specifications	Administrators can add specifications to a product.	Users can view specifications on the product-detail page.	Administrators can edit a specification name or value.	Administrators can delete a specification.

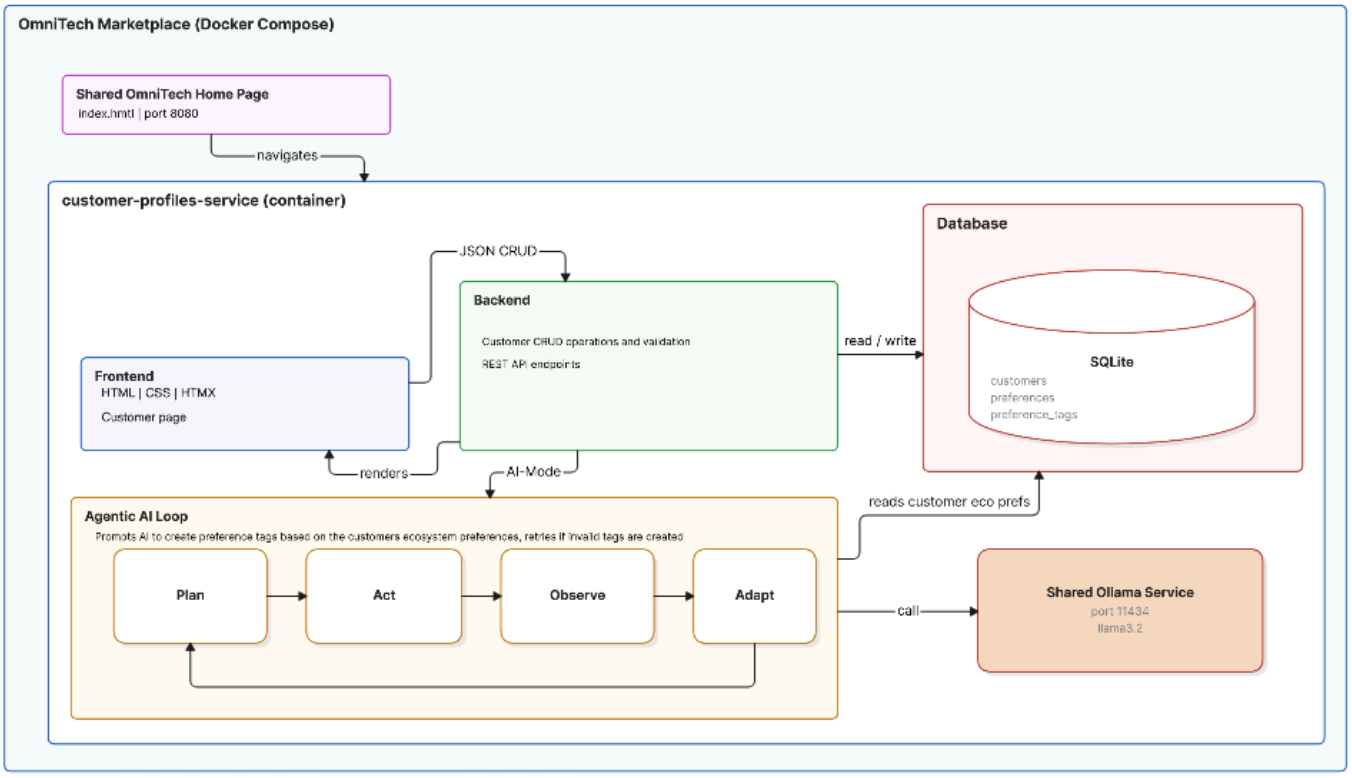
API and Data Ownership

This service owns the product-catalogue data, including categories, products, and product specifications. The current Release 0 feature accesses this SQLite data internally through its own backend. It also exposes REST API endpoints for categories, products, and specifications so that other features can use the catalogue data through the API in future releases, rather than accessing the SQLite file directly.

Student 2: Customer Profiles & Preferences

The microservice runs as one Docker Compose service on port 5002. In the container Flask serves the customer page, handles the API and AI requests, and accesses the seeded catalogue database. The backend sends AI requests to Ollama container

Diagram



Component Description

Components	Description
Frontend	HTML t, CSS, and HTMX create the customer page, which allows users to create, read, update, delete customers, as well as access to prompting the AI
Backend	The backend handles page requests, customer CRUD operations, input validation, REST API endpoints, and AI requests
Database	SQLite stores customer details, ecosystem preferences, and preference tags
Agentic AI Loop	Connects to the shared Ollama service generates preference tags based of the customers ecosystem preferences and spending
Shared Ollama Service	Runs llama3.2 on port 11434 and is used by Student 2 for generating customer preference tags

Data Flow

Data flow when using CRUD operations on customer

1. The user opens http://localhost:8080 and navigates to http://localhost:5002
2. The browser sends an initial request (GET /) to the Flask container on port 5002
3. The main.py entry route captures the request and queries SQLite for the initial seed data from Customers, Preferences and PreferenceTags for rendering
4. The user interacts with the UI at http://localhost:5002 and the browser issues a HTTP request
5. Flask processes the request parameters and executes an SQL statement against student_2.db, if required fields are missing it aborts the database write
6. The SQLite engine updates records in the Customers or Preferences table and returns the modified row payload to Flask
7. Flask renders a HTML partial and returns it to the browser

Dataflow when using the AI

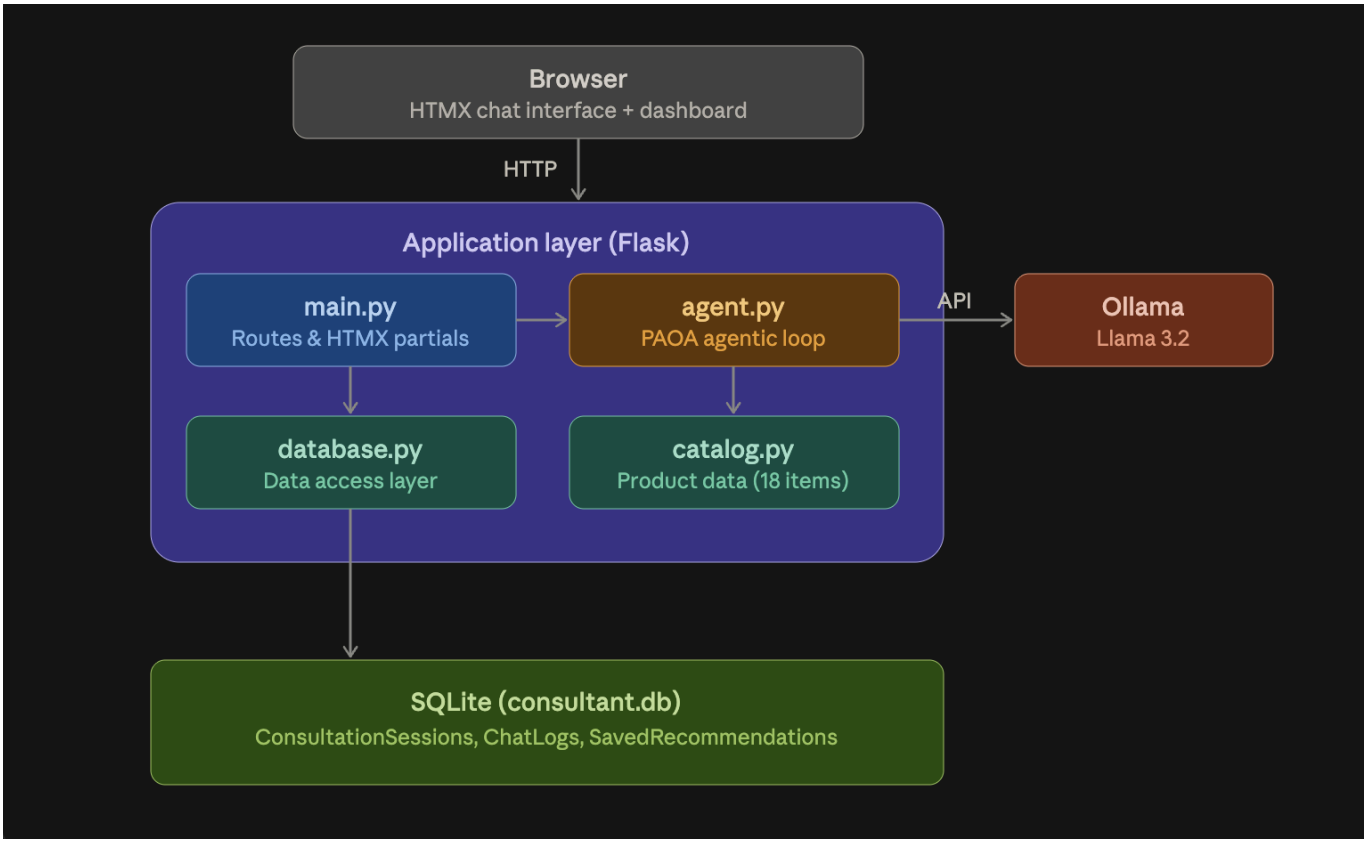
1. The user opens http://localhost:8080 and navigates to http://localhost:5002
2. The browser sends an initial request (GET /) to the Flask container on port 5002
3. The main.py entry route captures the request and queries SQLite for the initial seed data from Customers, Preferences and PreferenceTags for rendering
4. PLAN: The user generates suggestions for a customer and Flask queries Customers and Preferences tables in SQLite to retrieve customer notes, budget tier, and primary ecosystem
5. ACT: The backend constructs a structured prompt using SYSTEM_PROMPT and sends an HTTP POST request to Ollama on port 11434
6. OBSERVE: The backend intercepts raw LLM output text and parses the JSON body to validate candidate tags
7. ADAPT: If tags match system categories, the payload is marked as ready
8. The user selects preferred candidate tags and applies suggestions. Flask writes new records to PreferenceTags and HTMX refreshes the active tag list badge on the profile panel

CRUD Coverage

Data entity	Create	Read	Update	Delete
Customers	POST /customers registers a new customer profile record	GET /customers retrieves customer directory and profile details	PUT /customers/<id> updates personal info and shipping address	DELETE /customers/<id> deletes customer profile
Preferences	N/A	GET /customers/by-id reads preference context for profile views	N/A	DELETE /customers/<id> deletes preference record on profile
Preference Tags	POST /apply-tags persists AI generated tags	GET /customers/by-id displays active tags	Is updated when tags are reassigned and replaces existing tags on customer	DELETE /customers/<cid>/tags/<tid> deletes an individual preference tag

Student 3: AI Product Consultatant

The microservice that I worked on follows a three-tier architecture. It has a frontend, backend and lastly a data layer, with an external dependency on the shared Ollama service for AI interface.



Component Descriptions:

Component	File(s)	What it does
Frontend	frontend/templates/index.html, dashboard.html, partials/*.html	The user-facing pages, the chat interface, session sidebar, and dashboard. HTMX swaps in new content without reloading the page.
Backend API	backend/main.py	The Flask app that handles all the REST routes, serves the HTMX partials, and wires up the chat endpoint to the agent loop.
Agent Loop	backend/agent.py	Runs the Plan → Act → Observe → Adapt cycle. Talks to Ollama, validates the response, and falls back to keyword matching if needed.
Catalog	backend/catalog.py	Holds the 18-product catalog in memory. Used by Plan (to find relevant products) and Observe (to validate that recommended IDs are real).
Database	database/database.py	Manages the SQLite connection, defines the schema, and seeds demo data. The schema self-applies on every connection so it can recover from a missing DB file.
Ollama	External service	The shared LLM runtime that all student services connect to. My backend sends it a structured prompt and gets back a JSON response.

Data Flow:
This shows the flow of how the data flows whenever the user sends a message to the chatbot

1. The user types a message in the chat and hits Send (HTMX form POST to /api/chat).
2. main.py saves the user's message into the ChatLogs table.
3. main.py calls agent.run_consultation() with the message and the conversation history.
4. Plan: The agent searches the catalog for products that match what the user is asking about.
5. Act: It builds a prompt with the system instructions, the relevant product shortlist, the conversation history, and the user's message, then sends it to Ollama.
6. Observe: The agent validates the response, is it valid JSON? Do all the recommended product IDs exist in the shortlist?
7. Adapt: If validation fails, the agent builds a corrective prompt explaining what went wrong and tries again. If that also fails, it falls back to keyword matching.
8. main.py saves the AI reply into ChatLogs and, if products were recommended, creates a new SavedRecommendations row.
9. The response goes back to the browser, either as an HTMX fragment (chat bubbles + recommendation dock update) or plain JSON.

Student 4: Cart and Order Processing

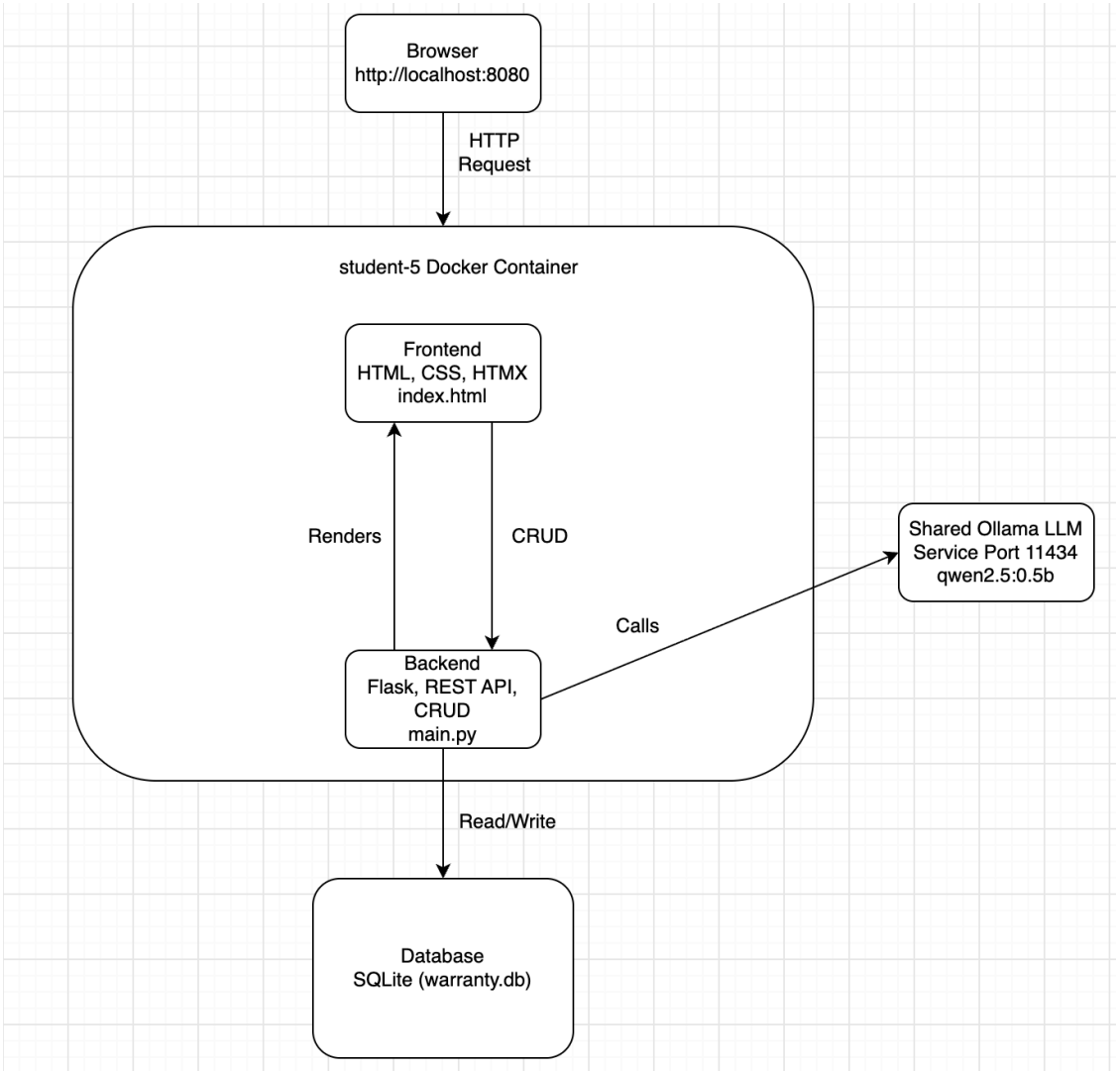
My feature handles the transactional side of OmniTech the cart, the checkout and the order records sitting underneath whatever the frontend shows. For Release 0 the focus was on plumbing rather than anything fancy: a working REST API, a database service the frontend can't reach into directly, and an AI pathway that runs the full chain instead of being tacked on the side. Later releases build retrieval and multi-agent reasoning on top of this, so I kept it simple on purpose.

Microservice	What it does
Frontend	HTML5, CSS3 and HTMX. Uses hx-get and hx-post with hx-target to refresh the cart rows, and an out-of-band swap to update the order summary at the same time, no full page reload and no separate JS framework. Sits behind the shared portal on port 8080 and follows the team CSS.
Backend / API	Flask 3.0.3 on port 5004. Holds the cart, validates each action, works out subtotal, delivery fee and GST, orchestrates checkout against the database service, and builds the prompts that go to Ollama. /api/orders/ai-validate-cart is the AI route; everything under /api/cart and /api/orders is standard REST.
Database	Flask and SQLite on port 5014 with init_db.py setting up the orders, order_line_items and carts schema and seeding 10 orders, 13 line items and 2 active carts. Only this service touches the .db file the backend reaches it over HTTP like any other client. It currently starts alongside the backend in the same container like splitting it out is a Release 1 clean-up.

CRUD coverage

Operation	How it works
Create	POST /orders on the database service writes a new order at checkout. Not yet implemented the backend already calls it, but the route is missing, so checkout falls back to a placeholder order number.
Read	GET endpoints list all orders, fetch one order by ID with its line items, and fetch an active cart with its items. A missing record returns 404.
Update	Cart quantities are updated through POST /api/cart/modify. PUT /orders/<id> for fulfilment status is not yet implemented.
Delete	Items are removed from the cart through the same modify endpoint. DELETE /orders/<id> is not yet implemented.

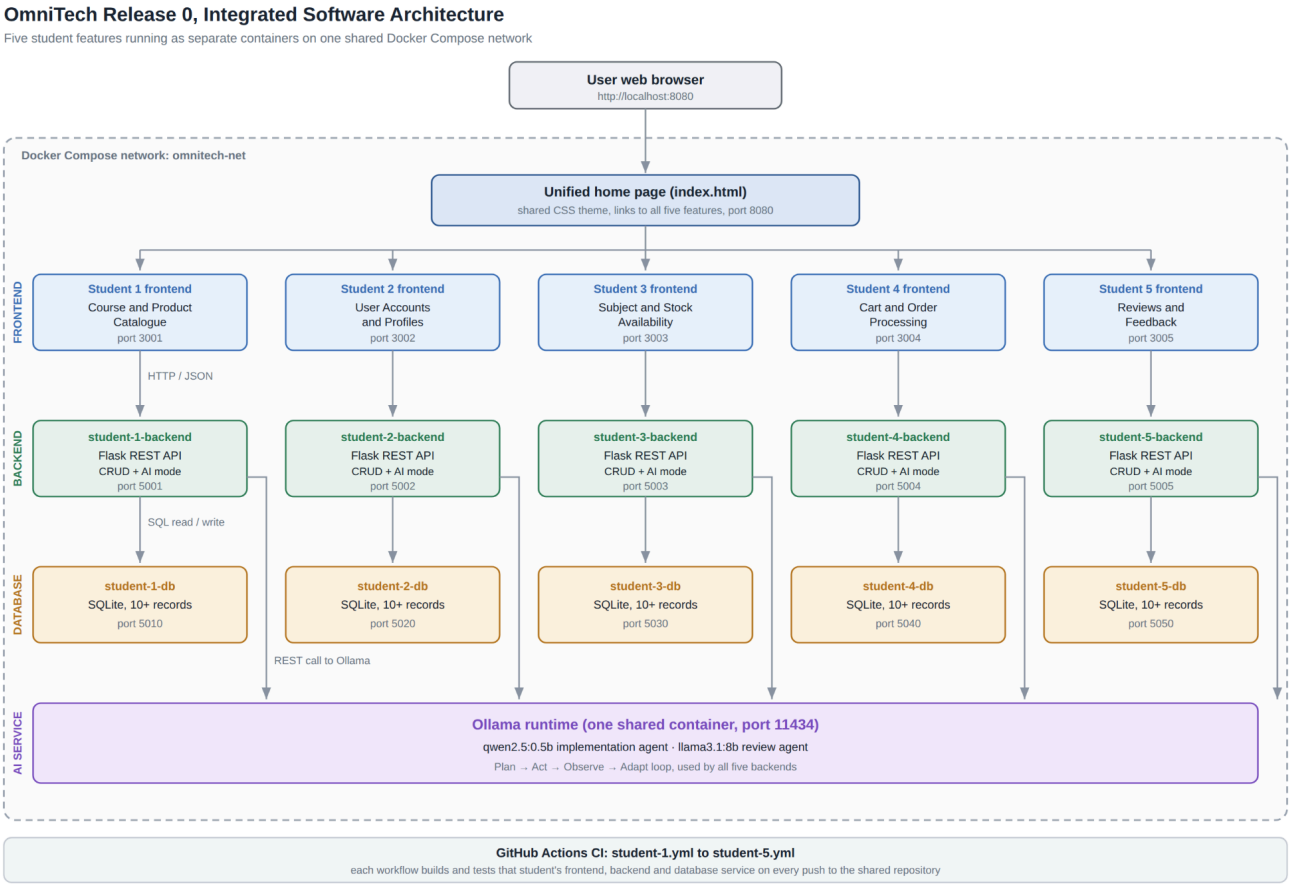
Student 5: Warranty



Components Description

Component	What it does
Frontend	HTML, CSS & HTMX creates a warranty support page where they can access the create tickets page to create a ticket from an order or navigate to the view tickets where they can get AI to evaluate a ticket.
Backend	Backend handles all the requests, CRUD operations, REST API endpoints and AI requests for evaluation.
Database	SQLite initialises the database with seed data in each table for tickets, products and orders. Is initialised every time the application starts.
Ollama	Runs qwen2.5 to evaluate a ticket claim using its product category and policy rules to generate a decision with reasonings.

11. Integrated Software Architecture Diagram



The application is one website made of five student features. For Release 0, each feature runs as a single **student-x** container that contains its frontend, backend/API, and seeded data. Along with the shared home page and Ollama service, the current Docker Compose configuration runs **seven** containers in total, and they all start from one **docker-compose.yml** file.

The user opens the home page on port 8080 and clicks through to any feature. Each feature is currently self-contained and manages its own data within its student container. This allows the team to develop and test features independently while keeping them accessible through one integrated entry point.

The one shared part is Ollama on port 11434. All five AI modes send their requests there using the **llama3.2** model. In later releases, each feature will be separated further into individual frontend, backend/API, and database containers, with REST API integration between services.

Containers

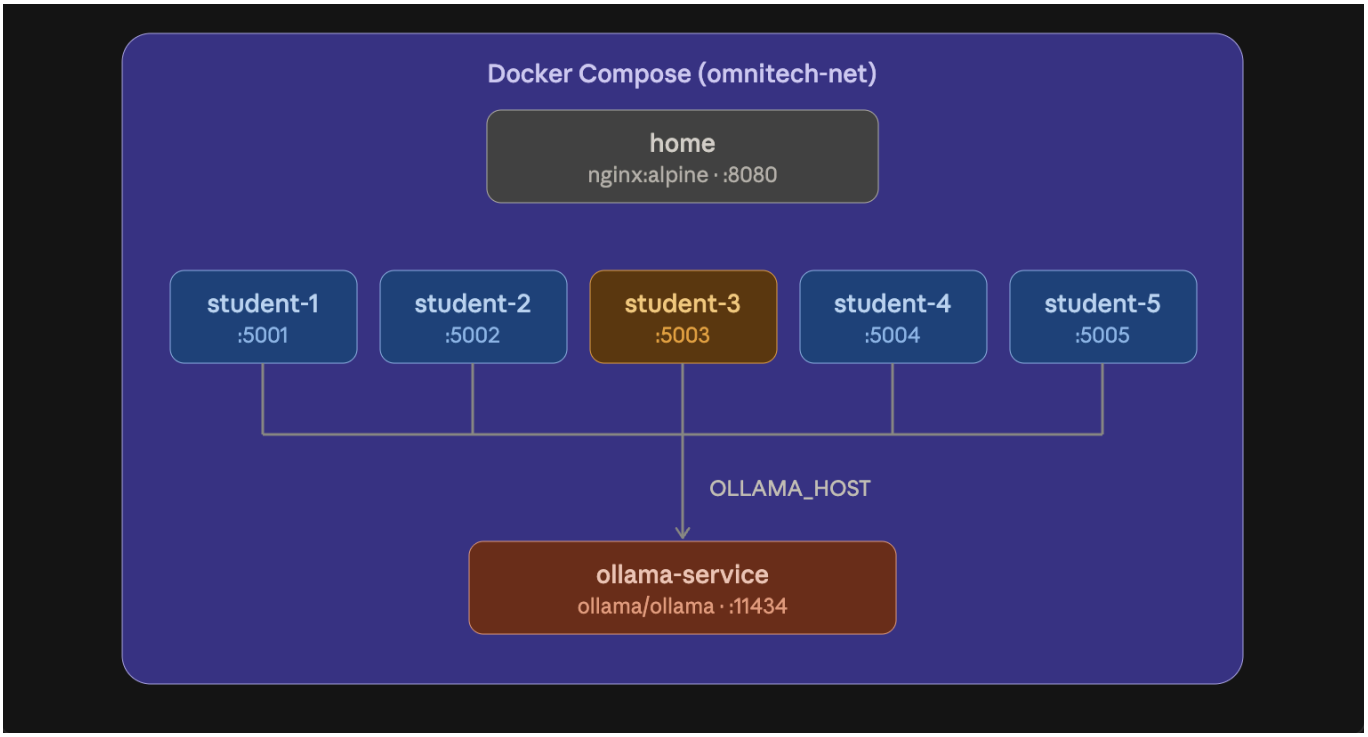
Layer	How many	Technology	Ports
Home page	1	Nginx, HTML5, CSS3	8080:80
Student feature service	5	Python, Flask, HTML, CSS, HTMX, SQLite	5001:5000, 5002:5000, 5003:5000, 5004:5004 and 5014:5014, 5005:5000
Shared AI service	5	Ollama, **llama3.2**	11434:11434

12. Docker Compose Architecture Diagram

12.1 Overview

Docker compose is managing all the docker containers needed to run the OmniTech marketplace, everything in there sits on a shared bridge (omnitech-net). This allows the student feature services to communicate with the shared Ollama service and each other.

12.2 Architecture



12.3 Service Details

Service	Image/Build	Host Port	Container Port	Purpose
home	nginx:alpine	8080	80	Serves the shared landing page with links to each student's service
student-1	Build from ./student-1	5001	5000	Product Catalog
student-2	Build from ./student-2	5002	5000	Customer Profiles
student-3	Build from ./student-3	5003	5000	AI Product Consultant
student-4	Build from ./student-4	5004	5000	Shopping Cart
student-5	Build from ./student-5	5005	5000	Student 5's service
ollama-service	ollama/ollama	11434	11434	Shared LLM runtime, all student containers connect to it via OLLAMA_HOST

12.4 Network Configuration

- As mentioned before, all the 7 containers are on the same bridge network, so they can reach each other by service name
- Each of the student containers are getting the environment variable, so the backend is able to know where it has to send the LLM requests
- The home container serves the shared landing page at http://localhost:8080

12.5 Volume Mounts

For development, each of the services mounts its source code into the container so the changes show up without having to rebuild the whole thing:

```
volumes:
  - ./student-3:/app
```

The home page mounts the shared frontend folder as read-only:

```
volumes:
  - ./shared/frontend:/usr/share/nginx/html:ro
```

13. Plan → Act → Observe → Adapt Workflow Diagram

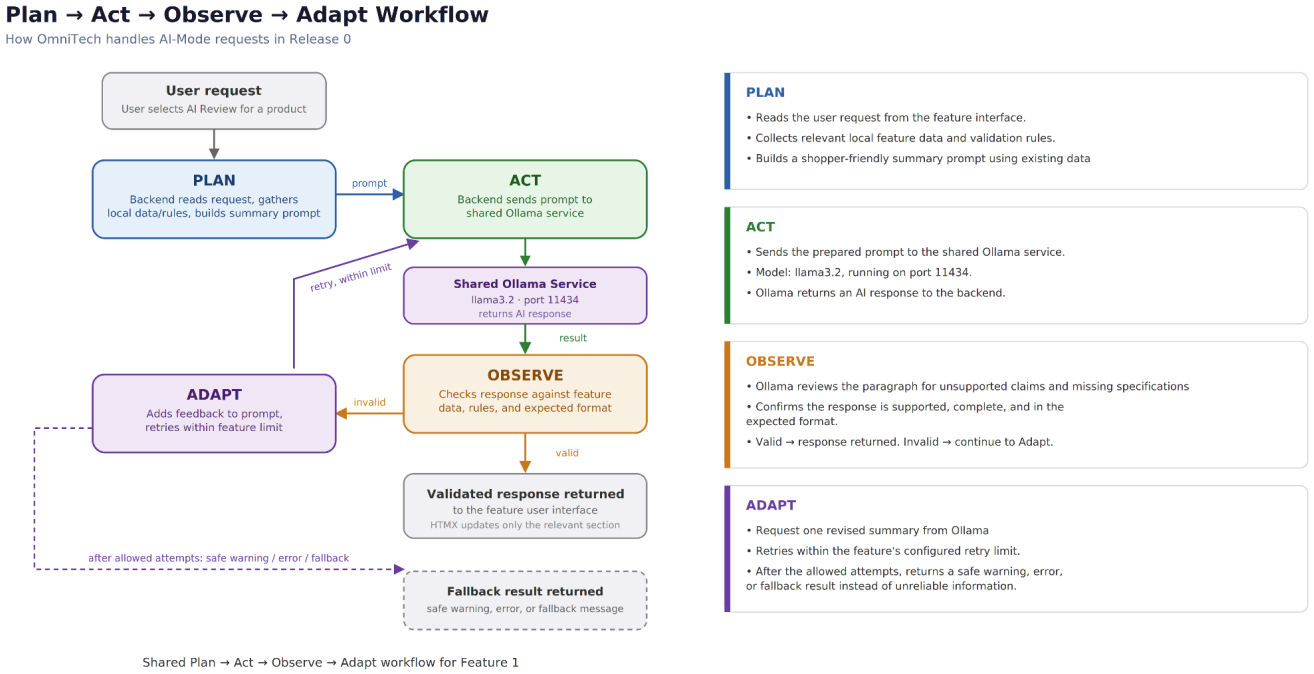
Student 1 - Product Catalog and Specifications

Workflow Overview

When a user selects **Summarise specifications** for a product, the system creates a shopper-friendly summary from the product's data. The generated summary is then checked before it is shown to the user. If it includes unsupported claims or numbers, the system sends a correction prompt and retries once. If the second result still fails validation, the summary is withheld and warnings are displayed.

The workflow is available through the AI review endpoint:
POST /api/products/<product_id>/ai-review

Workflow Diagram



Shared Plan → Act → Observe → Adapt workflow for Feature 1

Stage Details

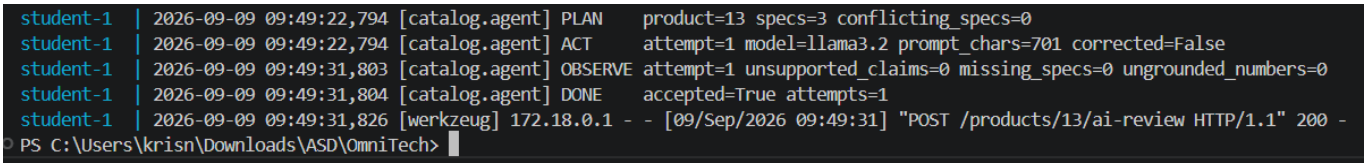
Stage	Student 1 implementation
Plan	The Flask backend loads the selected product, its category, and its stored technical specifications from SQLite. It also checks whether the same specification has conflicting stored values. The system then builds a prompt using only these product facts.
Act	**backend/agent.py** sends the prepared prompt to the shared Ollama service using **llama3.2**. Ollama returns a proposed shopper-friendly product-summary paragraph.
Observe	The system sends the generated paragraph for review and checks for unsupported claims and missing specifications. Python also compares numbers in the response against the stored product facts to identify unsupported figures.

Adapt	If unsupported claims or numbers are found, the backend adds the rejected information to a correction prompt and asks Ollama to rewrite the summary. Student 1 allows a maximum of two attempts in total.
Result	If the response passes validation, the summary is returned to the product interface. If it still fails after the second attempt, the summary is not shown and the user receives validation warnings instead.

Observability

The workflow stages in this feature are recorded through application logs. These logs can be viewed with:

```
docker compose logs student-1
```

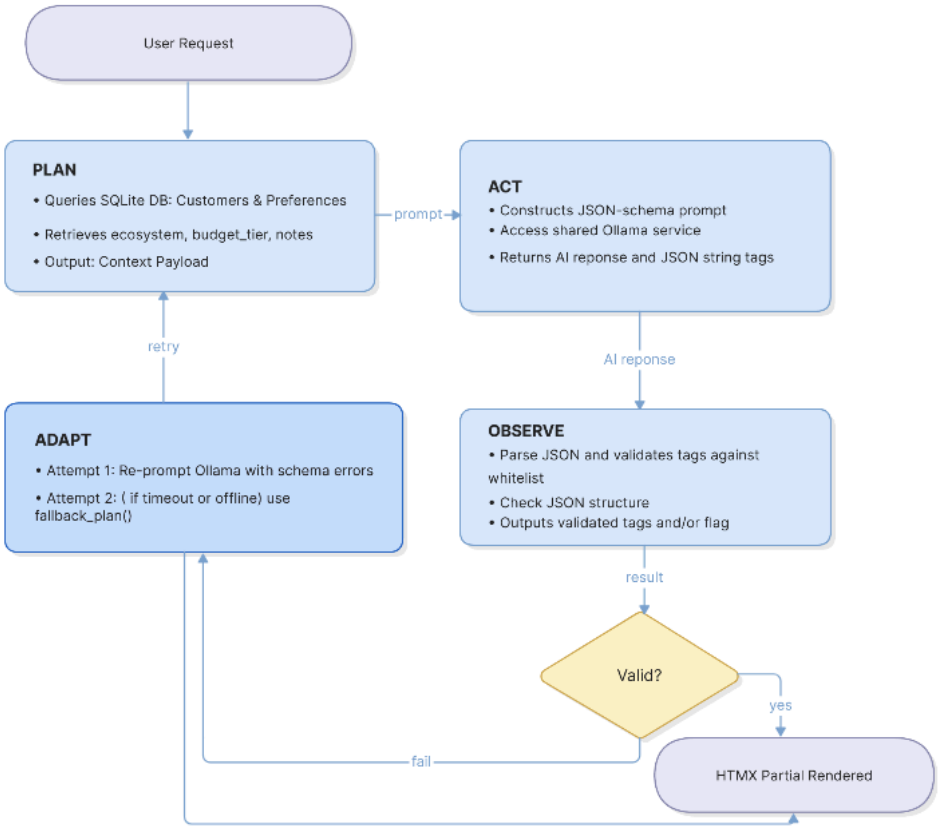


Student 2 - Customer Profiles & Preferences

The Student 2 microservice implements a agentic AI loop that translates customer notes into preference tags. It is executed when a user generate AI suggestion and the workflow follows four phases

- Plan: Queries SQLite to retrieve the customer's profile, active hardware ecosystem , budget tier, and descriptive notes
- Act: Formats the retrieved context into a JSON-constrained prompt and dispatches an HTTP request to the local Ollama LLM running on port 11434
- Observe: Intercepts LLM output text and parses the JSON payload, and validates all candidate tags against an allowed whitelist
- Adapt: If tags are valid, they are formatted for user confirmation. If validation fails, the agent re-prompts Ollama with explicit error feedback then routes execution to a fallback engine

Workflow Diagram



Stage Details

Stage	Module	What happens
Plan	backend/agent.py plan()	Executes SQLite queries on Customers and Preferences to make the context payload
Act	backend/agent.py act()	Constructs a JSON prompt using customer notes and system restrictions. Makes a HTTP POST request to the local Ollama LLM with a 120 second request timeout
Observe	backend/agent.py observe()	Gets response text from Ollama, and parses JSON. It looks at the candidate tags against the system whitelist. Passes if tags match allowed categories, fails if JSON parsing fails or invaild tags are found
Adapt	backend/agent.py adapt( )	If schema errors occur, re-prompts Ollama once, else formats validated tags into a candidate list and renders an HTMX partial
Fallback	backend/agent.py fallback_plan()	If Ollama is fails validation or times out redirects to fallback_plan()which is a keyword matcher that scans customer notes to return valid system tags without using the AI

Observability

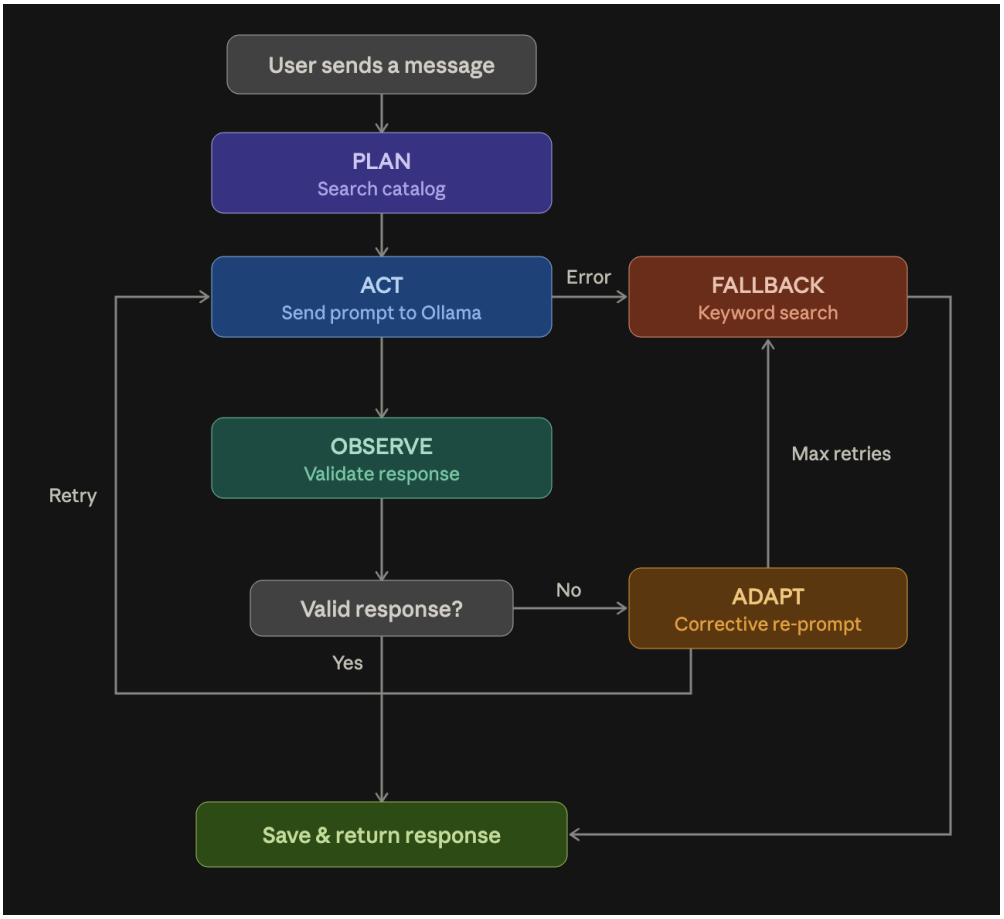
Seen by running docker compose logs student-2

```
student-2  | INFO:users.agent:PLAN Context retrieved for Customer #1 (Alex Mercer)
student-2  | INFO:users.agent:PLAN Preferences Summary: Ecosystem: Apple, Budget: premium, Notes: Focuses on high-end creative setups and 4K media work.
student-2  | INFO:users.agent:ACT Prompting Ollama (qwen2.5:0.5b) at http://ollama-service:11434...
student-2  | INFO:users.agent:OBSERVE Inspecting output payload against allowed system categories
student-2  | INFO:users.agent:OBSERVE Validation Passed Validated Tags: ['apple-ecosystem', 'budget-conscious', '4k-video-editing']
```

Student 3 - AI Product Consultant

Workflow Overview: Every time a user sends a message to the chatbot, it goes through a four-stage agentic loop in the backend/agent.py. The process looks something like, Plan pulled the relevant slice of the catalog using the whole conversation. Act sent a structured JSON prompt to Llama 3.2, which replied in about two seconds. Observe validated the output, it has to be valid JSON, and every product it recommends has to be one that was actually shown to the model. Adapts in the case something went wrong.

Workflow Diagram:



Stage Details:

Stage	Module	What happens
Plan	agent.plan()	Takes the last 5 user messages and searches the catalog. Products are scored by keyword matches in the name/category (weighted 3×) and specs/description (weighted 1×), plus a 5-point bonus if a category keyword is spotted (e.g., "laptop" maps to the Laptops category). Returns up to 6 products.

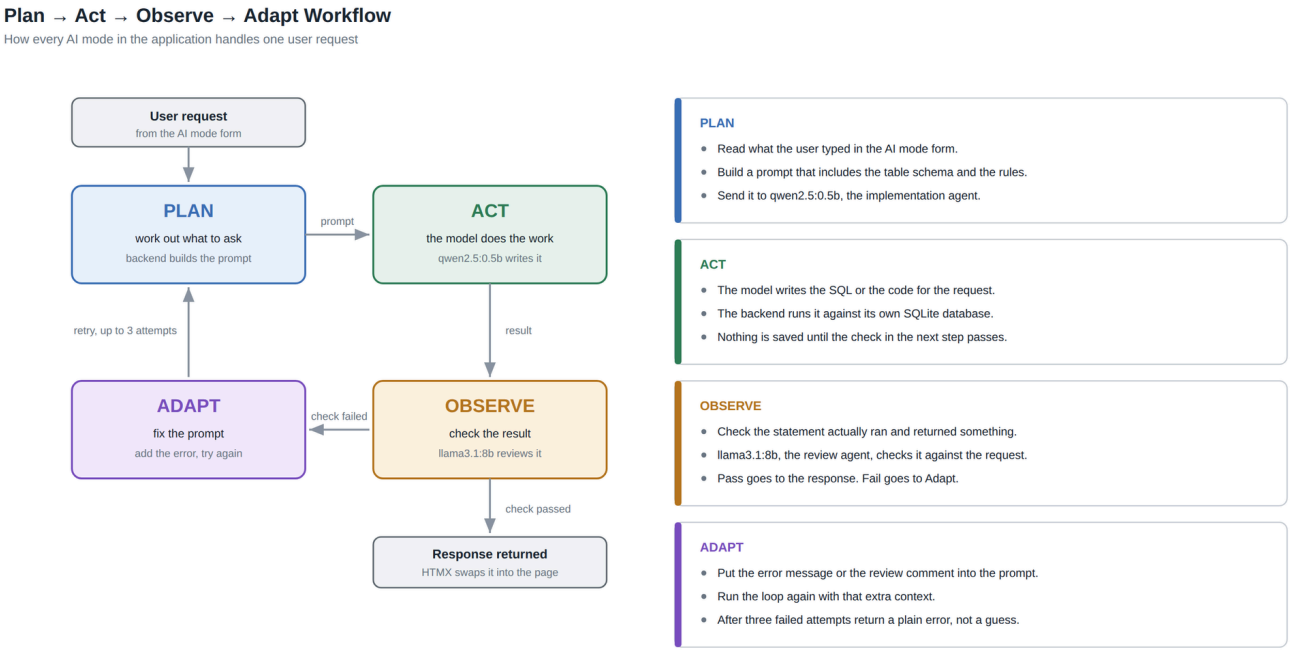
Act	agent.act()	Sends the prompt to Ollama at OLLAMA_HOST/api/generate with temperature=0.3 and format=json. The prompt tells the model to reply with a specific JSON shape: reply, recommended_product_ids, and summary. Timeout is 120 seconds.
Observe	agent.observe()	Tries to parse the response as JSON (falls back to regex extraction if the model wraps it in extra text). Then checks: is reply non-empty? Are all recommended IDs in the shortlist? Are there any placeholder IDs like <ID> or XXX? Products mentioned in the prose text are also picked up.
Adapt	agent.run_consultation()	If Observe finds problems, the agent builds a new prompt that includes the specific issues (e.g., "these product ids are not in the catalog: FAKE-001") and sends it to Ollama again. Only one re-prompt is allowed. If the second try still fails, it falls back.
Fallback	agent._fallback()	Does a catalog.search() for the top 2 keyword matches and returns a pre-written response with those products. This guarantees a valid answer with real product IDs even if the LLM is completely down.

Observability:
Every stage of the process gets logged onto the console which makes it easy to what happened during each of the requests, here is an example below:

```
student-3  | 192.168.65.1 - - [03/Sep/2026 17:30:26] "POST /api/sessions HTTP/1.1" 201 -
student-3  | 2026-09-03 17:30:26,980  agent  PLAN   query='I need a laptop for 4k video editing within $2500'
student-3  | 2026-09-03 17:30:26,981  agent  PLAN   5 relevant catalog items (recommend only these): ['LAP-001', 'LAP-002', 'LAP-003', 'CAM-001', 'S
TOR-001']
student-3  | 2026-09-03 17:30:26,982  agent  ACT    calling ollama model=llama3.2 (attempt 1)
student-3  | 2026-09-03 17:31:00,056  agent  ACT    ollama replied in 33.1s (388 chars)
student-3  | 2026-09-03 17:31:00,056  agent  OBSERVE ok=True ids=['LAP-001'] issues=[]
student-3  | 2026-09-03 17:31:00,056  agent  DONE   attempts=1 reprompts=0 ids=['LAP-001']
student-3  | 192.168.65.1 - - [03/Sep/2026 17:31:00] "POST /api/chat HTTP/1.1" 200 -
```

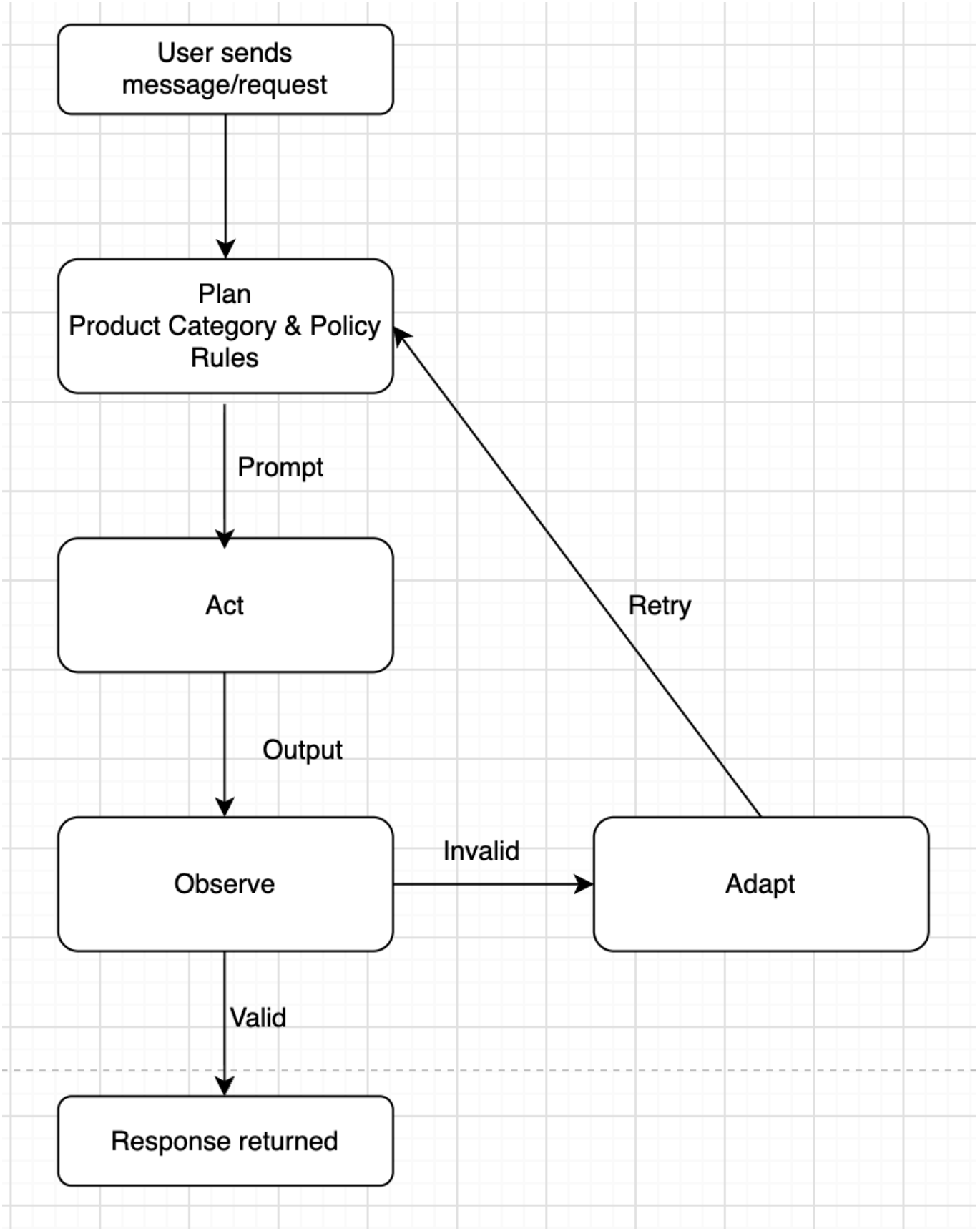
Student 4 - Cart and Order Processing

Workflow Diagram:



Student 5 - Warranty

Workflow Diagram:



Flow	Description
Plan	Reads the user's ticket claim and builds a prompt using the claim, product category and policy rules.

Act	Sends the prompt to Ollama qwen2.5 to generate a decision and reason.
Observe	Review the response to see if it is valid or not.
Adapt	Reattempts the prompt with error message and review comments to get a better response.

14. Description of Workflow Files (student-1.yml – student-5.yml)

Student 1

The **student-1.yml** workflow validates the Product Catalog and Specifications feature whenever changes are pushed to **main** or another branch. It also runs when a pull request to **main** changes files in **student-1/** or the Student 1 workflow file. The workflow contains 2 jobs:

Job	Description
validate	Run using Python. It installs the Student 1 dependencies, compiles the backend and database source files, and runs the complete pytest suite from the student-1 folder.
container	Runs only after the validation job passes. It builds the Student 1 Docker image, starts a temporary container, and checks that the /health endpoint responds successfully. It also calls the categories and products API endpoints to confirm that each contains at least ten seeded records. The temporary container is removed at the end of the workflow, even if a previous step fails.

Student 2

student-2.yml workflow file automates testing and validation of the Customer Profiles and preferences feature when triggered but a pull request to main or pushes to main or student2-customer branch. The workflow's runs the ubuntu-latest and Python 3.11 where it will install the dependencies inside student-2/requirments.txt. A syntax check is run checking the 2 backend files and database file before running pytest to automate the test. After passing the tests, it builds a docker image to ensure it is compatible with docker containerisation.

Student 3

student-3.yml workflow file automates testing and validation of the AI product consultation feature when a pull request is made to merge the student3-AIConsultantion branch into main. It is also triggered by pushes into the student3-AIConsultantion or main branch. The workflow runs on the ubuntu-latest environment and Python 3.11 before installing all dependencies inside student-3/requirments.txt. A syntax check on the 3 backend and 1 database file is run before running automated tests using pytest. After passing the automated tests, it ends on building a docker image to verify its compatibility.

Student 4

Job	Description
build-and-test	Runs on Python 3.11. It installs the Student 4 dependencies from student-4/requirements.txt, initialises the SQLite database by running init_db.py, then starts both microservices in the background the database service on port 5014 and the backend/API service on port 5004 and runs the complete pytest suite from the student-4/tests folder. The workflow is triggered on every push and pull request that touches files under student-4/ or the workflow file itself.

Student 5

student-5.yml workflow file automates testing and validation of the warranty support feature whenever a pull request is made to merge the student5-warranty branch into the main branch. It also runs whenever a change is pushed into the main or student5-warranty branch. The workflow runs on ubuntu-latest and Python 3.11. It installs all the dependencies inside student-5/requirments.txt before running a syntax check on student 5's 3 backend files and 2 database files to make sure there won't be any syntax problems during testing. The workflow initialises the SQLite database by running student-5/database/init_db.py on port 5006. It then starts the backend server and runs the automated test using pytest in student-5/database/test_tickets.py. After these tests, the docker image is built verifying the application can be built in a docker container.

15. Implementation Summary

Student 1:
The Product Catalog feature allows users to browse OmniTech's range of products. Users can search for products and filter them by specification keyword, brand, and price range. Each product displays its stored specifications, price, and stock availability. The AI review feature uses the selected product data from the SQLite database to generate and validate a shopper-friendly summary through the Plan → Act → Observe → Adapt workflow. The AI has a maximum of two attempts to produce an accepted summary. If the second attempt fails validation, the summary is not displayed and warnings are returned. The administrator can create, update, and delete products, categories, and product specifications through HTTP requests to the backend.

Student 2:
The customer profile and preference feature allows users to create a customer account by filling in their details. Users can view all customer accounts where they can update and delete the accounts. Users can find profiles by ID by entering a valid customer ID where their details are displayed including their preference tags. The AI preference tag recommender can find preference tags that suit the user's preference and the user can add it to their profile.

Student 3:
The AI product consultation feature allows customers to utilise the AI to find products based on their budget and needs. The AI will respond with some products that fit the needs and criteria of what the customer wants. The customer can create multiple chats and also rename them. They can view their chat history and delete chats.

Student 4:
The Orders and Cart feature allows customers to add products to their cart, increase or decrease item quantities, and remove items before checkout. Customers can view seeded order history and create an order through the cart checkout process. The AI assistant checks the current cart items and provides compatibility advice or suggests relevant accessories before checkout.

Student 5:
The Warranty and Returns feature allows customers to create a warranty return ticket using their order information. Users can view their existing tickets, update ticket details, and delete tickets when needed. The AI feature evaluates the warranty claim and provides a suggested decision with a reason based on the ticket's claim and policy rules.

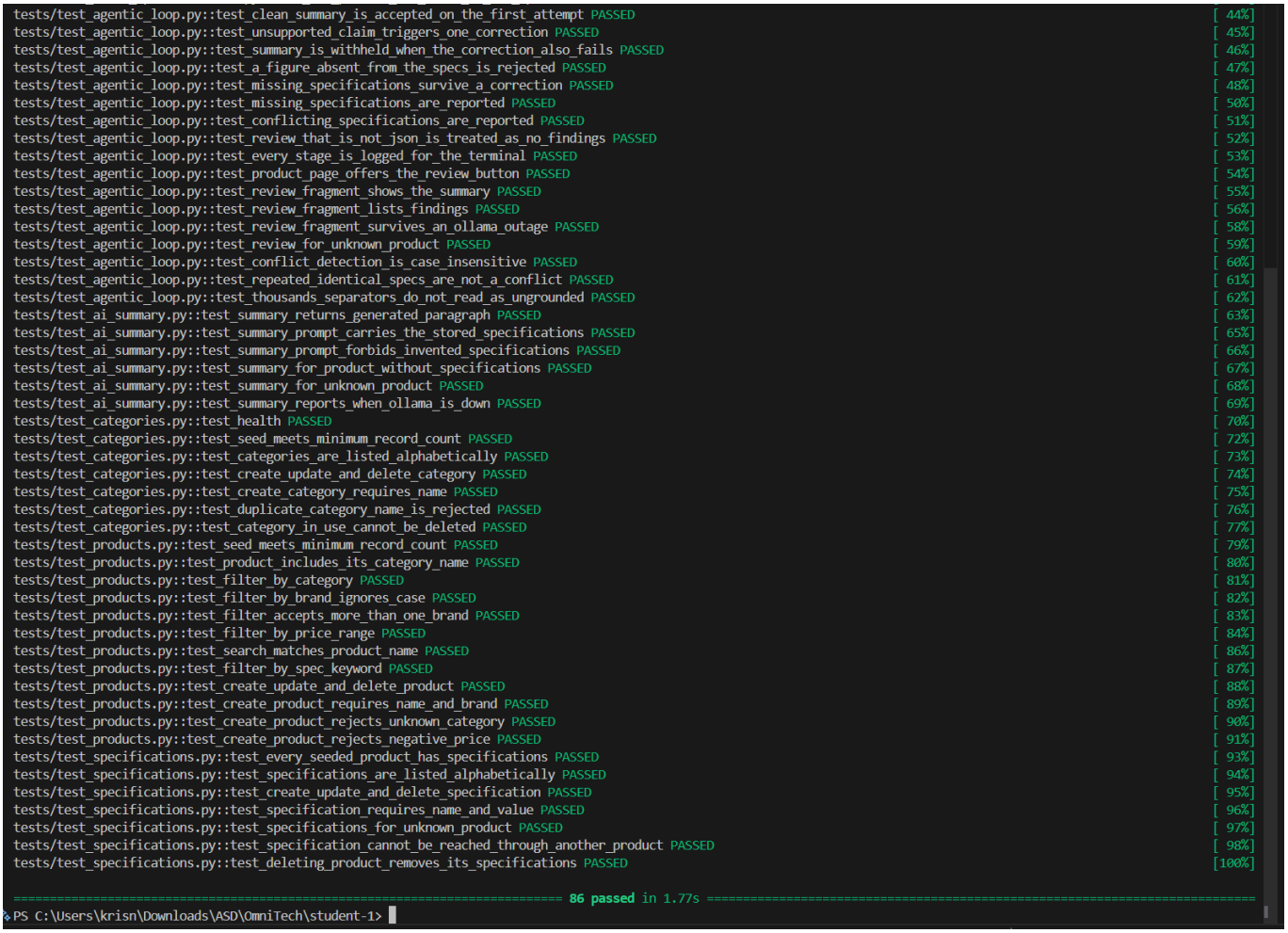
16. Local Testing Evidence

Student 1
Automated Test Results:
The Student 1 Product Catalog and Specifications feature was tested using the pytest suite in **student-1/tests/**. The test suite completed successfully with **86 tests passed in 1.77 seconds**.

Command to run the test:
cd student-1
python -m pytest tests/ -v

```
PS C:\Users\krisn\Downloads\ASD\OmniTech> cd student-1
PS C:\Users\krisn\Downloads\ASD\OmniTech\student-1> python -m pytest tests/ -v
============================= test session starts =============================
platform win32 -- Python 3.14.4, pytest-8.3.5, pluggy-1.6.0 -- C:\Users\krisn\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\krisn\Downloads\ASD\OmniTech\student-1
collected 86 items

tests/test_admin_categories.py::test_admin_page_lists_existing_categories PASSED [  1%]
tests/test_admin_categories.py::test_only_the_admin_page_is_badged_as_admin_view PASSED [  2%]
tests/test_admin_categories.py::test_adding_a_category_returns_the_updated_table PASSED [  3%]
tests/test_admin_categories.py::test_adding_a_category_without_a_name_reports_the_problem PASSED [  4%]
tests/test_admin_categories.py::test_adding_a_duplicate_category_reports_the_clash PASSED [  5%]
tests/test_admin_categories.py::test_edit_returns_the_row_as_a_form PASSED [  6%]
tests/test_admin_categories.py::test_editing_an_unknown_category_is_not_found PASSED [  8%]
tests/test_admin_categories.py::test_updating_a_category_saves_the_new_name PASSED [  9%]
tests/test_admin_categories.py::test_updating_a_category_without_a_name_keeps_the_row_in_edit_mode PASSED [ 10%]
tests/test_admin_categories.py::test_deleting_an_empty_category_removes_it PASSED [ 11%]
tests/test_admin_categories.py::test_deleting_a_category_with_products_is_refused PASSED [ 12%]
tests/test_admin_categories.py::test_deleting_an_unknown_category_is_not_found PASSED [ 13%]
tests/test_admin_products.py::test_admin_page_lists_existing_products PASSED [ 15%]
tests/test_admin_products.py::test_adding_a_product_returns_the_updated_table PASSED [ 16%]
tests/test_admin_products.py::test_adding_a_product_without_a_brand_reports_the_problem PASSED [ 17%]
tests/test_admin_products.py::test_adding_a_product_with_a_negative_price_is_refused PASSED [ 18%]
tests/test_admin_products.py::test_adding_a_product_to_an_unknown_category_is_refused PASSED [ 19%]
tests/test_admin_products.py::test_a_rejected_form_keeps_what_was_typed PASSED [ 20%]
tests/test_admin_products.py::test_edit_prefills_the_form_with_the_stored_values PASSED [ 22%]
tests/test_admin_products.py::test_editing_an_unknown_product_is_not_found PASSED [ 23%]
tests/test_admin_products.py::test_updating_a_product_saves_the_new_values PASSED [ 24%]
tests/test_admin_products.py::test_updating_a_product_with_a_bad_price_stays_in_edit_mode PASSED [ 25%]
tests/test_admin_products.py::test_deleting_a_product_removes_it_and_its_specifications PASSED [ 26%]
tests/test_admin_products.py::test_deleting_an_unknown_product_is_not_found PASSED [ 27%]
tests/test_admin_specifications.py::test_page_lists_the_stored_specifications PASSED [ 29%]
tests/test_admin_specifications.py::test_page_offers_the_ai_review_to_the_admin PASSED [ 30%]
tests/test_admin_specifications.py::test_page_for_an_unknown_product_is_not_found PASSED [ 31%]
tests/test_admin_specifications.py::test_adding_a_specification_returns_the_updated_table PASSED [ 32%]
tests/test_admin_specifications.py::test_adding_a_specification_without_a_value_reports_the_problem PASSED [ 33%]
tests/test_admin_specifications.py::test_a_rejected_form_keeps_what_was_typed PASSED [ 34%]
tests/test_admin_specifications.py::test_edit_returns_the_row_as_a_form PASSED [ 36%]
tests/test_admin_specifications.py::test_updating_a_specification_saves_the_new_value PASSED [ 37%]
tests/test_admin_specifications.py::test_updating_with_a_blank_name_keeps_the_row_in_edit_mode PASSED [ 38%]
tests/test_admin_specifications.py::test_deleting_a_specification_removes_it PASSED [ 39%]
tests/test_admin_specifications.py::test_a_specification_cannot_be_edited_through_another_product PASSED [ 40%]
tests/test_admin_specifications.py::test_cancel_returns_the_plain_table PASSED [ 41%]
tests/test_admin_specifications.py::test_the_product_row_links_to_its_specifications PASSED [ 43%]
tests/test_agentic_loop.py::test_clean_summary_is_accepted_on_the_first_attempt PASSED [ 44%]
tests/test_agentic_loop.py::test_unsupported_claim_triggers_one_correction PASSED [ 45%]
tests/test_agentic_loop.py::test_summary_is_withheld_when_the_correction_also_fails PASSED [ 46%]
tests/test_agentic_loop.py::test_a_figure_absent_from_the_specs_is_rejected PASSED [ 47%]
tests/test_agentic_loop.py::test_missing_specifications_survive_a_correction PASSED [ 48%]
tests/test_agentic_loop.py::test_missing_specifications_are_reported PASSED [ 50%]
tests/test_agentic_loop.py::test_conflicting_specifications_are_reported PASSED [ 51%]
tests/test_agentic_loop.py::test_review_that_is_not_json_is_treated_as_no_findings PASSED [ 52%]
tests/test_agentic_loop.py::test_every_stage_is_logged_for_the_terminal PASSED [ 53%]
tests/test_agentic_loop.py::test_product_page_offers_the_review_button PASSED [ 54%]
tests/test_agentic_loop.py::test_review_fragment_shows_the_summary PASSED [ 55%]
tests/test_agentic_loop.py::test_review_fragment_lists_findings PASSED [ 56%]
tests/test_agentic_loop.py::test_review_fragment_survives_an_ollama_outage PASSED [ 58%]
tests/test_agentic_loop.py::test_review_for_unknown_product PASSED [ 59%]
tests/test_agentic_loop.py::test_conflict_detection_is_case_insensitive PASSED [ 60%]
```

Test Evidence 1.1



Test Evidence 1.2

Test area	Evidence
Categories	Tests cover category creation, retrieval, updating, deletion, duplicate-name handling, and validation.
Products	Tests cover product CRUD operations, filters, search behaviour, invalid input, and product details.
Product specifications	Tests cover specification creation, retrieval, updating, deletion, and validation.
Administrator functions	Tests cover the administrator pages and HTMX CRUD responses for categories, products, and specifications.
AI summary	Tests cover AI summary requests, unavailable AI handling, and returned summary content.
Agentic AI workflow	Tests cover accepted summaries, retry behaviour, unsupported claims, missing specifications, and unsupported numeric values.

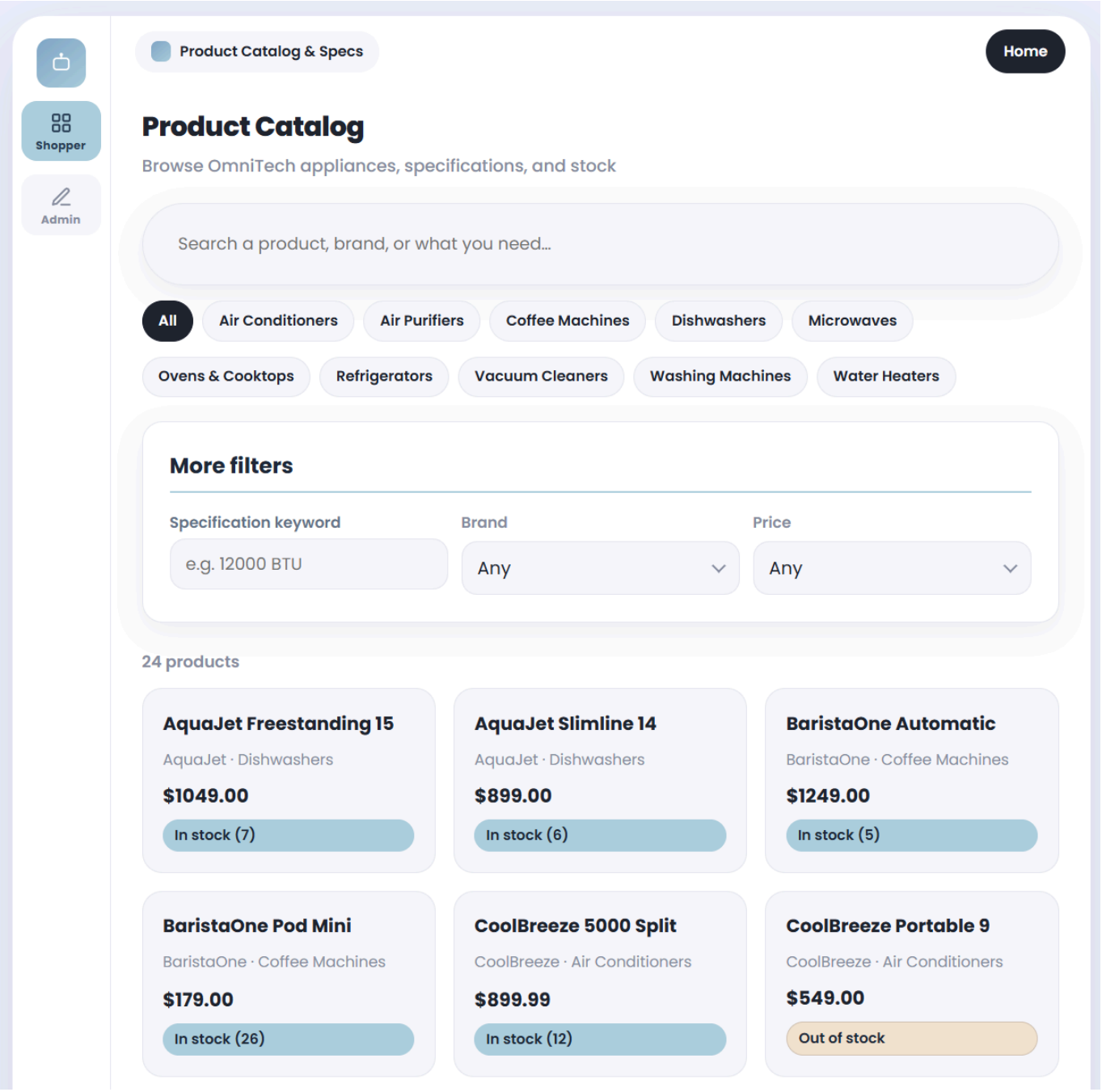
Test Isolation:

- Each test uses a fresh temporary SQLite database created with pytest's **tmp_path** fixture. The test fixture changes **DATABASE_PATH** to this temporary test.db, so the tests never change the real **student-1/data/catalog.db** file.

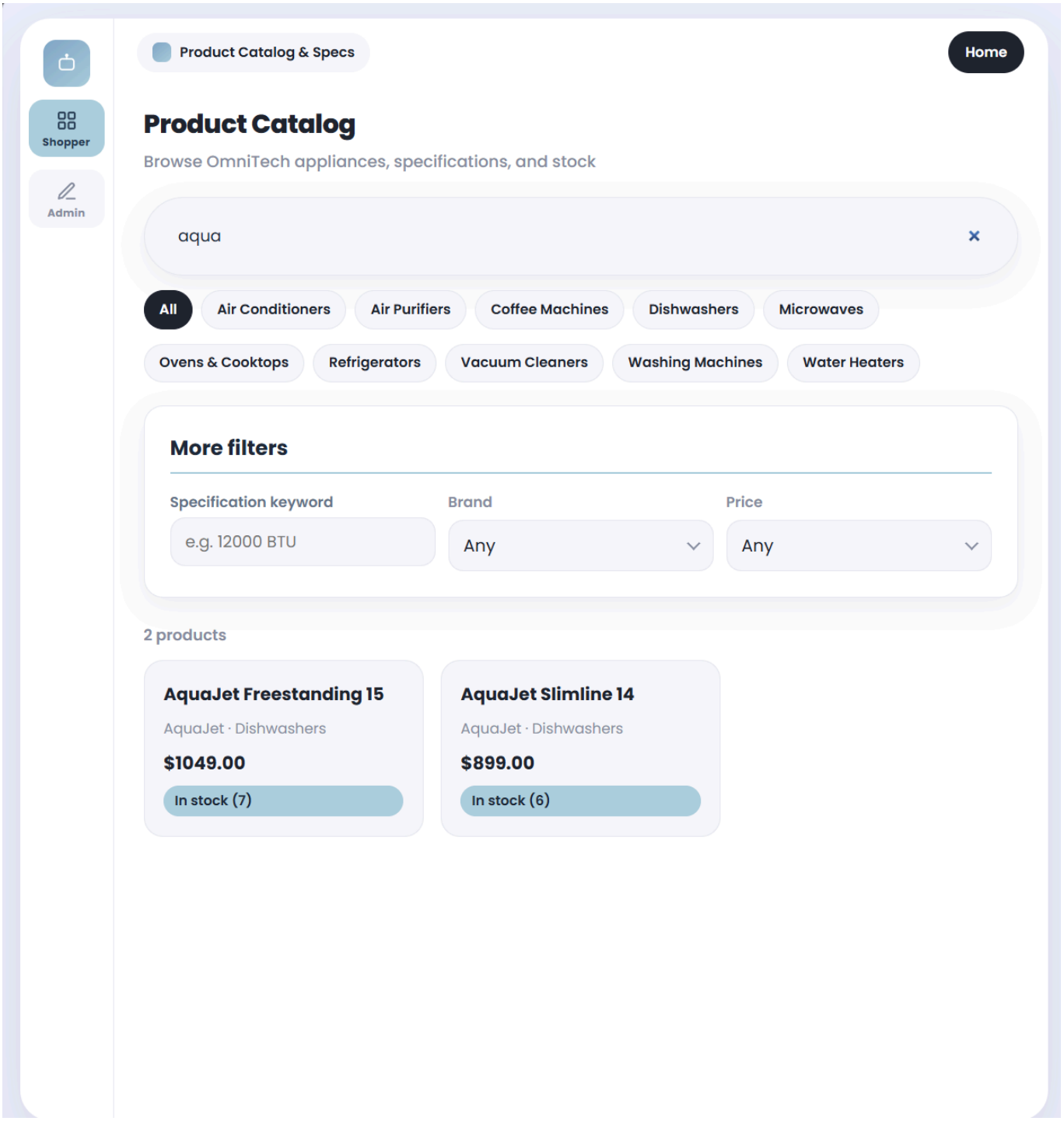
- The Flask application and database modules are reloaded after the temporary database path is set. This gives each test a clean, seeded database state.
- The automated tests do not call the real Ollama service. Tests that require an AI response monkeypatch **backend.ai.generate** with controlled fake responses.
- AI failure cases are also simulated by raising **AIUnavailable**, so the error-handling behaviour is tested consistently on both local machines and GitHub Actions CI.

Manual Browser Testing:

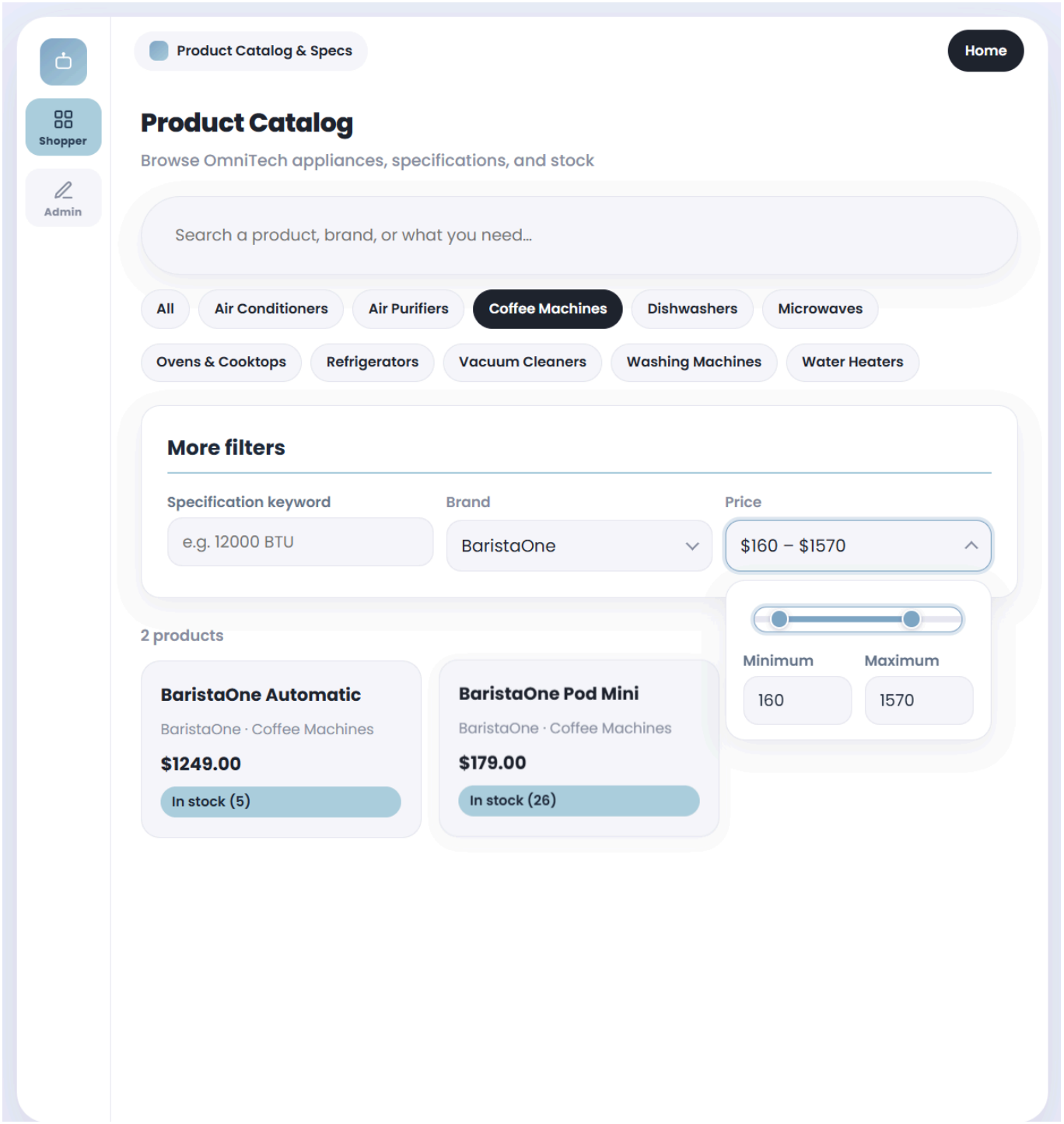
What I tested	Result
Product catalogue page loads and shows the seeded products	Pass
Searching for a product by name returns the matching product	Pass
Filtering products by category, brand, price range, and specification keyword updates the catalogue results	Pass
Opening a product displays its stored price, stock availability, category, and specifications	Pass
The basic AI summary is generated for a selected product	Pass
The AI review generates a summary based on the selected product's stored specifications and shows warnings if a claim cannot be supported	Pass
Admin can add a product	Pass
Admin can edit a product's details, including its price, stock, and category	Pass
Admin can add, edit, and delete product specifications	Pass
Admin can delete a product where allowed	Pass
Invalid product input, such as missing required fields or a negative price, is rejected with an error message	Pass
HTMX updates the relevant catalogue or admin section without a full page refresh where used	Pass
The shared home page link opens the Product Catalogue feature on port 5001	Pass



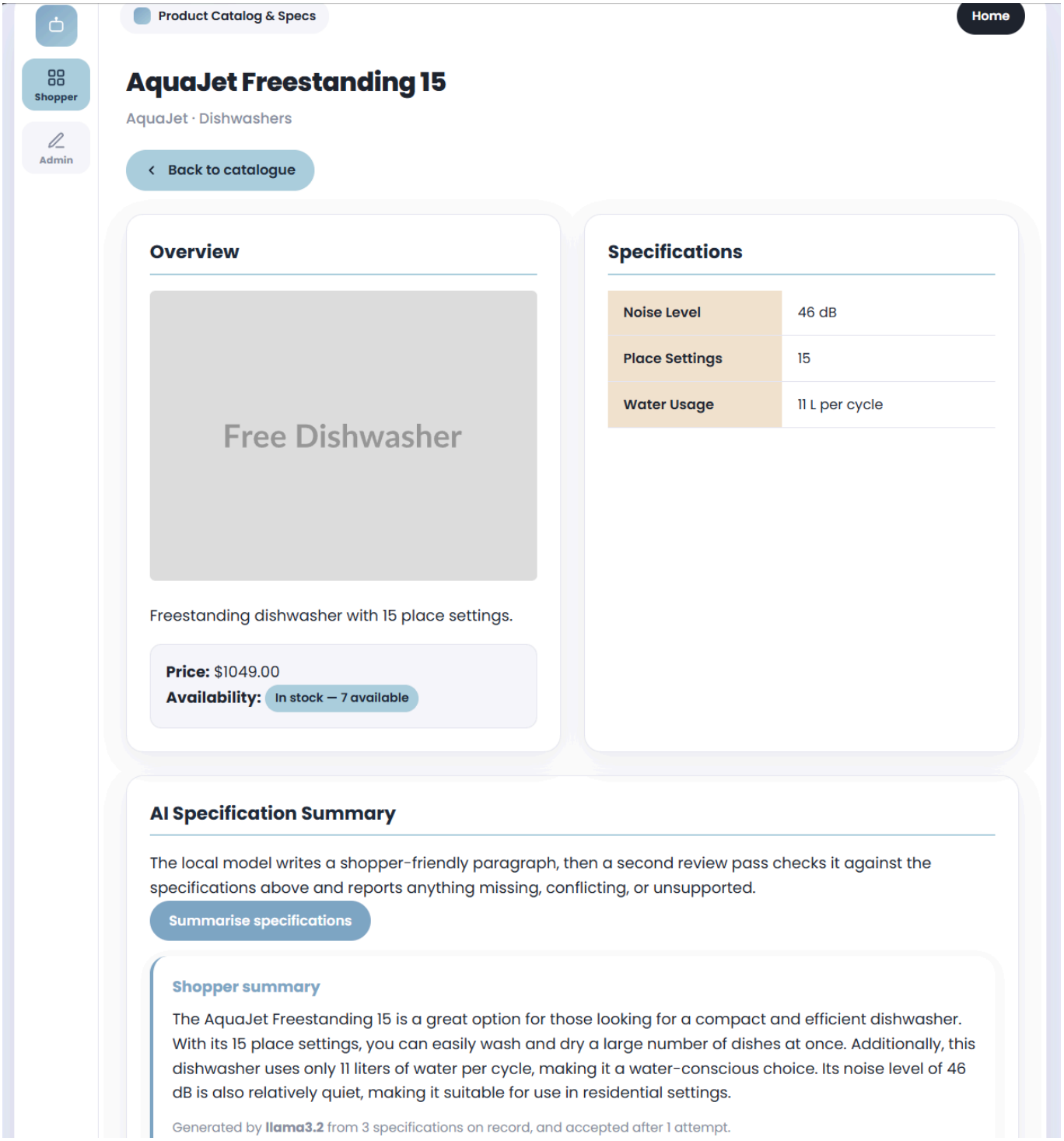
Test Evidence 1.3



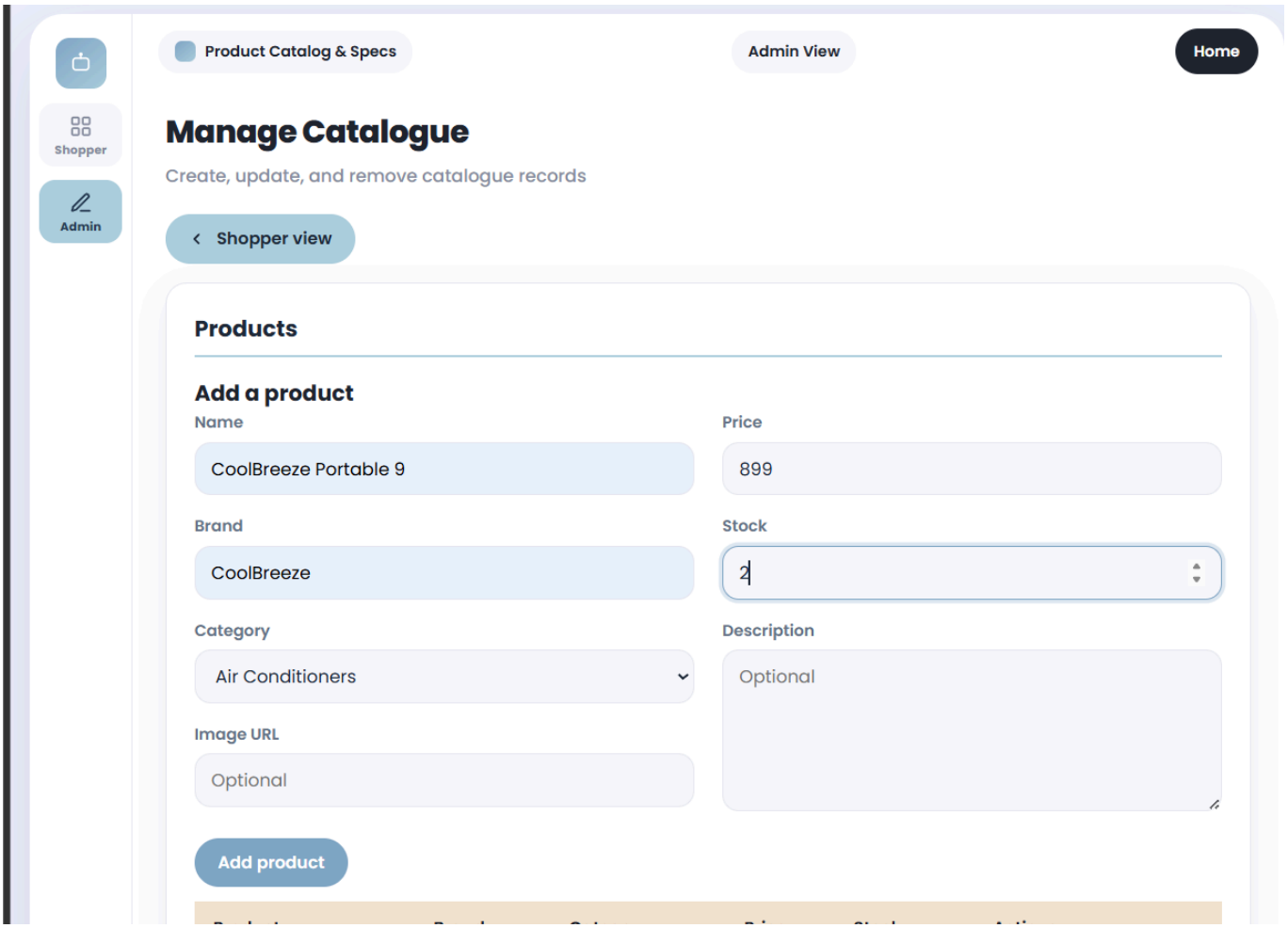
Test Evidence 1.4



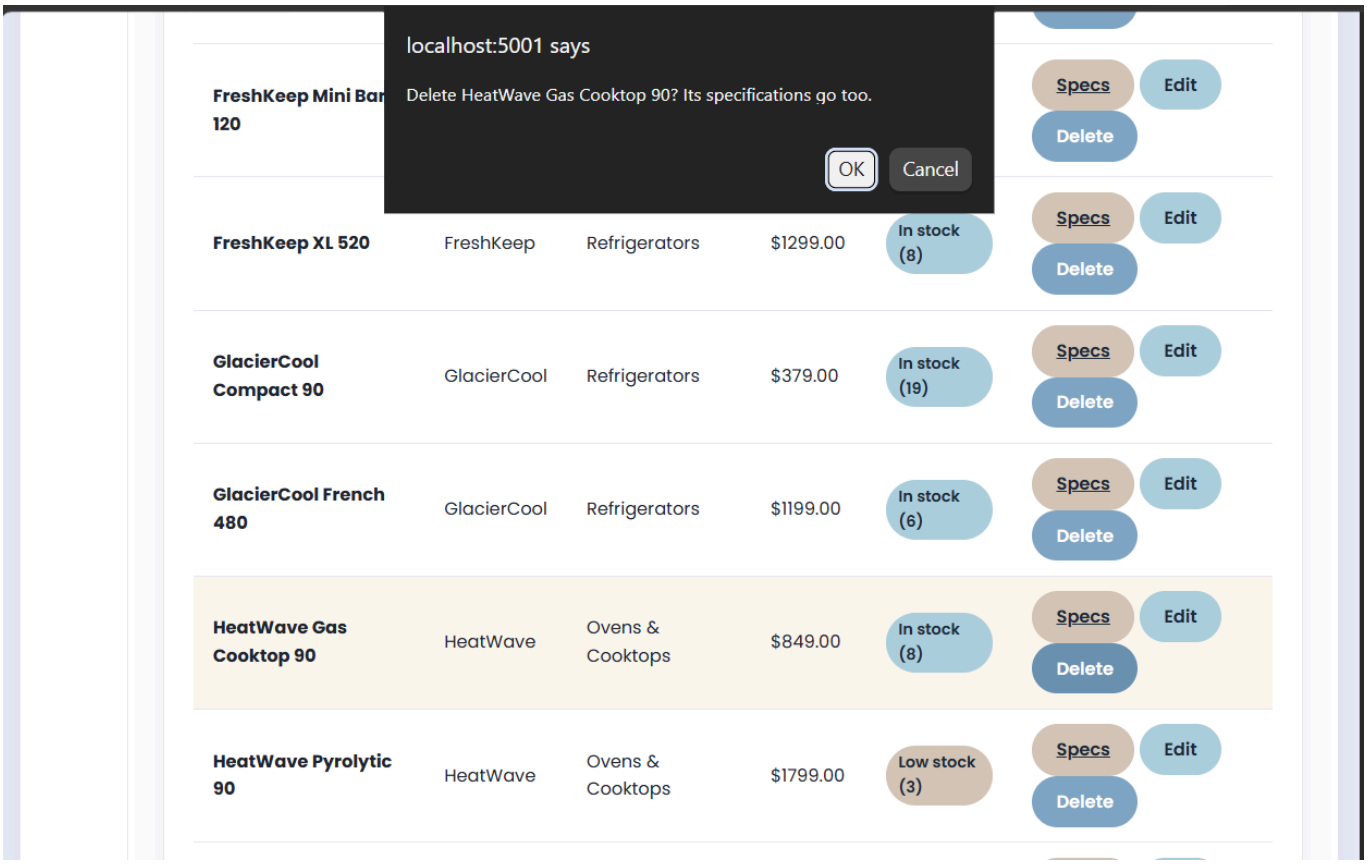
Test Evidence 1.5



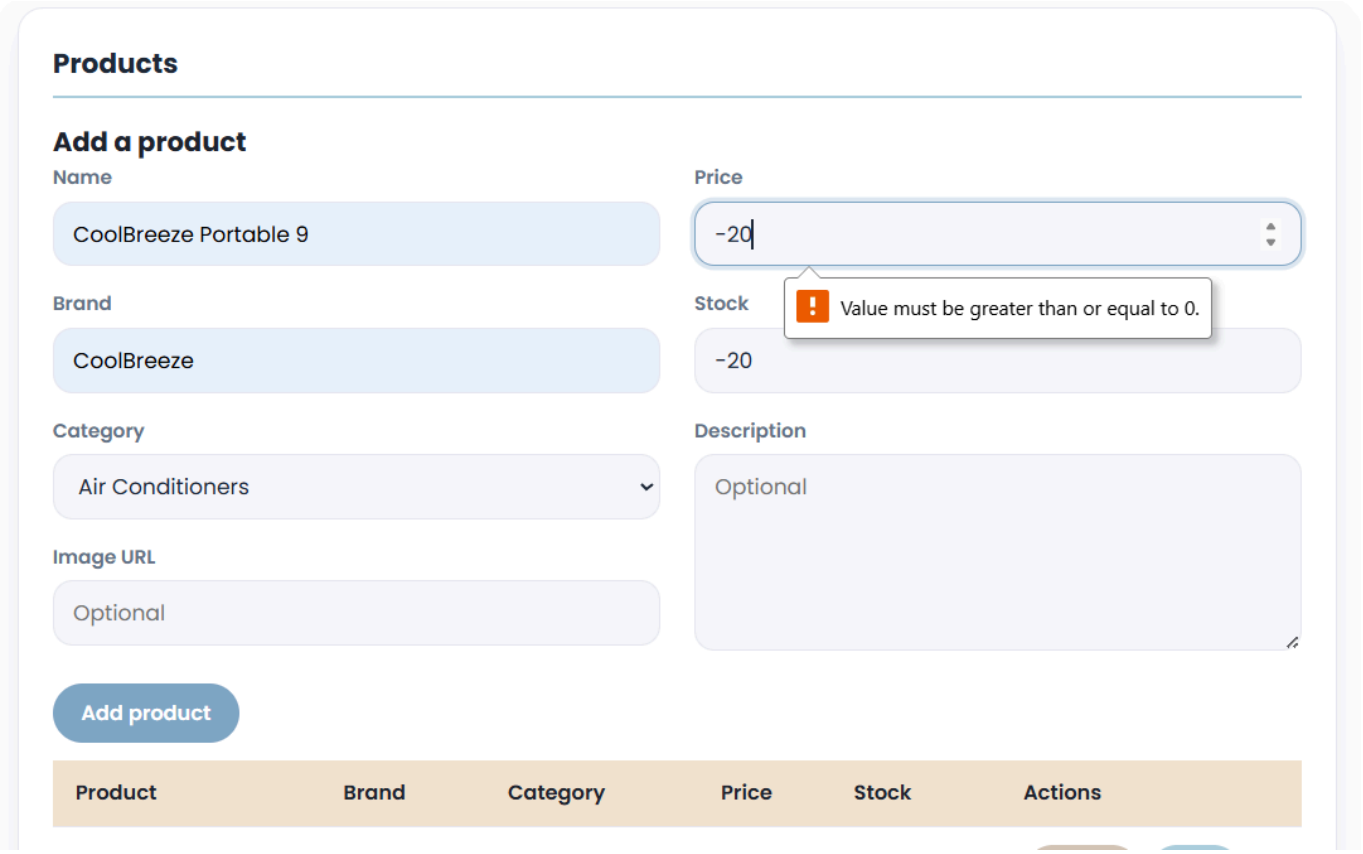
Test Evidence 1.6



Test evidence 1.7



Test evidence 1.8



Test Evidence 1.9

Student 2

Tests

Test File	Tests	What it covers
tests/test_app.py	7	Tests if the index page loads, CRUD operations for customers, applying and deleting tags, and generating tags
tests/test_agent.py	3	Tests if the created tags are in a valid format, checks if the AI created are not in the list of valid tags, if the fallback plan works

Command to run the test:
cd student-2
python -m pytest tests/ -v

```
PS C:\Users\User\Documents\GitHub\OmniTech> cd student-2
PS C:\Users\User\Documents\GitHub\OmniTech\student-2> python -m pytest tests/ -v
============================================= test session starts =============================================
platform win32 -- Python 3.14.3, pytest-8.3.5, pluggy-1.6.0 -- C:\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\User\Documents\GitHub\OmniTech\student-2
collected 10 items

tests/test_agent.py::test_observe_valid_tags PASSED                                          [ 10%]
tests/test_agent.py::test_observe_rejects_hallucinated_tags PASSED                           [ 20%]
tests/test_agent.py::test_fallback_plan PASSED                                               [ 30%]
tests/test_app.py::test_index_page PASSED                                                    [ 40%]
tests/test_app.py::test_create_customer PASSED                                               [ 50%]
tests/test_app.py::test_update_customer PASSED                                               [ 60%]
tests/test_app.py::test_delete_customer PASSED                                               [ 70%]
tests/test_app.py::test_generate_ai_suggestions PASSED                                       [ 80%]
tests/test_app.py::test_apply_tags PASSED                                                    [ 90%]
tests/test_app.py::test_delete_preference_tag PASSED                                         [100%]

============================================== 10 passed in 1.22s ==============================================
```

Manual Testing

What I tested	Result
Customer directory page loads on port 5002 and displays seeded customer profiles	Pass
Searching for a customer by ID via the lookup search bar returns the matching customer profile	Pass
Registering a new customer account creates a database record and updates the directory table	Pass
Clicking inline edit converts a table row into an editable form pre-filled with current customer details	Pass
Updating customer personal details and shipping address persists changes to the database and refreshes the row	Pass
Submitting invalid or missing fields during profile editing triggers a native browser validation alert	Pass
Clicking Generate AI Suggestions executes the Plan-Act-Observe-Adapt loop and displays recommended ecosystem tags	Pass
Selecting candidate tags and clicking Apply Suggestions persists tags to the profile	Pass
Clicking the delete button on an assigned preference tag badge removes the tag from the UI and database	Pass
Deleting a customer profile purges the record along with all associated preferences and tags via cascading delete	Pass
Disconnecting the Ollama service causes the agentic workflow to gracefully degrade to the keyword-matched fallback model	Pass
HTMX updates profile cards, table rows, and tag containers dynamically without causing a full page refresh	Pass

See 19. Screenshots of the Integrated Application student 2 for screenshots of the application working

Student 3

Automated Test Results:
I have 44 automated tests spread across three files:

Test File	Tests	What it covers
tests/test_app.py	32	All the REST API endpoints, HTMX partials, the chat endpoint, and both page routes
tests/test_agent.py	6	The Observe validation logic, Adapt re-prompting, and fallback behaviour
tests/test_catalog.py	6	Product data integrity, search ranking, and the context block generator

To run them:
cd student-3
python -m pytest tests/ -v

```
kartikaysingh@Kartikays-MacBook-Air student-3 % cd student-3
python3 -m pytest tests/ -v
platform darwin -- Python 3.14.4, pytest-8.3.5, pluggy-1.6.0 -- /Library/Frameworks/Python.framework/Versions/3.14/bin/python3
cachedir: .pytest_cache
rootdir: /Users/kartikaysingh/Documents/GitHub/OmniTech/student-3
collected 45 items

tests/test_agent.py::test_observe_accepts_well_formed_answer PASSED                                  [  2%]
tests/test_agent.py::test_observe_rejects_non_json PASSED                                            [  4%]
tests/test_agent.py::test_observe_flags_hallucinated_ids PASSED                                      [  6%]
tests/test_agent.py::test_observe_salvages_ids_named_only_in_reply PASSED                            [  8%]
tests/test_agent.py::test_observe_ignores_literal_ID_placeholder PASSED                              [ 11%]
tests/test_agent.py::test_run_consultation_uses_fallback_when_ollama_down PASSED                     [ 13%]
tests/test_agent.py::test_run_consultation_adapts_after_bad_first_answer PASSED                      [ 15%]
tests/test_app.py::test_index_page PASSED                                                            [ 17%]
tests/test_app.py::test_dashboard_page PASSED                                                        [ 20%]
tests/test_app.py::test_list_sessions PASSED                                                         [ 22%]
tests/test_app.py::test_list_sessions_filtered_by_user PASSED                                        [ 24%]
tests/test_app.py::test_create_session_with_user PASSED                                              [ 26%]
tests/test_app.py::test_get_session_bundle PASSED                                                    [ 28%]
tests/test_app.py::test_get_session_not_found PASSED                                                 [ 31%]
tests/test_app.py::test_update_session_title PASSED                                                  [ 33%]
tests/test_app.py::test_update_session_requires_title PASSED                                         [ 35%]
tests/test_app.py::test_delete_session PASSED                                                        [ 37%]
tests/test_app.py::test_delete_session_cascades PASSED                                               [ 40%]
tests/test_app.py::test_list_messages PASSED                                                         [ 42%]
tests/test_app.py::test_create_message PASSED                                                        [ 44%]
tests/test_app.py::test_create_message_rejects_bad_sender PASSED                                     [ 46%]
tests/test_app.py::test_clear_chat_history PASSED                                                    [ 48%]
tests/test_app.py::test_delete_single_message PASSED                                                 [ 51%]
tests/test_app.py::test_list_recommendations PASSED                                                  [ 53%]
tests/test_app.py::test_create_recommendation PASSED                                                 [ 55%]
tests/test_app.py::test_create_recommendation_rejects_unknown_products PASSED                        [ 57%]
tests/test_app.py::test_update_recommendation_tags PASSED                                            [ 60%]
tests/test_app.py::test_add_single_tag PASSED                                                        [ 62%]
tests/test_app.py::test_add_tag_not_found PASSED                                                     [ 64%]
tests/test_app.py::test_delete_recommendation PASSED                                                 [ 66%]
tests/test_app.py::test_chat_requires_fields PASSED                                                  [ 68%]
tests/test_app.py::test_chat_session_not_found PASSED                                                [ 71%]
tests/test_app.py::test_chat_persists_both_turns PASSED                                              [ 73%]
tests/test_app.py::test_chat_valid_model_answer_saves_recommendation PASSED                          [ 75%]
tests/test_app.py::test_chat_htmx_returns_html_fragment PASSED                                       [ 77%]
tests/test_app.py::test_partial_session_list PASSED                                                  [ 80%]
tests/test_app.py::test_partial_chat_panel PASSED                                                    [ 82%]
tests/test_app.py::test_partial_recommendations PASSED                                               [ 84%]
tests/test_app.py::test_stylesheet_served PASSED                                                     [ 86%]
tests/test_catalog.py::test_every_product_is_complete PASSED                                         [ 88%]
tests/test_catalog.py::test_ids_are_unique PASSED                                                    [ 91%]
tests/test_catalog.py::test_get_many_drops_unknown_and_dedupes PASSED                                [ 93%]
tests/test_catalog.py::test_search_ranks_relevant_products_first PASSED                              [ 95%]
tests/test_catalog.py::test_search_never_returns_empty PASSED                                        [ 97%]
tests/test_catalog.py::test_context_block_lists_ids_and_prices PASSED                                [100%]

============================================== 45 passed in 0.53s ==============================================
kartikaysingh@Kartikays-MacBook-Air student-3 %
```

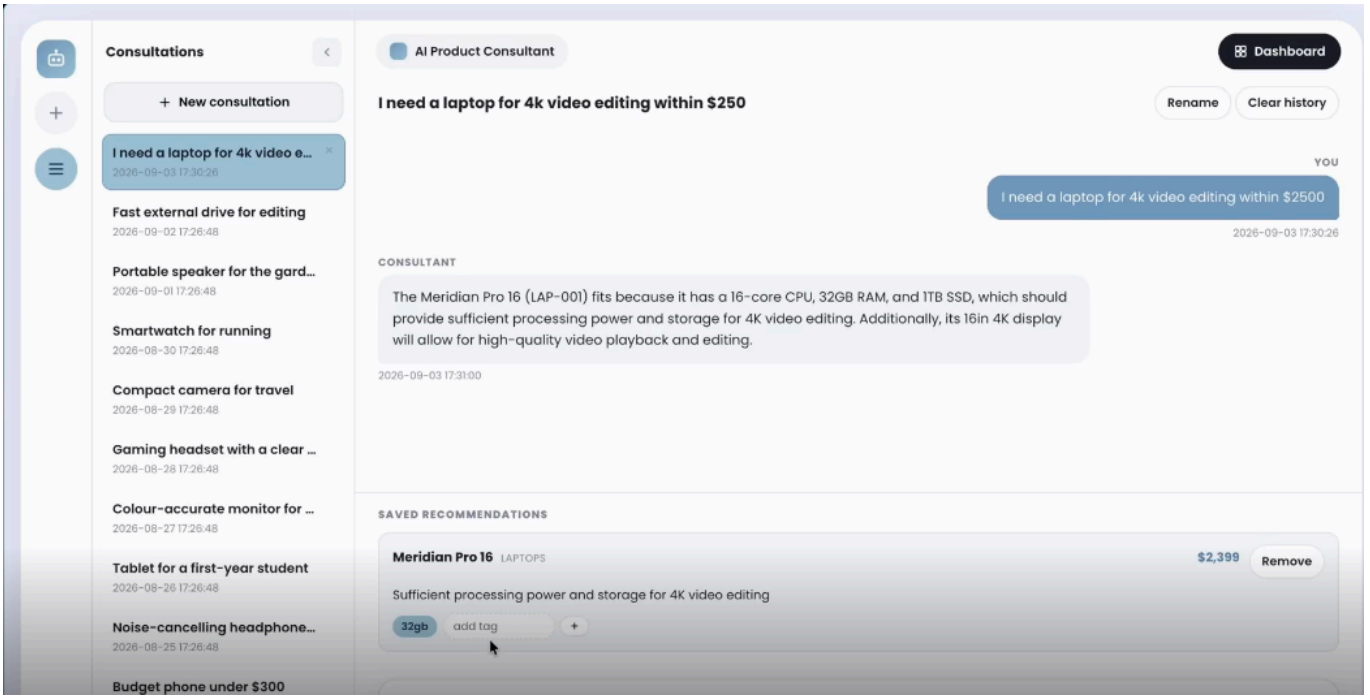
Test Evidence 3.1

Test Isolation:

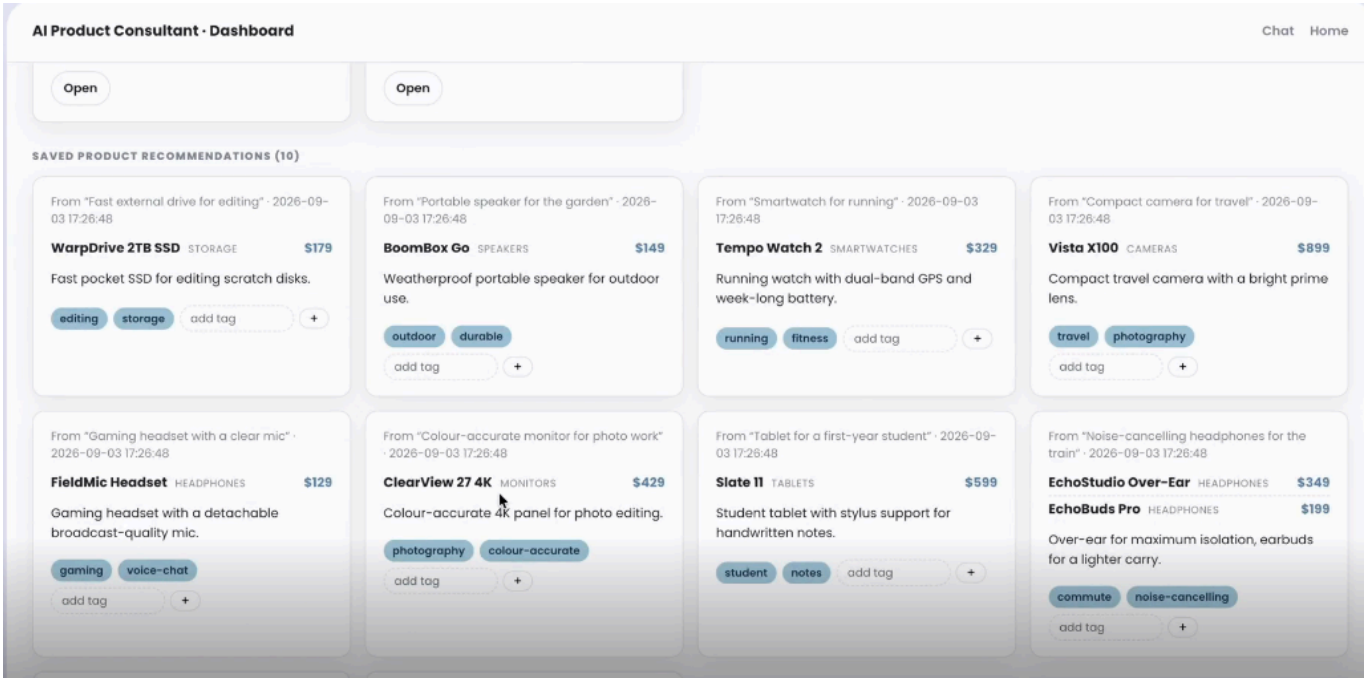
- Every test gets its own fresh SQLite database (using pytest's tmp_path fixture), so one test will not affect any of the other tests
- Ollama is never being called, all tests that need a model to respond monkeypatches agent.act with a fake function. This makes it so the test work exactly the same on my laptop and in the CI.

Manual Browser Testing:

What I tested	Result
Home page loads, session list appears via HTMX	Pass
Creating a new consultation from the hero section	Pass
Sending a message, user and AI bubbles appear without page reload	Pass
Recommendations dock updates automatically after each message	Pass
Renaming a session inline	Pass
Deleting a session (with the inline confirmation prompt)	Pass
Clearing chat history (with inline confirmation)	Pass
Adding a preference tag to a recommendation	Pass
Deleting a recommendation (with inline confirmation)	Pass
Dashboard shows session cards with message and recommendation counts	Pass
Dashboard "Open" button deep-links to the right session	Pass
With Ollama off, the fallback still returns real catalog products	Pass
No errors in the browser console	Pass



Test Pass 3.1



Test Pass 3.2

Student 4 - Sangyun Kim (Incomplete)

Student 5:

```
student-5  | 192.168.65.1 - - [09/Sep/2026 12:05:12] "GET / HTTP/1.1" 200 -
student-5  | 192.168.65.1 - - [09/Sep/2026 12:05:12] "GET /css/style.css HTTP/1.1" 304 -
student-5  | 192.168.65.1 - - [09/Sep/2026 12:05:26] "GET /create-ticket HTTP/1.1" 200 -
student-5  | 192.168.65.1 - - [09/Sep/2026 12:05:26] "GET /css/style.css HTTP/1.1" 304 -
student-5  | 192.168.65.1 - - [09/Sep/2026 12:05:31] "POST /tickets/create HTTP/1.1" 201 -
student-5  | 192.168.65.1 - - [09/Sep/2026 12:05:35] "GET / HTTP/1.1" 200 -
student-5  | 192.168.65.1 - - [09/Sep/2026 12:05:35] "GET /css/style.css HTTP/1.1" 304 -
student-5  | 192.168.65.1 - - [09/Sep/2026 12:05:36] "GET /tickets HTTP/1.1" 200 -
student-5  | 192.168.65.1 - - [09/Sep/2026 12:05:36] "GET /css/style.css HTTP/1.1" 304 -
```

```
student-5  | 192.168.65.1 - - [09/Sep/2026 12:05:42] "POST /api/tickets/11/evaluate HTTP/1.1" 200 -
student-5  | 192.168.65.1 - - [09/Sep/2026 12:05:44] "POST /tickets/11/update HTTP/1.1" 200 -
student-5  | 192.168.65.1 - - [09/Sep/2026 12:05:50] "DELETE /tickets/11/delete HTTP/1.1" 200 -
```

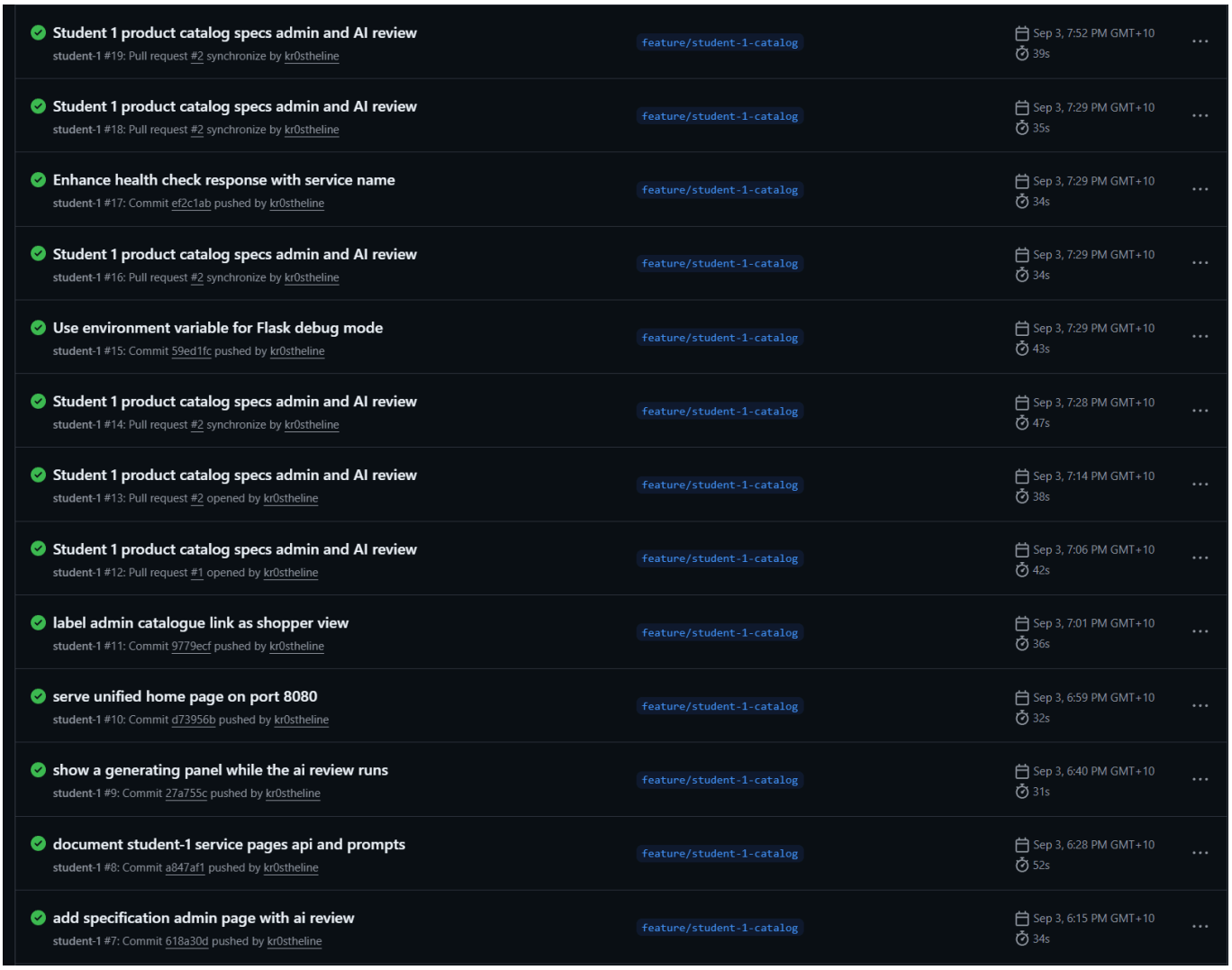
Manual Testing:

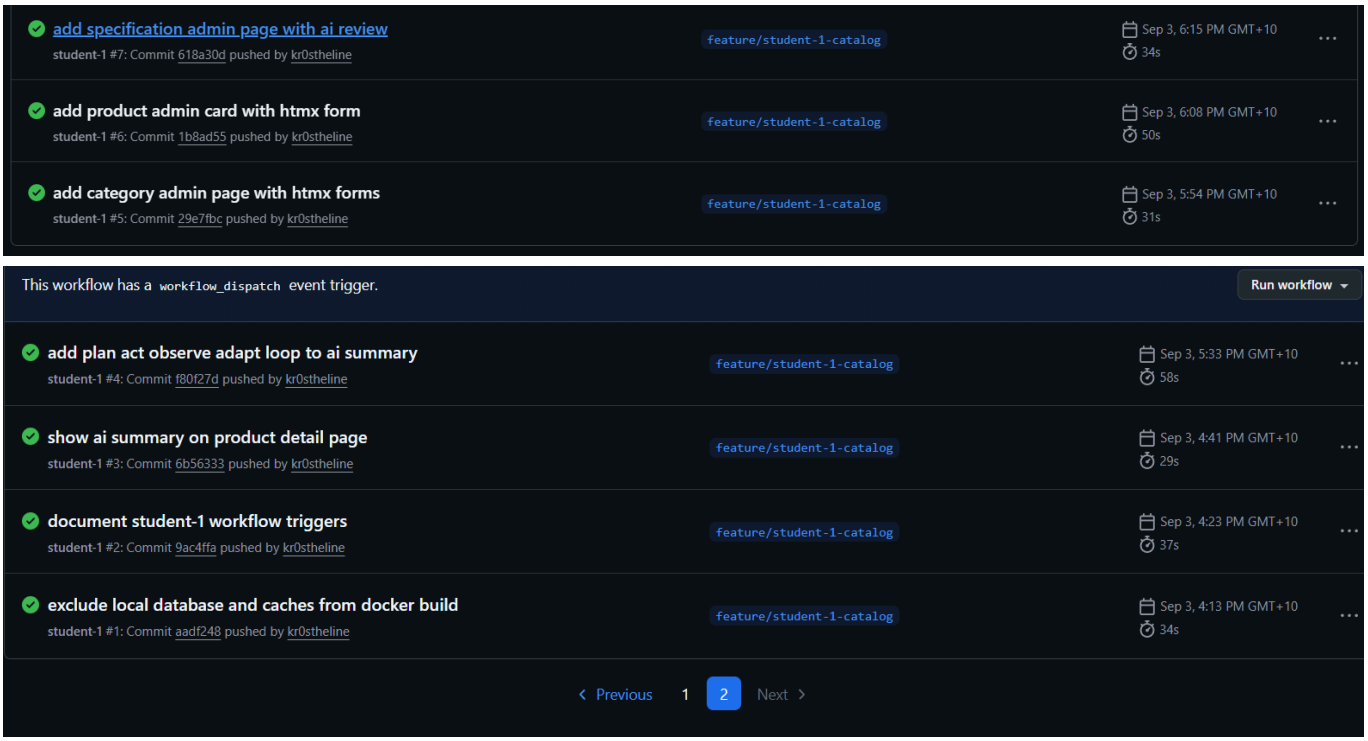
What I tested	Result
Home page loads	Pass
Create tickets page loads	Pass
View ticket page loads	Pass
Orders table displays 10 orders (seed data)	Pass
Tickets table displays 10 tickets (seed data)	Pass
Create ticket with order id, product category and claim	Pass
AI returns decision and reasoning	Pass
Updating ticket updates ticket status on ticket table	Pass
Deleting ticket updates ticket table	Pass
No errors in the browser console	Pass

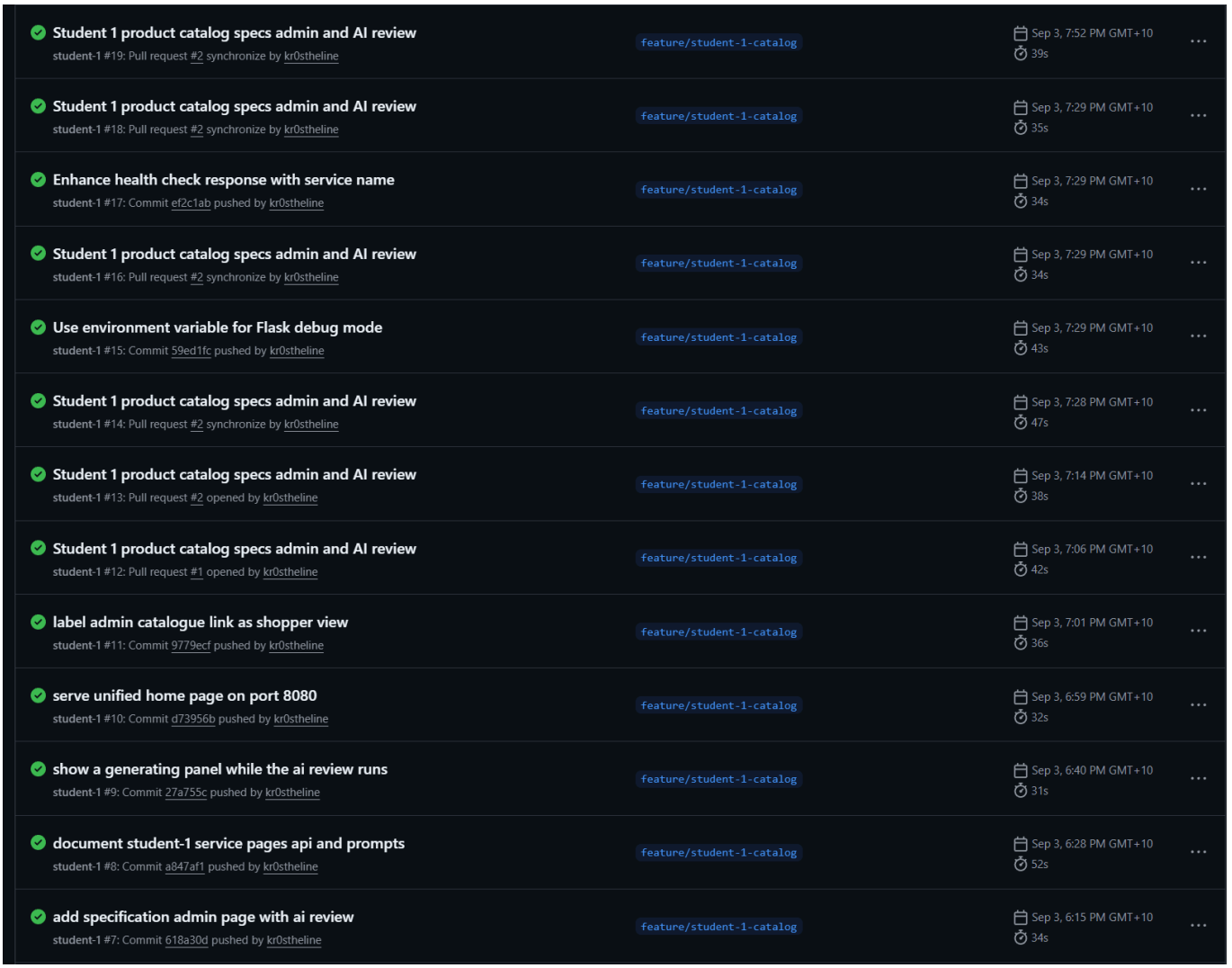
See 19. Screenshots of the Integrated Application student 5 for screenshots of the application working

17. GitHub Actions Workflow Execution Evidence

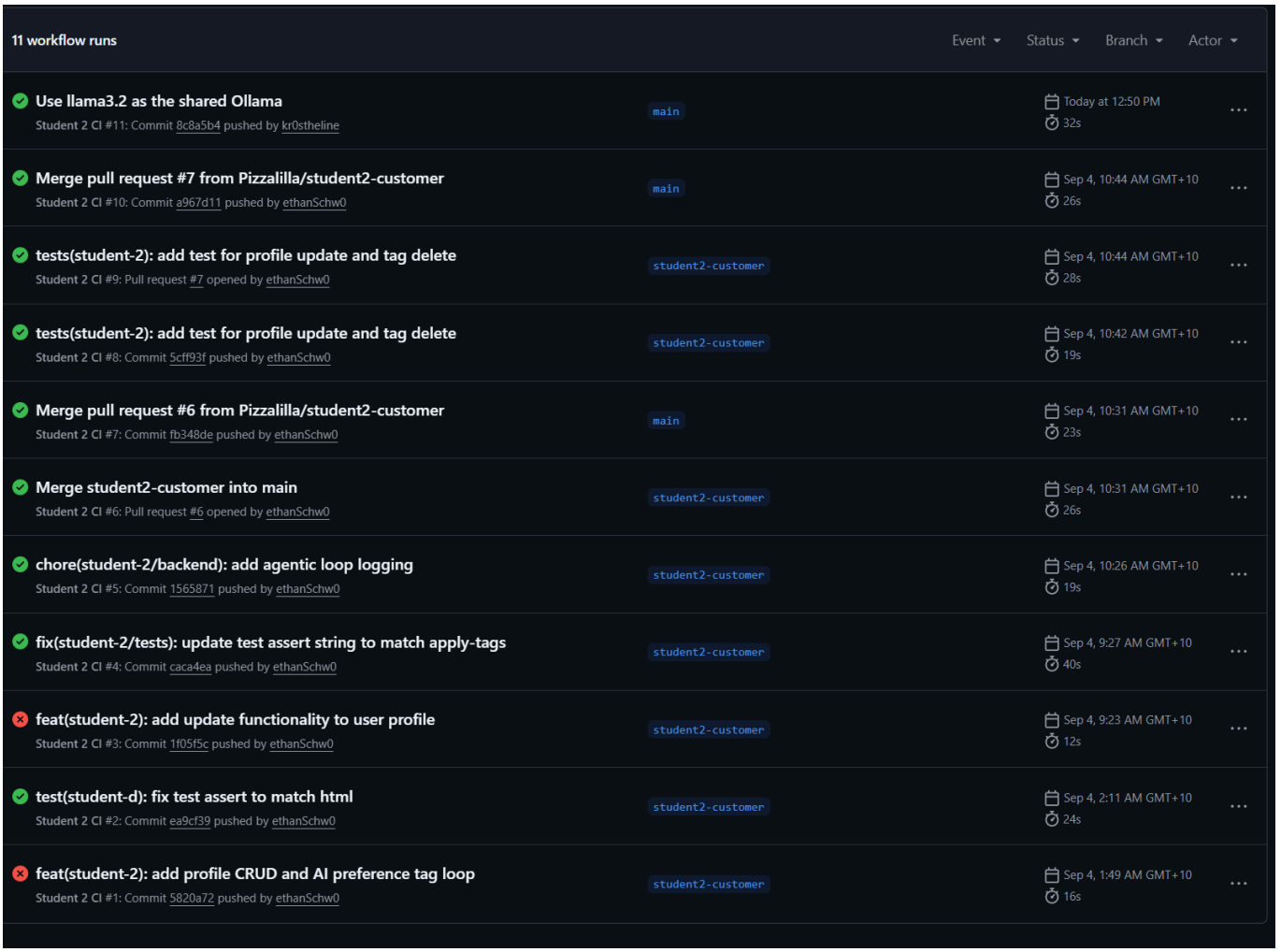
Student 1



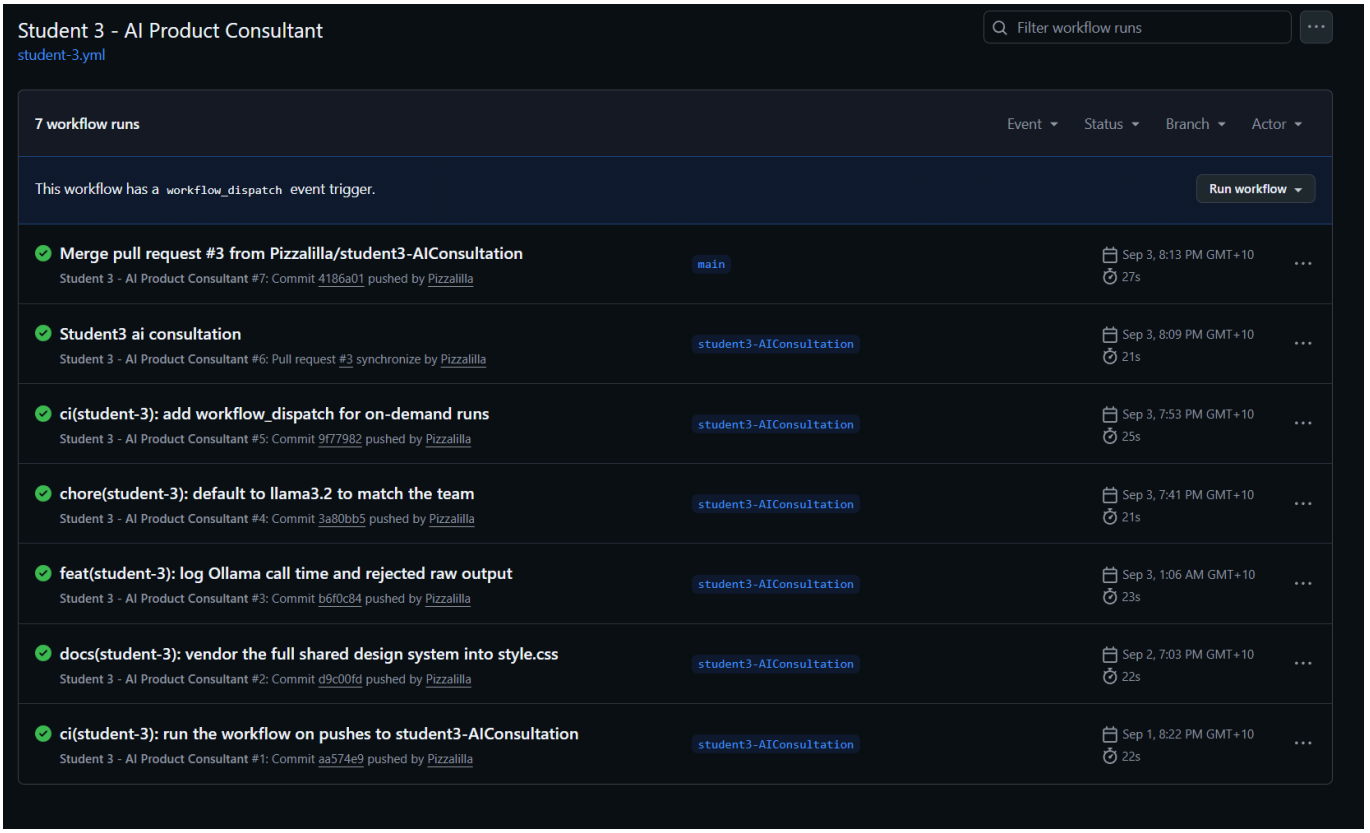



Student 1 product catalog specs admin and AI review
student-1 #19: Pull request #2 synchronize by kr0stheline
feature/student-1-catalog
Sep 3, 7:52 PM GMT+10
39s
Student 1 product catalog specs admin and AI review
student-1 #18: Pull request #2 synchronize by kr0stheline
feature/student-1-catalog
Sep 3, 7:29 PM GMT+10
35s
Enhance health check response with service name
student-1 #17: Commit ef2c1ab pushed by kr0stheline
feature/student-1-catalog
Sep 3, 7:29 PM GMT+10
34s
Student 1 product catalog specs admin and AI review
student-1 #16: Pull request #2 synchronize by kr0stheline
feature/student-1-catalog
Sep 3, 7:29 PM GMT+10
34s
Use environment variable for Flask debug mode
student-1 #15: Commit 59ed1fc pushed by kr0stheline
feature/student-1-catalog
Sep 3, 7:29 PM GMT+10
43s
Student 1 product catalog specs admin and AI review
student-1 #14: Pull request #2 synchronize by kr0stheline
feature/student-1-catalog
Sep 3, 7:28 PM GMT+10
47s
Student 1 product catalog specs admin and AI review
student-1 #13: Pull request #2 opened by kr0stheline
feature/student-1-catalog
Sep 3, 7:14 PM GMT+10
38s
Student 1 product catalog specs admin and AI review
student-1 #12: Pull request #1 opened by kr0stheline
feature/student-1-catalog
Sep 3, 7:06 PM GMT+10
42s
label admin catalogue link as shopper view
student-1 #11: Commit 9779ecf pushed by kr0stheline
feature/student-1-catalog
Sep 3, 7:01 PM GMT+10
36s
serve unified home page on port 8080
student-1 #10: Commit d73956b pushed by kr0stheline
feature/student-1-catalog
Sep 3, 6:59 PM GMT+10
32s
show a generating panel while the ai review runs
student-1 #9: Commit 27a755c pushed by kr0stheline
feature/student-1-catalog
Sep 3, 6:40 PM GMT+10
31s
document student-1 service pages api and prompts
student-1 #8: Commit a847af1 pushed by kr0stheline
feature/student-1-catalog
Sep 3, 6:28 PM GMT+10
52s
add specification admin page with ai review
student-1 #7: Commit 618a30d pushed by kr0stheline
feature/student-1-catalog
Sep 3, 6:15 PM GMT+10
34s

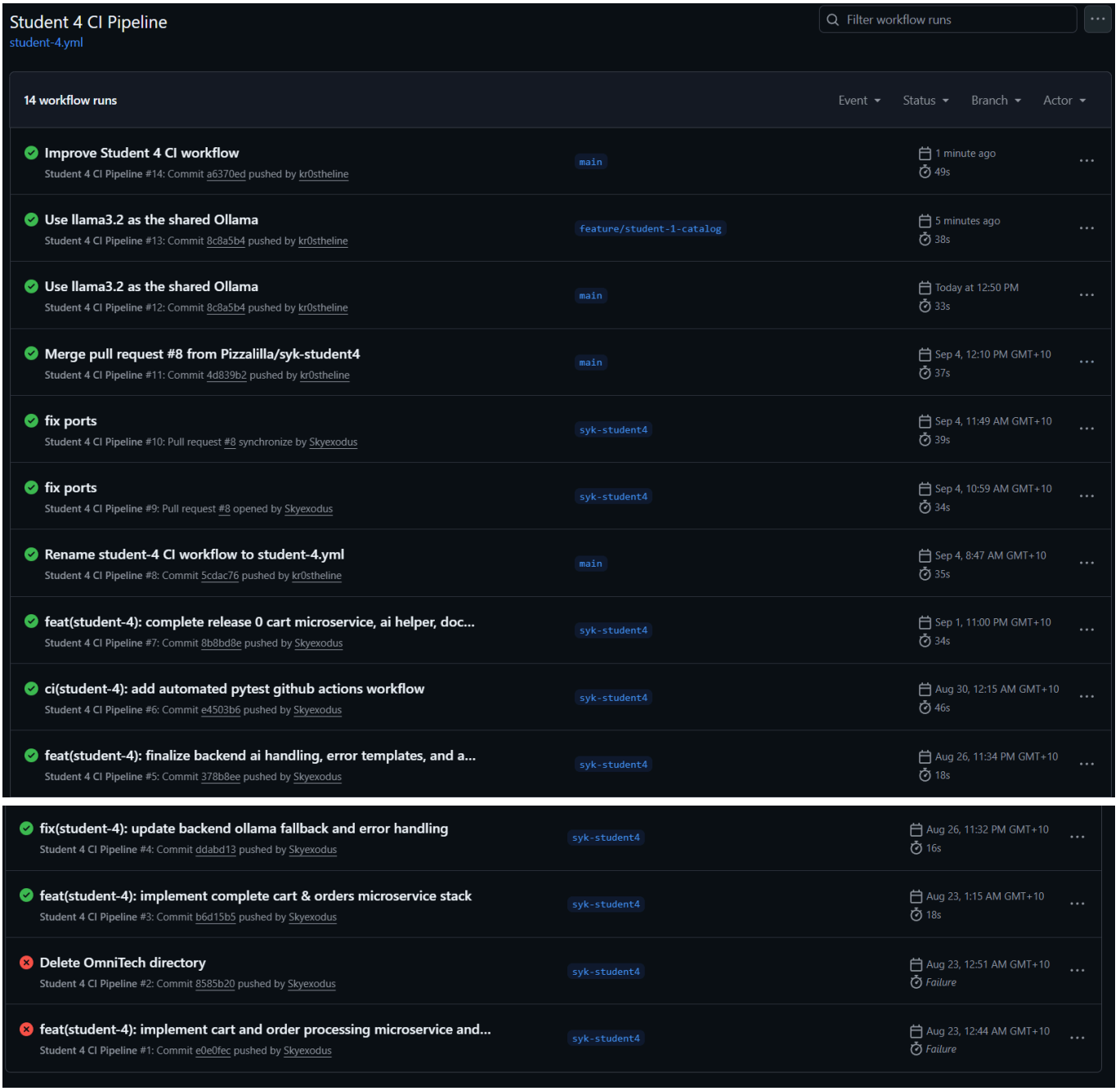
Student 2

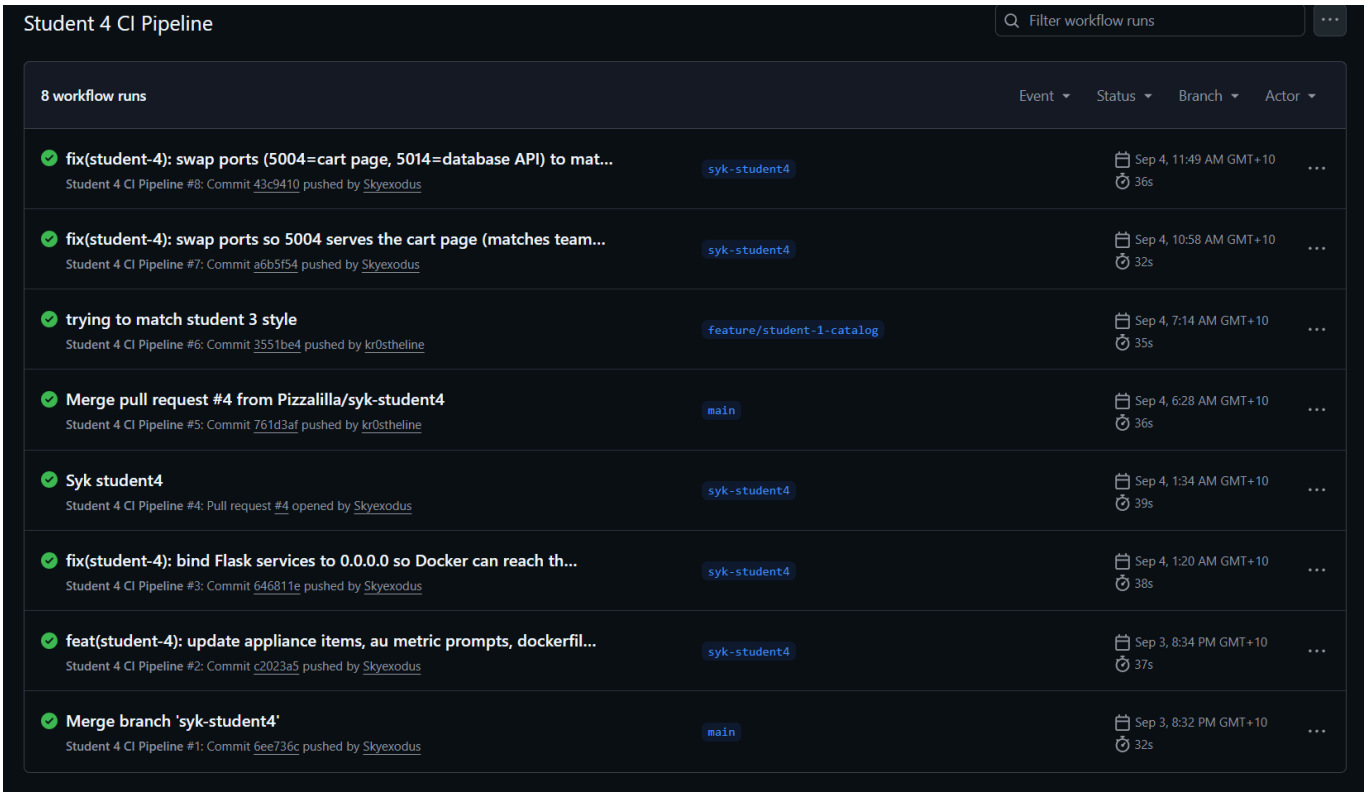

11 workflow runs
Event
Status
Branch
Actor
Use llama3.2 as the shared Ollama
Student 2 CI #11: Commit 8c8a5b4 pushed by kr0stheline
main
Today at 12:50 PM
32s
Merge pull request #7 from Pizzalilla/student2-customer
Student 2 CI #10: Commit a967d11 pushed by ethanSchw0
main
Sep 4, 10:44 AM GMT+10
26s
tests(student-2): add test for profile update and tag delete
Student 2 CI #9: Pull request #7 opened by ethanSchw0
student2-customer
Sep 4, 10:44 AM GMT+10
28s
tests(student-2): add test for profile update and tag delete
Student 2 CI #8: Commit 5cff93f pushed by ethanSchw0
student2-customer
Sep 4, 10:42 AM GMT+10
19s
Merge pull request #6 from Pizzalilla/student2-customer
Student 2 CI #7: Commit fb348de pushed by ethanSchw0
main
Sep 4, 10:31 AM GMT+10
23s
Merge student2-customer into main
Student 2 CI #6: Pull request #6 opened by ethanSchw0
student2-customer
Sep 4, 10:31 AM GMT+10
26s
chore(student-2/backend): add agentic loop logging
Student 2 CI #5: Commit 1565871 pushed by ethanSchw0
student2-customer
Sep 4, 10:26 AM GMT+10
19s
fix(student-2/tests): update test assert string to match apply-tags
Student 2 CI #4: Commit caca4ea pushed by ethanSchw0
student2-customer
Sep 4, 9:27 AM GMT+10
40s
feat(student-2): add update functionality to user profile
Student 2 CI #3: Commit 1f05f5c pushed by ethanSchw0
student2-customer
Sep 4, 9:23 AM GMT+10
12s
test(student-d): fix test assert to match html
Student 2 CI #2: Commit ea9cf39 pushed by ethanSchw0
student2-customer
Sep 4, 2:11 AM GMT+10
24s
feat(student-2): add profile CRUD and AI preference tag loop
Student 2 CI #1: Commit 5820a72 pushed by ethanSchw0
student2-customer
Sep 4, 1:49 AM GMT+10
16s

Student 3

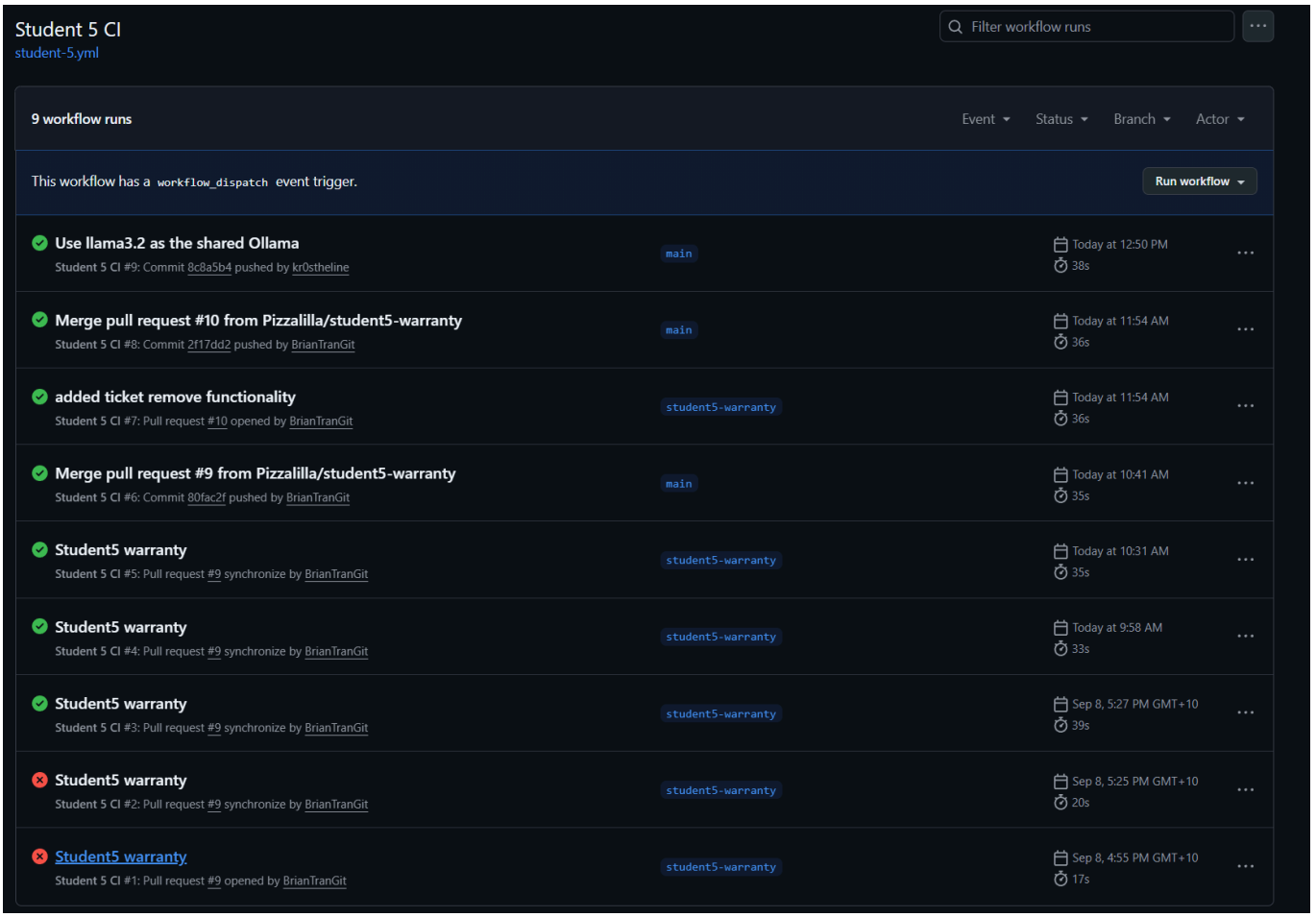

Student 3 - AI Product Consultant
student-3.yml
Filter workflow runs
7 workflow runs
Event
Status
Branch
Actor
This workflow has a workflow_dispatch event trigger.
Run workflow
Merge pull request #3 from Pizzalilla/student3-AIConsultation
Student 3 - AI Product Consultant #7: Commit 4186a01 pushed by Pizzalilla
main
Sep 3, 8:13 PM GMT+10
27s
Student3 ai consultation
Student 3 - AI Product Consultant #6: Pull request #3 synchronize by Pizzalilla
student3-AIConsultation
Sep 3, 8:09 PM GMT+10
21s
ci(student-3): add workflow_dispatch for on-demand runs
Student 3 - AI Product Consultant #5: Commit 9f77982 pushed by Pizzalilla
student3-AIConsultation
Sep 3, 7:53 PM GMT+10
25s
chore(student-3): default to llama3.2 to match the team
Student 3 - AI Product Consultant #4: Commit 3a80bb5 pushed by Pizzalilla
student3-AIConsultation
Sep 3, 7:41 PM GMT+10
21s
feat(student-3): log Ollama call time and rejected raw output
Student 3 - AI Product Consultant #3: Commit b6f0c84 pushed by Pizzalilla
student3-AIConsultation
Sep 3, 1:06 AM GMT+10
23s
docs(student-3): vendor the full shared design system into style.css
Student 3 - AI Product Consultant #2: Commit d9c00fd pushed by Pizzalilla
student3-AIConsultation
Sep 2, 7:03 PM GMT+10
22s
ci(student-3): run the workflow on pushes to student3-AIConsultation
Student 3 - AI Product Consultant #1: Commit aa574e9 pushed by Pizzalilla
student3-AIConsultation
Sep 1, 8:22 PM GMT+10
22s

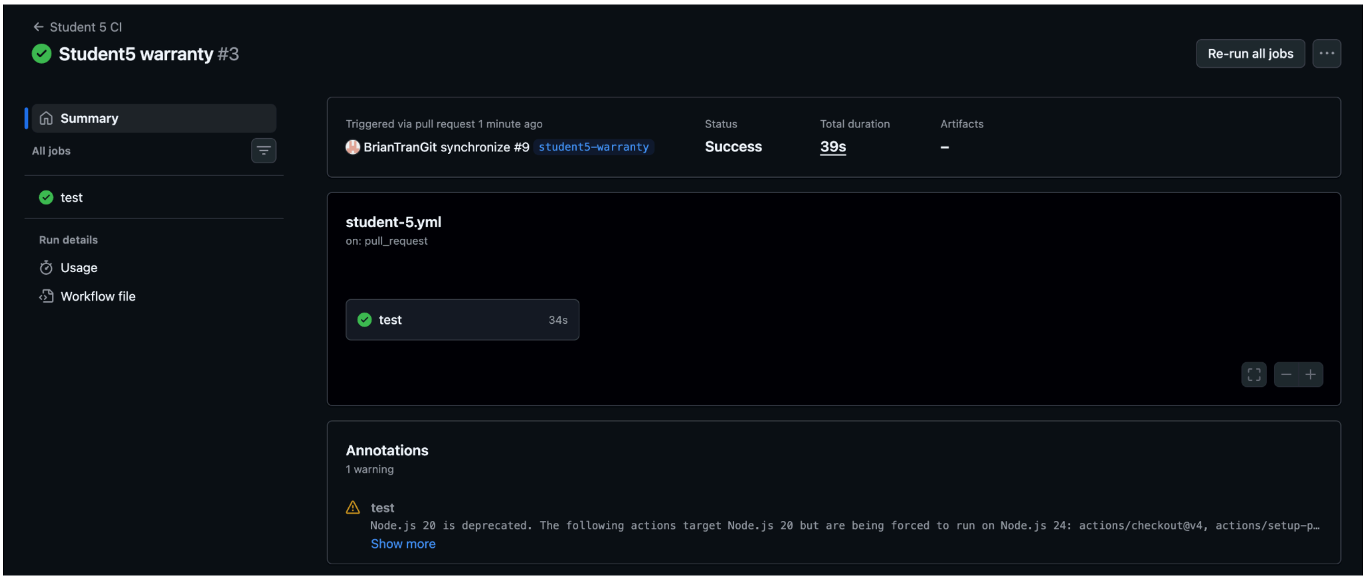
Student 4

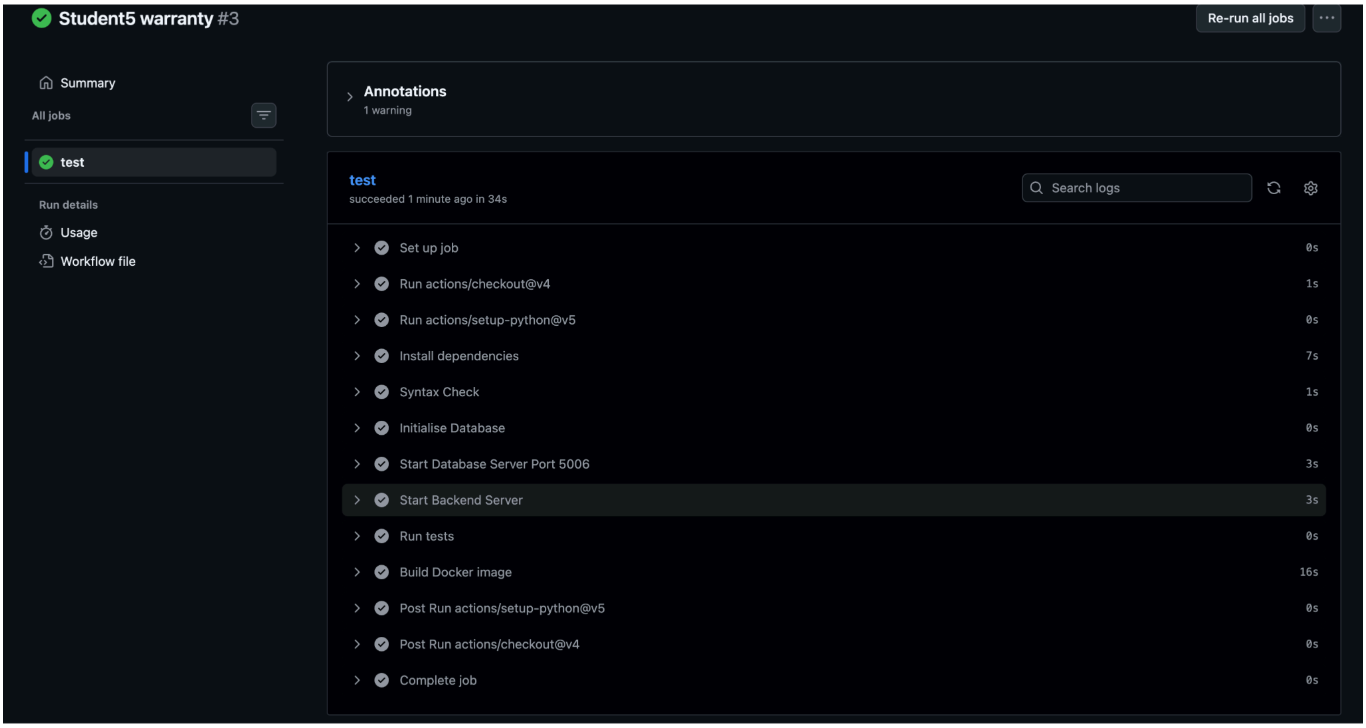

Student 4 CI Pipeline
student-4.yml
Filter workflow runs
14 workflow runs
Event
Status
Branch
Actor
Improve Student 4 CI workflow
Student 4 CI Pipeline #14: Commit a6370ed pushed by kr0stheline
main
1 minute ago
49s
Use llama3.2 as the shared Ollama
Student 4 CI Pipeline #13: Commit 8c8a5b4 pushed by kr0stheline
feature/student-1-catalog
5 minutes ago
38s
Use llama3.2 as the shared Ollama
Student 4 CI Pipeline #12: Commit 8c8a5b4 pushed by kr0stheline
main
Today at 12:50 PM
33s
Merge pull request #8 from Pizzalilla/syk-student4
Student 4 CI Pipeline #11: Commit 4d839b2 pushed by kr0stheline
main
Sep 4, 12:10 PM GMT+10
37s
fix ports
Student 4 CI Pipeline #10: Pull request #8 synchronize by Skyexodus
syk-student4
Sep 4, 11:49 AM GMT+10
39s
fix ports
Student 4 CI Pipeline #9: Pull request #8 opened by Skyexodus
syk-student4
Sep 4, 10:59 AM GMT+10
34s
Rename student-4 CI workflow to student-4.yml
Student 4 CI Pipeline #8: Commit 5cdac76 pushed by kr0stheline
main
Sep 4, 8:47 AM GMT+10
35s
feat(student-4): complete release 0 cart microservice, ai helper, doc...
Student 4 CI Pipeline #7: Commit 8b8bd8e pushed by Skyexodus
syk-student4
Sep 1, 11:00 PM GMT+10
34s
ci(student-4): add automated pytest github actions workflow
Student 4 CI Pipeline #6: Commit e4503b6 pushed by Skyexodus
syk-student4
Aug 30, 12:15 AM GMT+10
46s
feat(student-4): finalize backend ai handling, error templates, and a...
Student 4 CI Pipeline #5: Commit 378b8ee pushed by Skyexodus
syk-student4
Aug 26, 11:34 PM GMT+10
18s
fix(student-4): update backend ollama fallback and error handling
Student 4 CI Pipeline #4: Commit ddabd13 pushed by Skyexodus
syk-student4
Aug 26, 11:32 PM GMT+10
16s
feat(student-4): implement complete cart & orders microservice stack
Student 4 CI Pipeline #3: Commit b6d15b5 pushed by Skyexodus
syk-student4
Aug 23, 1:15 AM GMT+10
18s
Delete OmniTech directory
Student 4 CI Pipeline #2: Commit 8585b20 pushed by Skyexodus
syk-student4
Aug 23, 12:51 AM GMT+10
Failure
feat(student-4): implement cart and order processing microservice and...
Student 4 CI Pipeline #1: Commit e0e0fec pushed by Skyexodus
syk-student4
Aug 23, 12:44 AM GMT+10
Failure


Student 4 CI Pipeline
Filter workflow runs
8 workflow runs
Event
Status
Branch
Actor
fix(student-4): swap ports (5004=cart page, 5014=database API) to mat...
Student 4 CI Pipeline #8: Commit 43c9410 pushed by Skyexodus
syk-student4
Sep 4, 11:49 AM GMT+10
36s
fix(student-4): swap ports so 5004 serves the cart page (matches team...
Student 4 CI Pipeline #7: Commit a6b5f54 pushed by Skyexodus
syk-student4
Sep 4, 10:58 AM GMT+10
32s
trying to match student 3 style
Student 4 CI Pipeline #6: Commit 3551be4 pushed by kr0stheline
feature/student-1-catalog
Sep 4, 7:14 AM GMT+10
35s
Merge pull request #4 from Pizzalilla/syk-student4
Student 4 CI Pipeline #5: Commit 761d3af pushed by kr0stheline
main
Sep 4, 6:28 AM GMT+10
36s
Syk student4
Student 4 CI Pipeline #4: Pull request #4 opened by Skyexodus
syk-student4
Sep 4, 1:34 AM GMT+10
39s
fix(student-4): bind Flask services to 0.0.0.0 so Docker can reach th...
Student 4 CI Pipeline #3: Commit 646811e pushed by Skyexodus
syk-student4
Sep 4, 1:20 AM GMT+10
38s
feat(student-4): update appliance items, au metric prompts, dockerfil...
Student 4 CI Pipeline #2: Commit c2023a5 pushed by Skyexodus
syk-student4
Sep 3, 8:34 PM GMT+10
37s
Merge branch 'syk-student4'
Student 4 CI Pipeline #1: Commit 6ee736c pushed by Skyexodus
main
Sep 3, 8:32 PM GMT+10
32s

Student 5


Student 5 CI
student-5.yml
Filter workflow runs
9 workflow runs
Event
Status
Branch
Actor
This workflow has a workflow_dispatch event trigger.
Run workflow
Use llama3.2 as the shared Ollama
Student 5 CI #9: Commit 8c8a5b4 pushed by kr0stheline
main
Today at 12:50 PM
38s
Merge pull request #10 from Pizzalilla/student5-warranty
Student 5 CI #8: Commit 2f17dd2 pushed by BrianTranGit
main
Today at 11:54 AM
36s
added ticket remove functionality
Student 5 CI #7: Pull request #10 opened by BrianTranGit
student5-warranty
Today at 11:54 AM
36s
Merge pull request #9 from Pizzalilla/student5-warranty
Student 5 CI #6: Commit 80fac2f pushed by BrianTranGit
main
Today at 10:41 AM
35s
Student5 warranty
Student 5 CI #5: Pull request #9 synchronize by BrianTranGit
student5-warranty
Today at 10:31 AM
35s
Student5 warranty
Student 5 CI #4: Pull request #9 synchronize by BrianTranGit
student5-warranty
Today at 9:58 AM
33s
Student5 warranty
Student 5 CI #3: Pull request #9 synchronize by BrianTranGit
student5-warranty
Sep 8, 5:27 PM GMT+10
39s
Student5 warranty
Student 5 CI #2: Pull request #9 synchronize by BrianTranGit
student5-warranty
Sep 8, 5:25 PM GMT+10
20s
Student5 warranty
Student 5 CI #1: Pull request #9 opened by BrianTranGit
student5-warranty
Sep 8, 4:55 PM GMT+10
17s





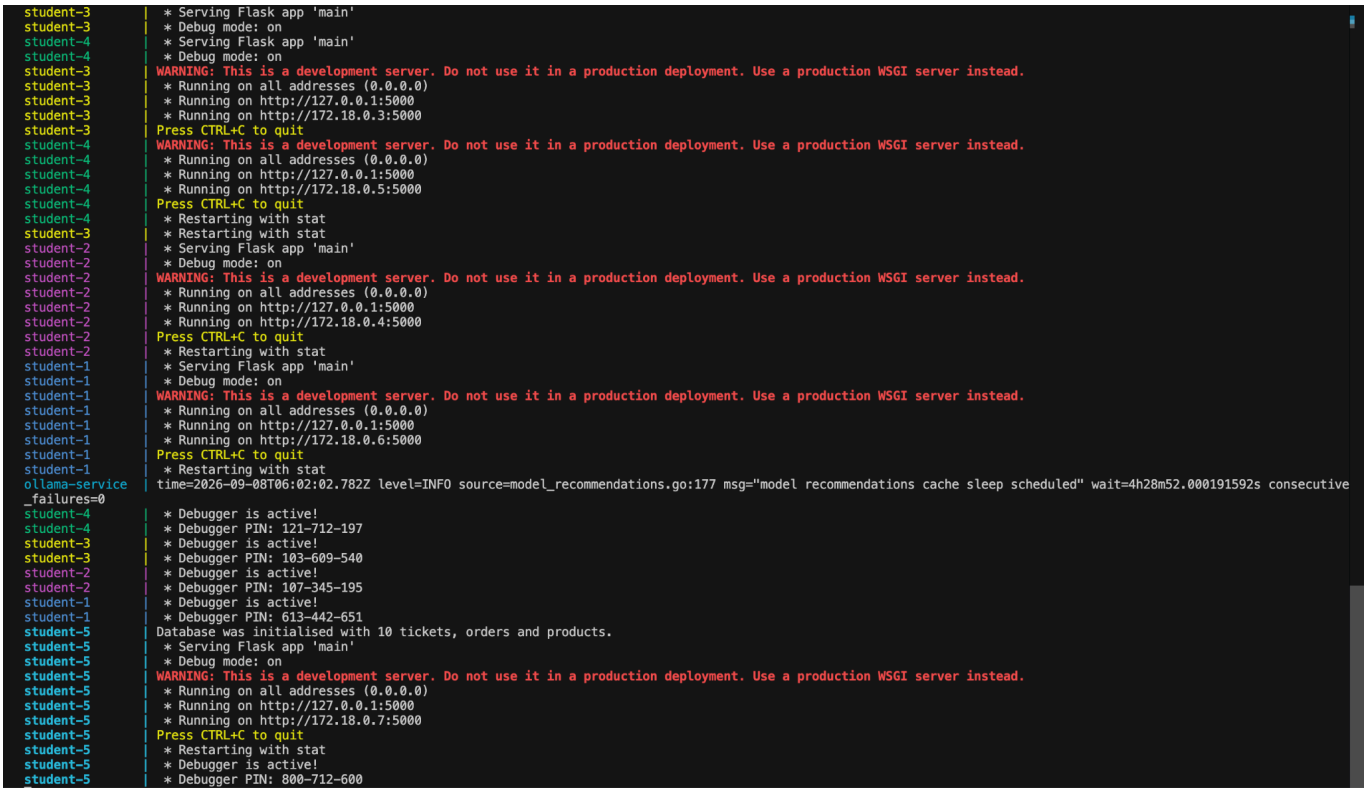
18. Docker Compose Execution Evidence

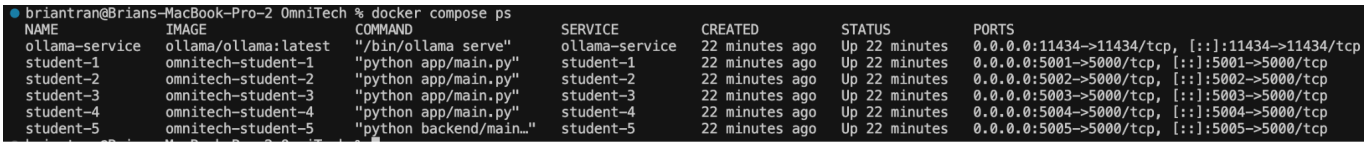
```
#47 [student-1] resolving provenance for metadata file
#47 DONE 0.0s
[+] up 12/12
 ✓ Image omnitech-student-1 Built                                                          2.4s
 ✓ Image omnitech-student-2 Built                                                          2.5s
 ✓ Image omnitech-student-3 Built                                                          2.5s
 ✓ Image omnitech-student-4 Built                                                          2.4s
 ✓ Image omnitech-student-5 Built                                                          2.4s
 ✓ Container omnitech-home  Running                                                        0.0s
 ✓ Container student-2      Running                                                        0.0s
 ✓ Container student-3      Running                                                        0.0s
 ✓ Container student-4      Running                                                        0.0s
 ✓ Container student-5      Running                                                        0.0s
 ✓ Container ollama-service Running                                                        0.0s
 ✓ Container student-1      Started                                                        3.6s
NAME             IMAGE                                                                     COMMAND                    SERVICE          CREATED          STATUS                  PORTS
ollama-service   ollama/ollama:latest                                                      "/bin/ollama serve"        ollama-service   5 hours ago      Up 2 hours              0.0.0.0:11434->1
1434/tcp, [::]:11434->11434/tcp
omnitech-home    nginx:alpine                                                              "/docker-entrypoint.…"     home             5 hours ago      Up 2 hours              0.0.0.0:8080->80
/tcp, [::]:8080->80/tcp
student-1        omnitech-student-1                                                        "python backend/app.…"     student-1        4 seconds ago    Up Less than a second   0.0.0.0:5001->50
00/tcp, [::]:5001->5000/tcp
student-2        sha256:9d86bdc476682735a0f88047d5a37e997a1dc5a5475ba2f4584a04d10a9c7484   "python backend/main…"     student-2        4 hours ago      Up 2 hours              0.0.0.0:5002->50
00/tcp, [::]:5002->5000/tcp
student-3        sha256:b44402760b2a7258d51e3d691f75b4ecbc5ba7dbd71f44018281745fc2f08436   "python backend/main…"     student-3        4 hours ago      Up 2 hours              0.0.0.0:5003->50
00/tcp, [::]:5003->5000/tcp
student-4        sha256:17077293c700ffe94cd1128107f79bb875977b5b5e1f19f94501f7c06c2bacde   "/bin/sh -c 'python …"     student-4        5 hours ago      Up 2 hours              0.0.0.0:5004->50
04/tcp, [::]:5004->5004/tcp, 0.0.0.0:5014->5014/tcp, [::]:5014->5014/tcp
student-5        sha256:448f93ec44b0f7b198a58ab18d762ddb6101bb8af178c6014970978e0f7f87a1   "python backend/main…"     student-5        4 hours ago      Up 2 hours              0.0.0.0:5005->50
00/tcp, [::]:5005->5000/tcp
PS C:\Users\krisn\Downloads\ASD\OmniTech>
```

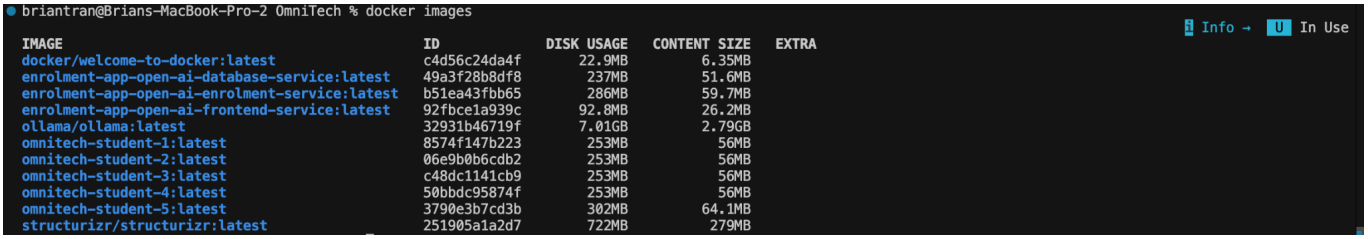
```
PS C:\Users\krisn\Downloads\ASD\OmniTech> docker compose ps
NAME              IMAGE                                                                COMMAND                  SERVICE          CREATED          STATUS          PORTS
ollama-service    ollama/ollama:latest                                                 "/bin/ollama serve"      ollama-service   5 hours ago      Up 2 hours      0.0.0.0:11434->11434/tcp, [::]:11434->11434/tcp
omnitech-home     nginx:alpine                                                         "/docker-entrypoint.…"   home             5 hours ago      Up 2 hours      0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
student-1         sha256:5766ecba20116715a19f9c5c6c65ce1643689ab922caca8771fd093841cac090   "python backend/app.…"   student-1        53 seconds ago   Up 49 seconds   0.0.0.0:5001->5000/tcp, [::]:5001->5000/tcp
student-2         sha256:9d86bdc476682735a0f88047d5a37e997a1dc5a5475ba2f4584a04d10a9c7484   "python backend/main…"   student-2        4 hours ago      Up 2 hours      0.0.0.0:5002->5000/tcp, [::]:5002->5000/tcp
student-3         sha256:b44402760b2a7258d51e3d691f75b4ecbc5ba7dbd71f44018281745fc2f08436   "python backend/main…"   student-3        4 hours ago      Up 2 hours      0.0.0.0:5003->5000/tcp, [::]:5003->5000/tcp
student-4         sha256:17077293c700ffe94cd1128107f79bb875977b5b5e1f19f94501f7c06c2bacde   "/bin/sh -c 'python …"   student-4        5 hours ago      Up 2 hours      0.0.0.0:5004->5004/tcp, [::]:5004->5004/tcp, 0.0
.0.0:5014->5014/tcp, [::]:5014->5014/tcp
student-5         sha256:448f93ec44b0f7b198a58ab18d762ddb6101bb8af178c6014970978e0f7f87a1   "python backend/main…"   student-5        4 hours ago      Up 2 hours      0.0.0.0:5005->5000/tcp, [::]:5005->5000/tcp
PS C:\Users\krisn\Downloads\ASD\OmniTech>
```

```
briantran@Brians-MacBook-Pro-2 OmniTech % docker compose up --build
 [+] up 5/5
  ✔ Image ollama/ollama:latest Pulled                                                                                                                  478.6s
 [+] Building 14.9s (44/44) FINISHED
```

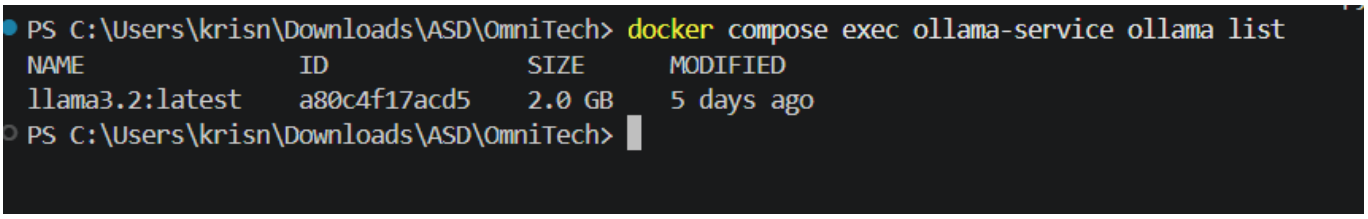
```
 [+] up 18/18king to docker.io/library/omnitech-student-5:latest                                                                                       0.3s
  ✔ Image ollama/ollama:latest    Pulled                                                                                                             478.6s
  ✔ Image omnitech-student-1      Built                                                                                                               15.0s
  ✔ Image omnitech-student-2      Built                                                                                                               15.0s
  ✔ Image omnitech-student-3      Built                                                                                                               15.0s
  ✔ Image omnitech-student-4      Built                                                                                                               15.0s
  ✔ Image omnitech-student-5      Built                                                                                                               15.0s
  ✔ Network omnitech_omnitech-net Created                                                                                                              0.0s
  ✔ Volume omnitech_ollama-data   Created                                                                                                              0.0s
  ✔ Container ollama-service      Created                                                                                                              0.1s
  ✔ Container student-5           Created                                                                                                              0.1s
  ✔ Container student-2           Created                                                                                                              0.1s
  ✔ Container student-4           Created                                                                                                              0.0s
  ✔ Container student-3           Created                                                                                                              0.1s
  ✔ Container student-1           Created                                                                                                              0.1s
Attaching to ollama-service, student-1, student-2, student-3, student-4, student-5
ollama-service  | Couldn't find '/root/.ollama/id_ed25519'. Generating new private key.
ollama-service  | Your new public key is:
ollama-service  |
ollama-service  | ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIAksMa8rs6gzv3+5zouhpDCSeuYl9ALXSa7EH7a9lqKM
ollama-service  |
ollama-service  | time=2026-09-08T06:02:02.450Z level=INFO source=routes.go:1955 msg="server config" env="map[CUDA_VISIBLE_DEVICES: GGML_VK_VISIBLE_DEVICES: GPU_DEVICE_ORDINAL: HIP
_VISIBLE_DEVICES: HSA_OVERRIDE_GFX_VERSION: HTTPS_PROXY: HTTP_PROXY: LLAMA_ARG_FIT: LLAMA_ARG_FIT_TARGET: NO_PROXY: OLLAMA_CONTEXT_LENGTH:0 OLLAMA_DEBUG:INFO OLLAMA_DEBUG_LOG_REQUE
STS:false OLLAMA_EDITOR: OLLAMA_FLASH_ATTENTION:false OLLAMA_GO_TEMPLATE:true OLLAMA_GPU_OVERHEAD:0 OLLAMA_HOST:http://0.0.0.0:11434 OLLAMA_IGPU_ENABLE: OLLAMA_KEEP_ALIVE:5m0s OLLA
MA_KV_CACHE_TYPE: OLLAMA_LLM_LIBRARY: OLLAMA_LOAD_TIMEOUT:5m0s OLLAMA_MAX_LOADED_MODELS:0 OLLAMA_MAX_QUEUE:512 OLLAMA_MAX_TRANSFER_STREAMS:4 OLLAMA_MODELS:/root/.ollama/models OLLA
MA_NOHISTORY:false OLLAMA_NOPRUNE:false OLLAMA_NO_CLOUD:false OLLAMA_NUM_PARALLEL:1 OLLAMA_ORIGINS:[http://localhost https://localhost http://localhost:* https://localhost:* http:/
/127.0.0.1 https://127.0.0.1 http://127.0.0.1:* https://127.0.0.1:* http://0.0.0.0 https://0.0.0.0 http://0.0.0.0:* https://0.0.0.0:* app://* file://* tauri://* vscode-webview://*
vscode-file://*] OLLAMA_REMOTES:[ollama.com] OLLAMA_SCHED_SPREAD:false OLLAMA_VULKAN:true ROCR_VISIBLE_DEVICES: http_proxy: https_proxy: no_proxy:]"
ollama-service  | time=2026-09-08T06:02:02.450Z level=INFO source=routes.go:1957 msg="Ollama cloud disabled: false"
ollama-service  | time=2026-09-08T06:02:02.451Z level=INFO source=images.go:957 msg="total blobs: 0"
ollama-service  | time=2026-09-08T06:02:02.451Z level=INFO source=images.go:964 msg="total unused blobs removed: 0"
ollama-service  | time=2026-09-08T06:02:02.451Z level=INFO source=routes.go:2012 msg="Listening on [::]:11434 (version 0.33.3)"
ollama-service  | time=2026-09-08T06:02:02.452Z level=INFO source=runner.go:60 msg="discovering available GPUs..."
ollama-service  | time=2026-09-08T06:02:02.452Z level=INFO source=model_list_cache.go:112 msg="model list cache hydration complete" models=0 failures=0 elapsed=951.416µs
ollama-service  | time=2026-09-08T06:02:02.522Z level=INFO source=types.go:50 msg="inference compute" id=cpu library=cpu compute="" name=cpu description=cpu libdirs=ollama driver="
" pci_id="" type="" total="7.7 GiB" available="7.7 GiB"
ollama-service  | time=2026-09-08T06:02:02.522Z level=INFO source=routes.go:2062 msg="vram-based default context" total_vram="0 B" default_num_ctx=4096
```



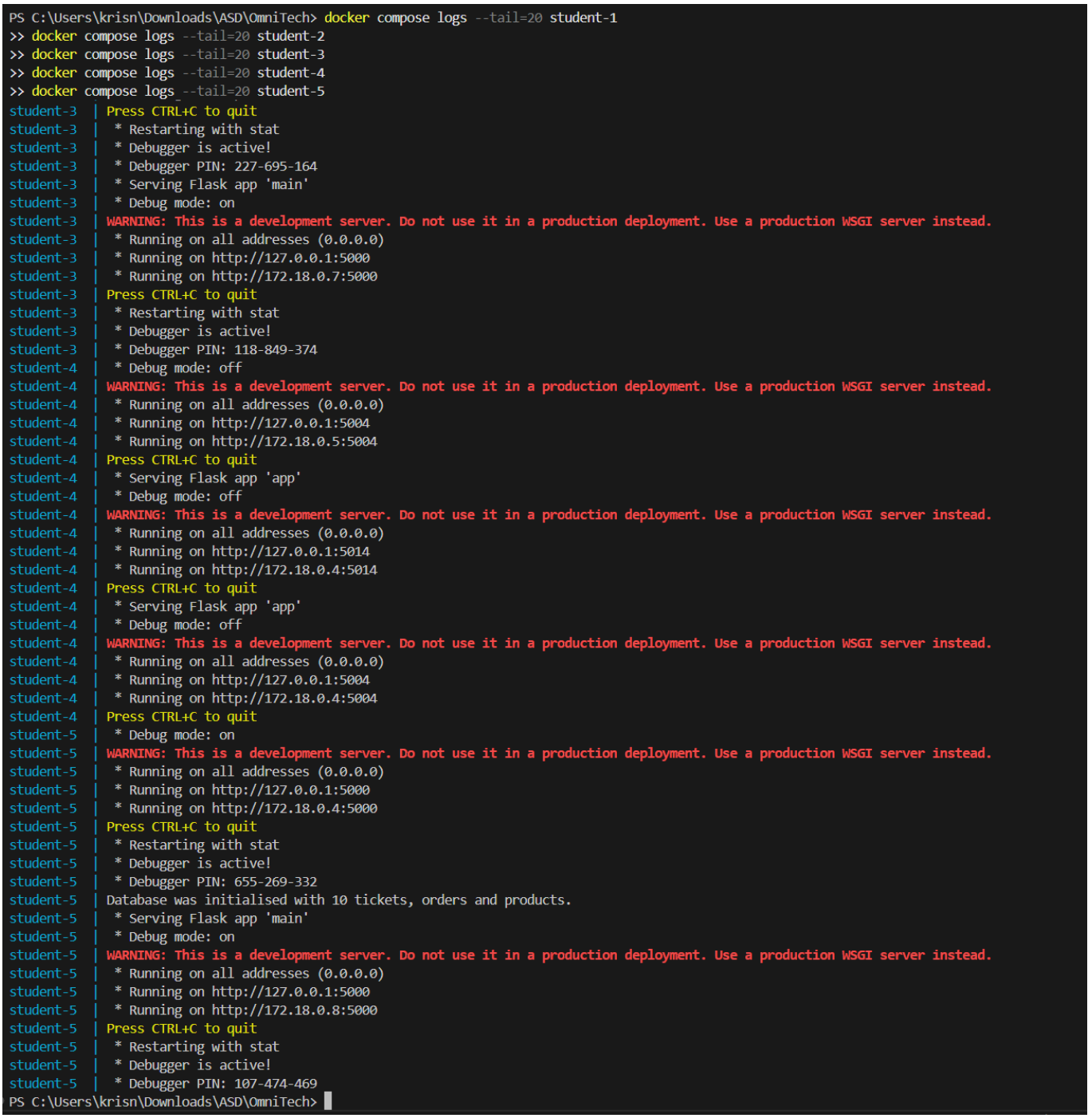




Ollama evidence:



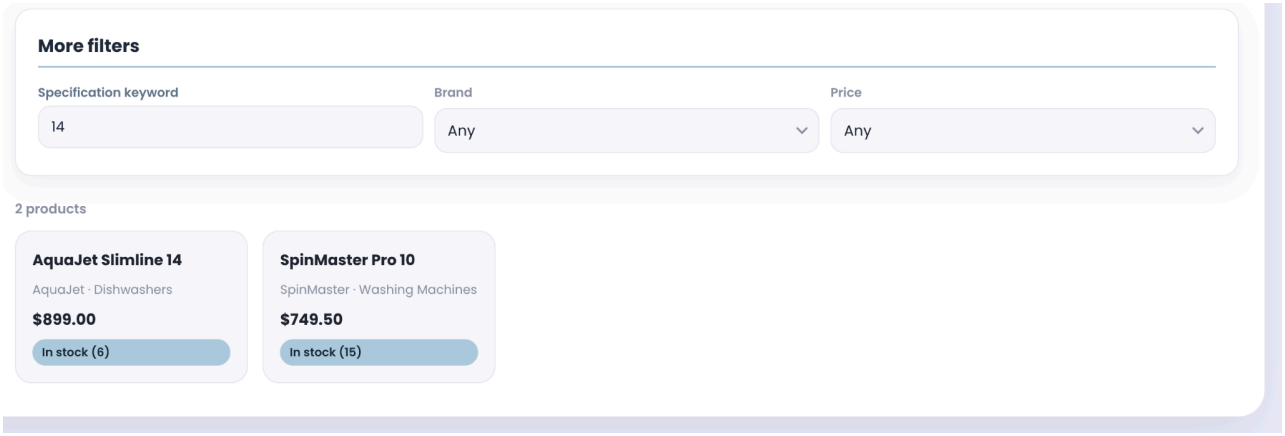
Individual containers:


PS C:\Users\krisn\Downloads\ASD\OmniTech> docker compose logs --tail=20 student-1
>> docker compose logs --tail=20 student-2
>> docker compose logs --tail=20 student-3
>> docker compose logs --tail=20 student-4
>> docker compose logs --tail=20 student-5
student-3 | Press CTRL+C to quit
student-3 | * Restarting with stat
student-3 | * Debugger is active!
student-3 | * Debugger PIN: 227-695-164
student-3 | * Serving Flask app 'main'
student-3 | * Debug mode: on
student-3 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
student-3 | * Running on all addresses (0.0.0.0)
student-3 | * Running on http://127.0.0.1:5000
student-3 | * Running on http://172.18.0.7:5000
student-3 | Press CTRL+C to quit
student-3 | * Restarting with stat
student-3 | * Debugger is active!
student-3 | * Debugger PIN: 118-849-374
student-4 | * Debug mode: off
student-4 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
student-4 | * Running on all addresses (0.0.0.0)
student-4 | * Running on http://127.0.0.1:5004
student-4 | * Running on http://172.18.0.5:5004
student-4 | Press CTRL+C to quit
student-4 | * Serving Flask app 'app'
student-4 | * Debug mode: off
student-4 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
student-4 | * Running on all addresses (0.0.0.0)
student-4 | * Running on http://127.0.0.1:5014
student-4 | * Running on http://172.18.0.4:5014
student-4 | Press CTRL+C to quit
student-4 | * Serving Flask app 'app'
student-4 | * Debug mode: off
student-4 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
student-4 | * Running on all addresses (0.0.0.0)
student-4 | * Running on http://127.0.0.1:5004
student-4 | * Running on http://172.18.0.4:5004
student-4 | Press CTRL+C to quit
student-5 | * Debug mode: on
student-5 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
student-5 | * Running on all addresses (0.0.0.0)
student-5 | * Running on http://127.0.0.1:5000
student-5 | * Running on http://172.18.0.4:5000
student-5 | Press CTRL+C to quit
student-5 | * Restarting with stat
student-5 | * Debugger is active!
student-5 | * Debugger PIN: 655-269-332
student-5 | Database was initialised with 10 tickets, orders and products.
student-5 | * Serving Flask app 'main'
student-5 | * Debug mode: on
student-5 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
student-5 | * Running on all addresses (0.0.0.0)
student-5 | * Running on http://127.0.0.1:5000
student-5 | * Running on http://172.18.0.8:5000
student-5 | Press CTRL+C to quit
student-5 | * Restarting with stat
student-5 | * Debugger is active!
student-5 | * Debugger PIN: 107-474-469
PS C:\Users\krisn\Downloads\ASD\OmniTech>

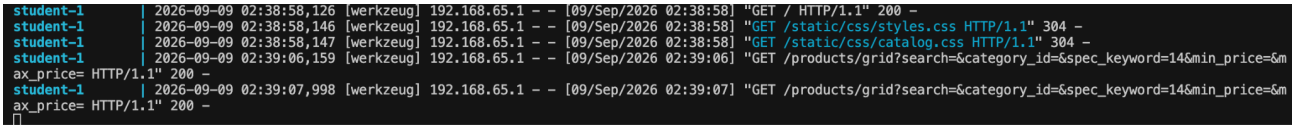
19. Screenshots of the Integrated Application

Student 1

These first 2 screenshots show that the filters work.

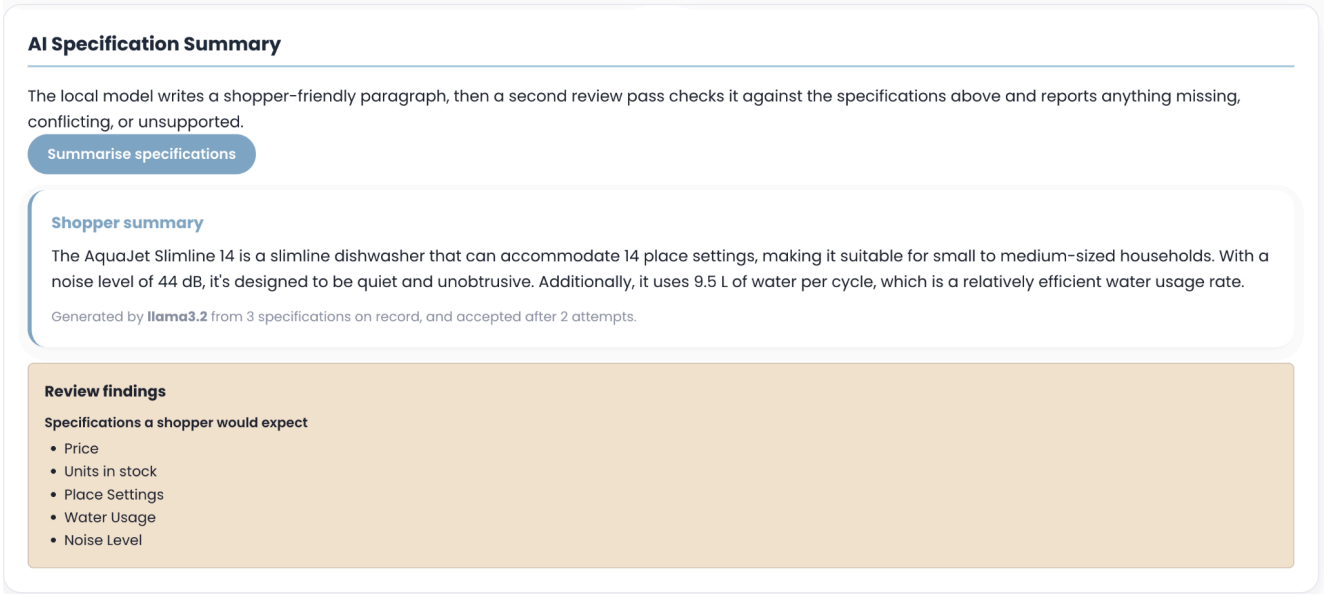


Screenshot 1

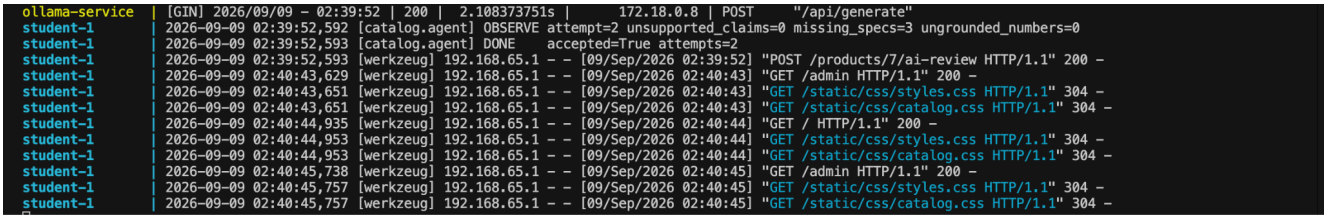


Screenshot 2

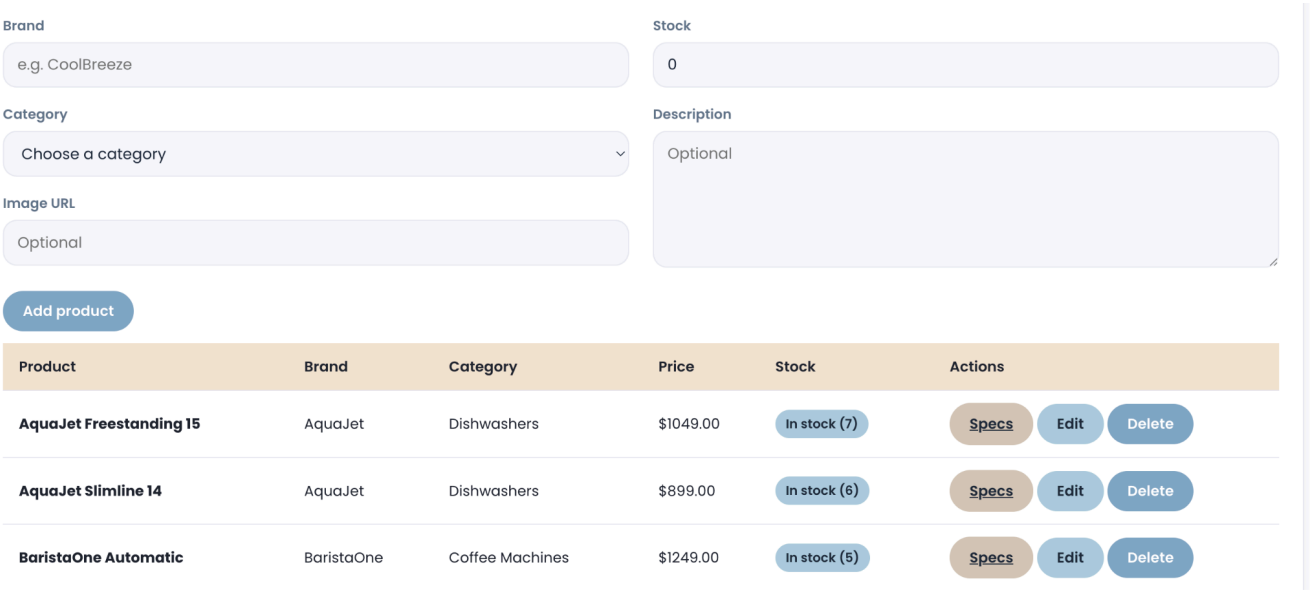
Screenshots 3 and 4 show the AI summarising AquaJet Slimline 14's specifications into a summary.



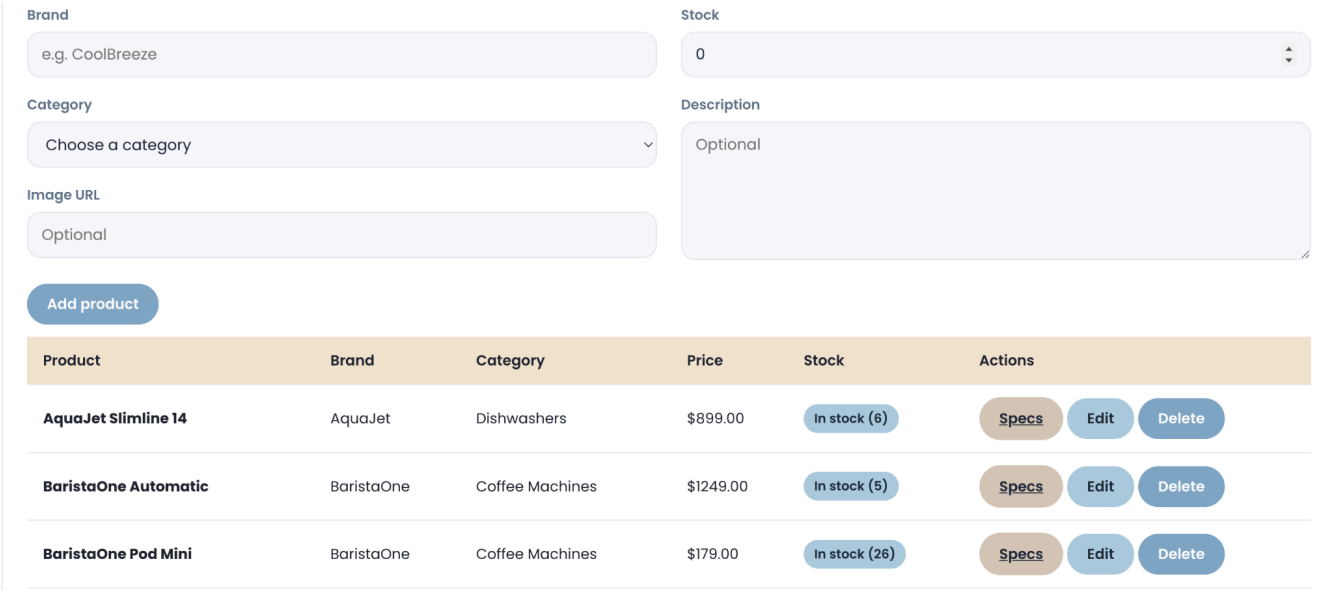
Screenshot 3



Screenshot 4

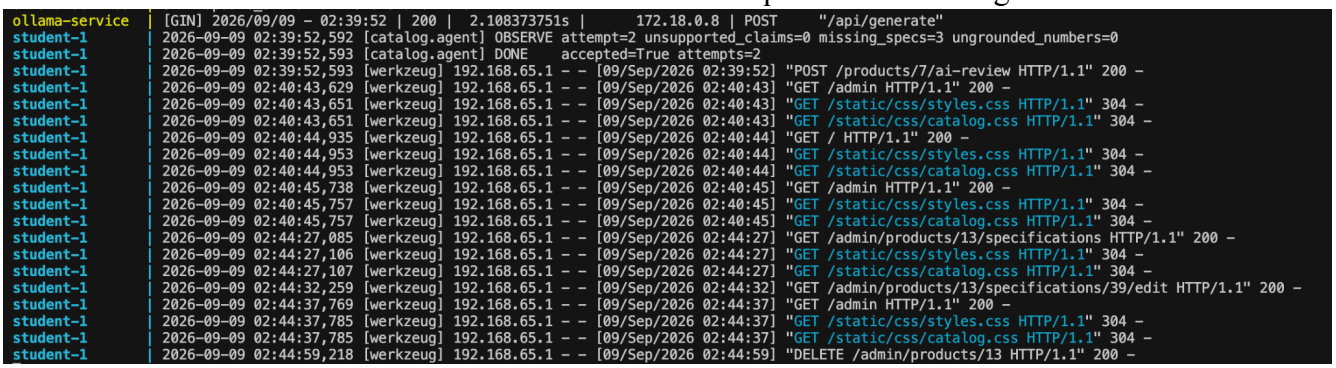


Screenshot 5

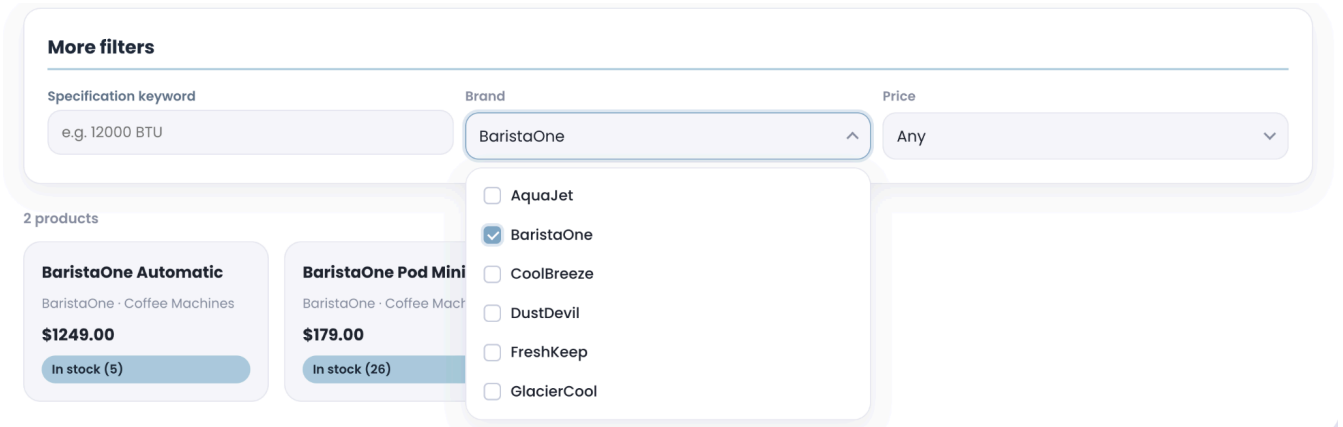


Screenshot 6

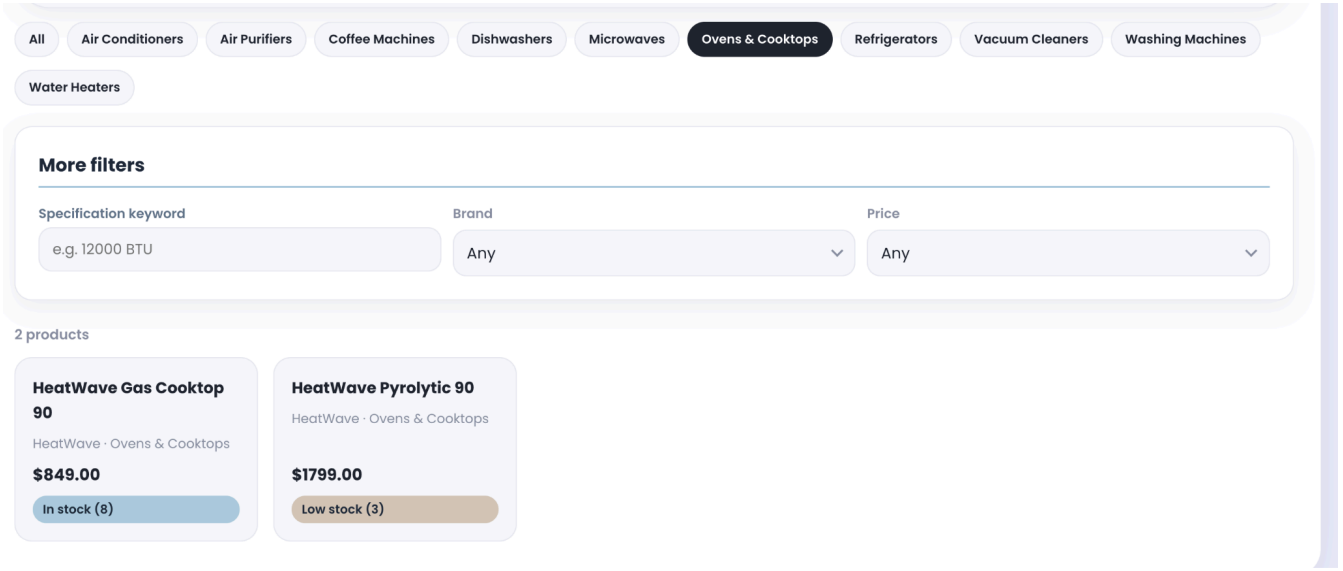
Screenshots 5 and 6 show the delete button works where AquaJet Freestanding 15 is removed



Screenshot 7



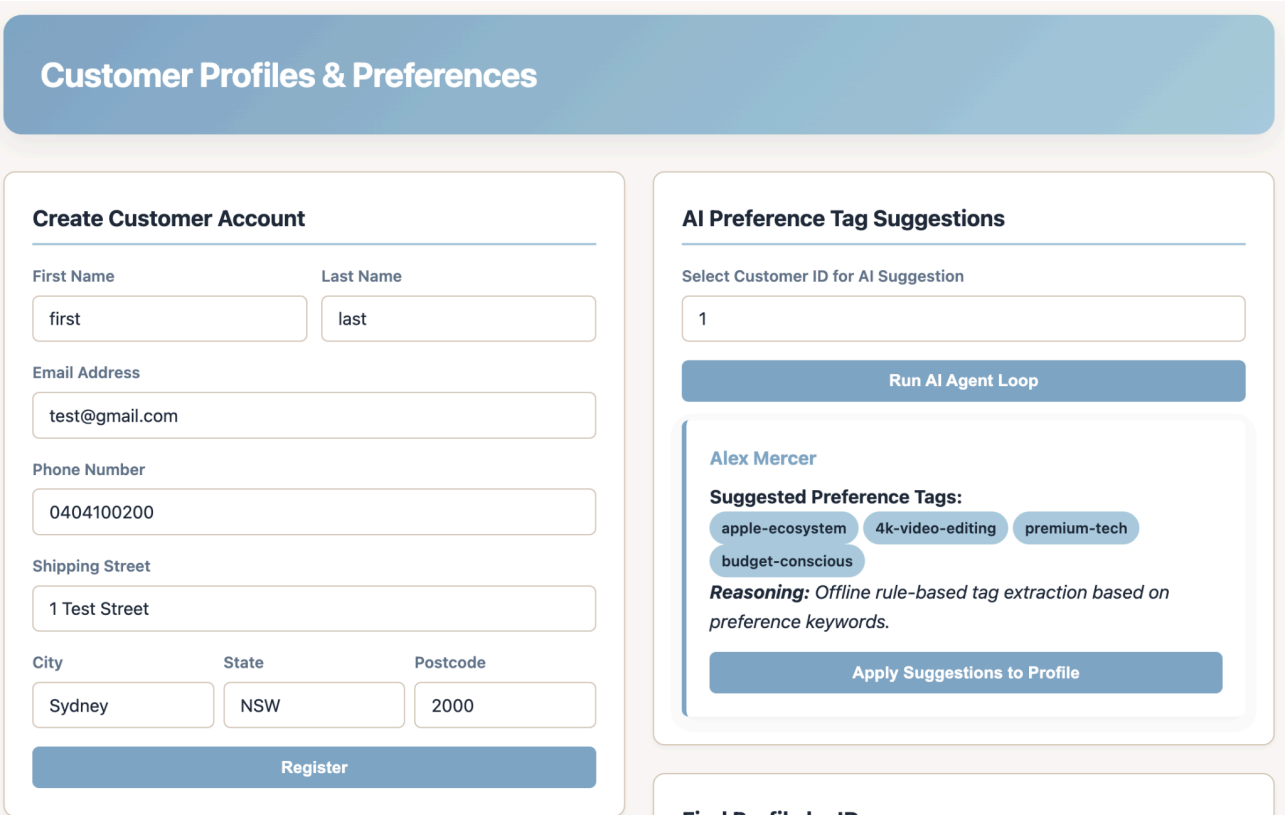
Screenshot 8



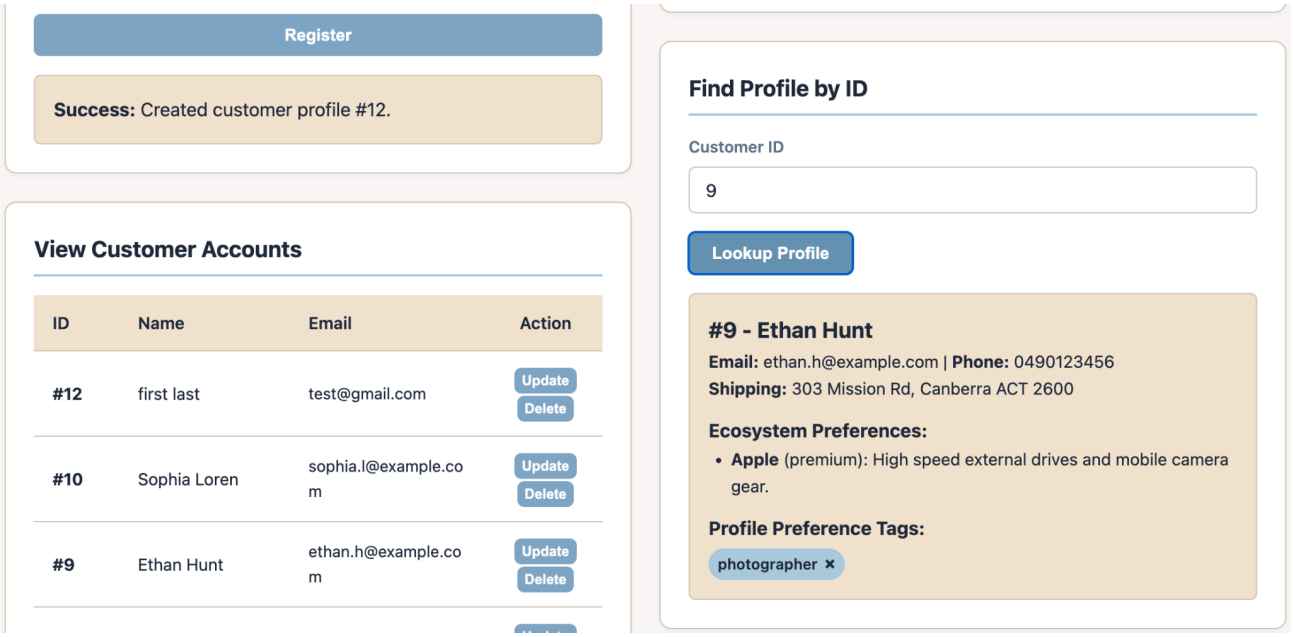
Screenshot 9

Screenshots 8 and 9 show the filters in work where you can filter by brand and also the top categories bar.

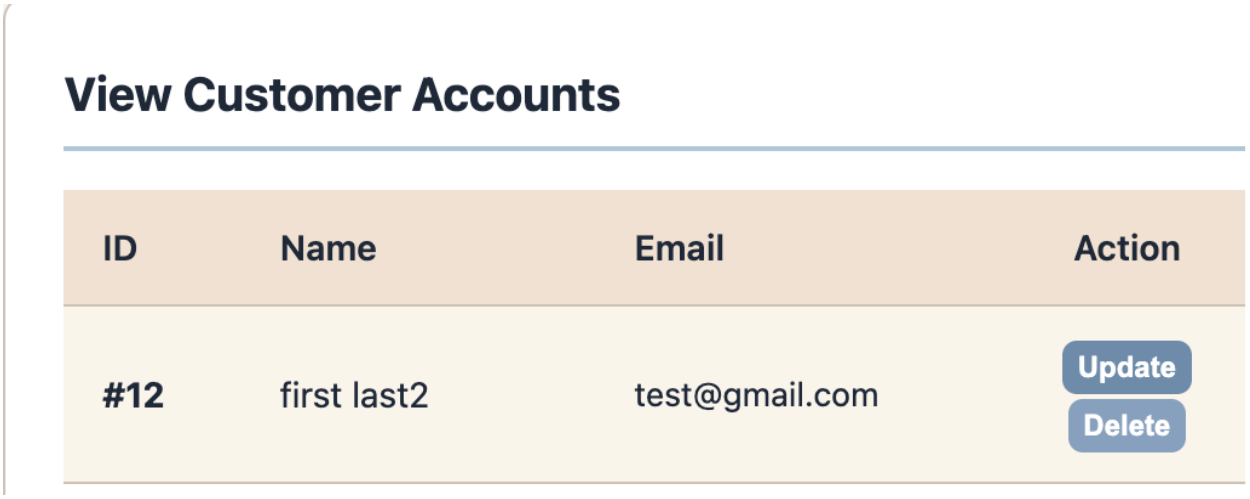
Student 2



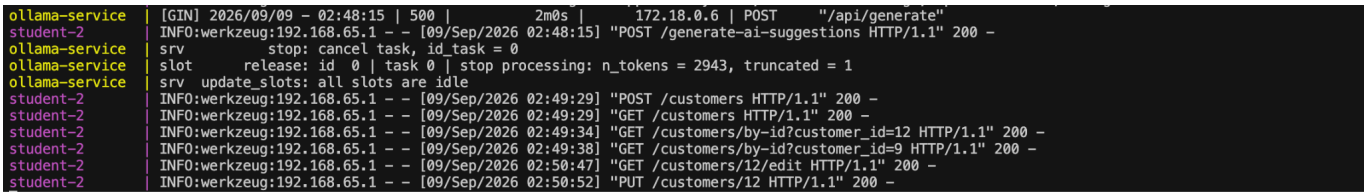
Screenshot 1 - shows the action of creating a new customer account and the AI preference tag suggestion working.



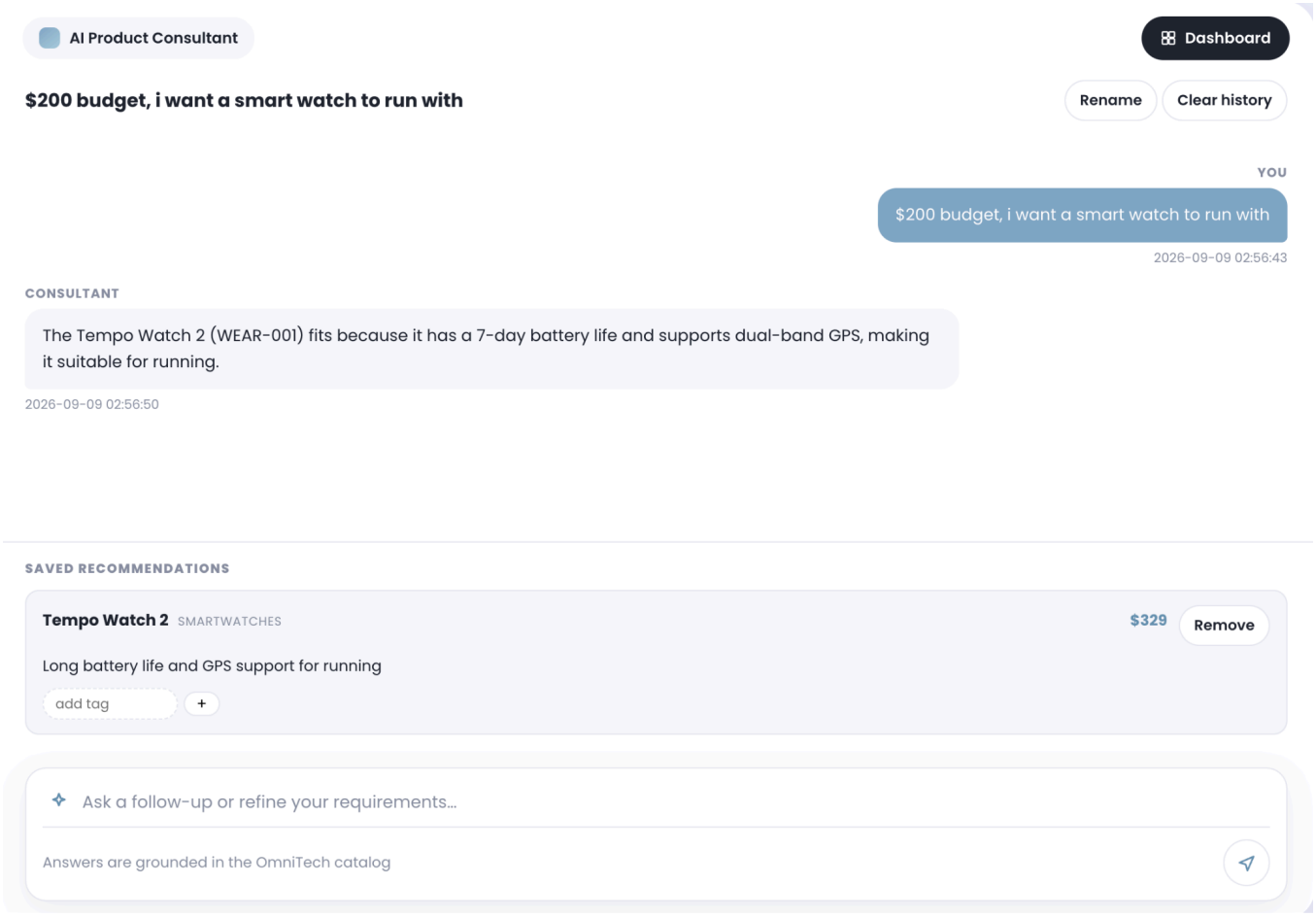
Screenshot 2 - shows the view customer accounts table is updated without reloading and shows the new customer as well as the customer profile id lookup displaying the customer's details and preference tags.



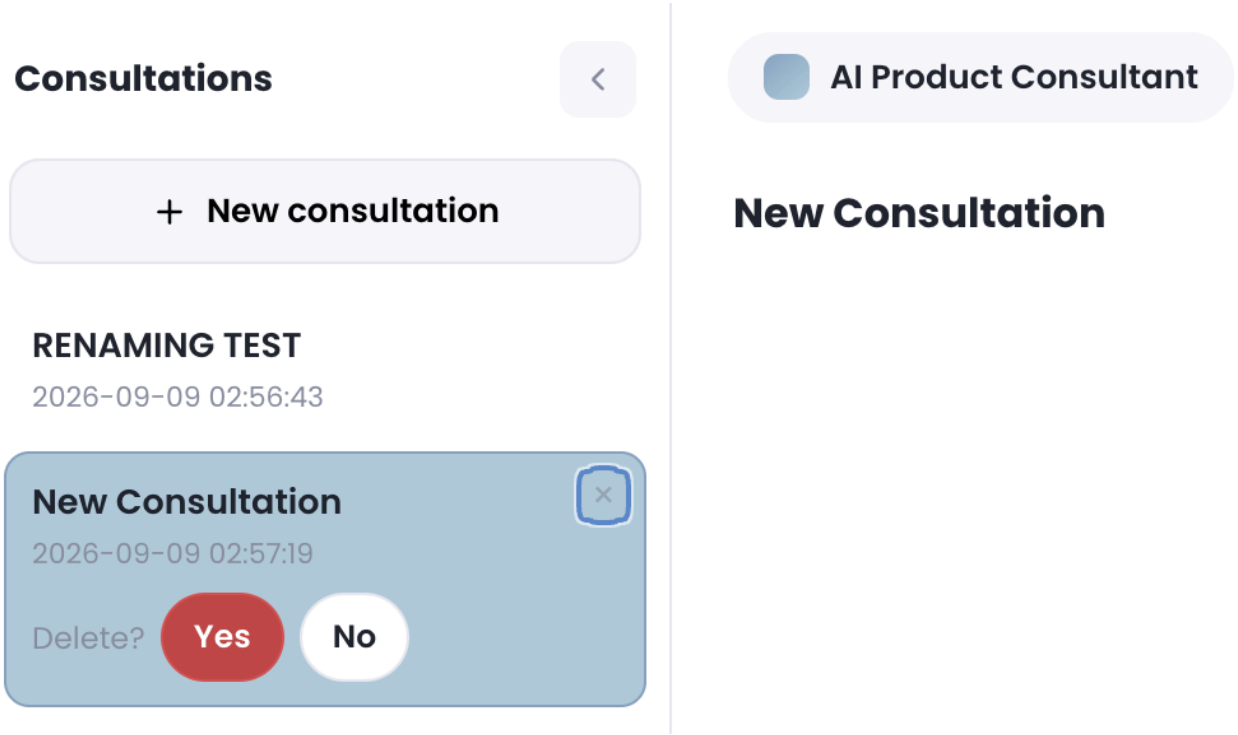
Screenshot 3 - shows how There is an update button in which I've updated the lastname to last2 to show it works.



Student 3

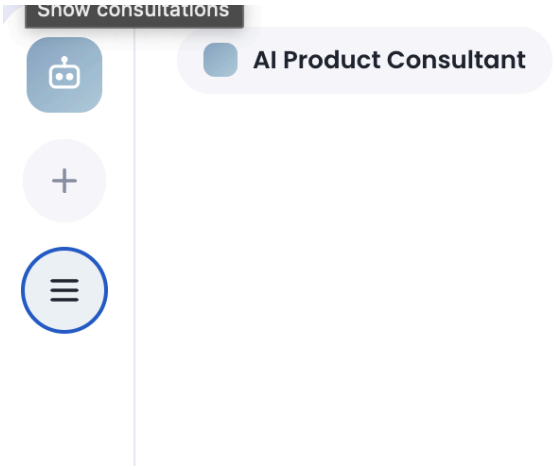


Screenshot 1



Screenshot 2

Screenshots 1 and 2 show the new chat session created when entering where the AI consultant gives a recommendation based on my budget and needs. Screenshot 2 specifically shows you can delete and rename a chat session as well as the chat history.



Screenshot 3

Screenshot 3 shows the triple lines icon button collapses the chat sessions bar for a cleaner aesthetic if the user wants.

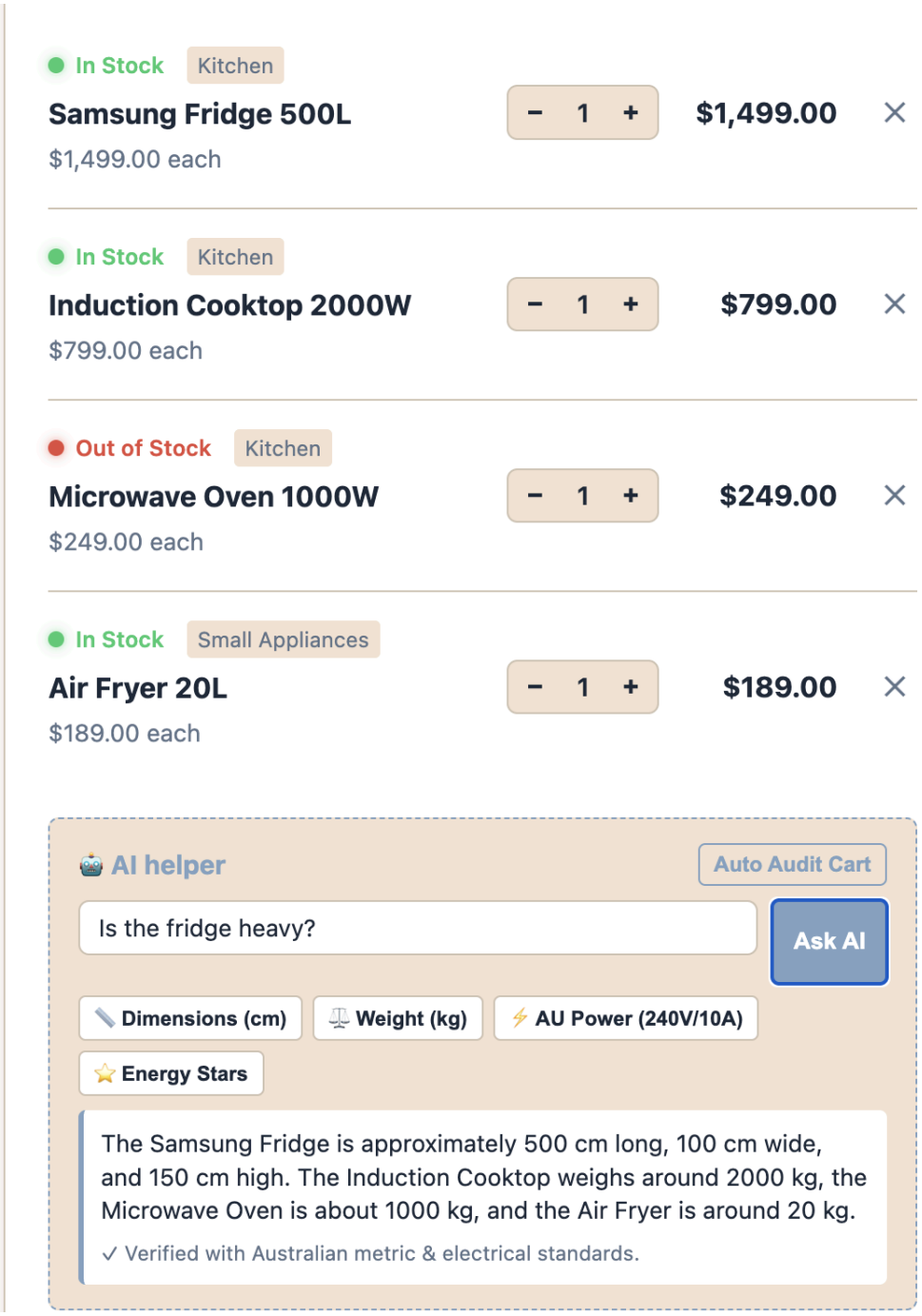
```
ollama-service | [GIN] 2026/09/09 - 02:56:50 | 200 |  7.591297462s |      172.18.0.5 | POST     "/api/generate"
student-3      | 2026-09-09 02:56:50,921  agent  ACT    ollama replied in 7.6s (243 chars)
student-3      | 2026-09-09 02:56:50,921  agent  OBSERVE ok=True ids=['WEAR-001'] issues=[]
student-3      | 2026-09-09 02:56:50,921  agent  DONE   attempts=1 reprompts=0 ids=['WEAR-001']
student-3      | 192.168.65.1 - - [09/Sep/2026 02:56:50] "POST /api/chat HTTP/1.1" 200 -
student-3      | 192.168.65.1 - - [09/Sep/2026 02:56:50] "GET /partials/session-list?active=13 HTTP/1.1" 200 -
student-3      | 192.168.65.1 - - [09/Sep/2026 02:56:50] "GET /partials/chat/13 HTTP/1.1" 200 -
student-3      | 192.168.65.1 - - [09/Sep/2026 02:57:14] "PUT /api/sessions/13 HTTP/1.1" 200 -
student-3      | 192.168.65.1 - - [09/Sep/2026 02:57:14] "GET /partials/chat/13 HTTP/1.1" 200 -
student-3      | 192.168.65.1 - - [09/Sep/2026 02:57:14] "GET /partials/session-list?active=13 HTTP/1.1" 200 -
student-3      | 192.168.65.1 - - [09/Sep/2026 02:57:19] "POST /api/sessions HTTP/1.1" 201 -
student-3      | 192.168.65.1 - - [09/Sep/2026 02:57:19] "GET /partials/session-list?active=14 HTTP/1.1" 200 -
student-3      | 192.168.65.1 - - [09/Sep/2026 02:57:19] "GET /partials/chat/14 HTTP/1.1" 200 -
student-3      | 192.168.65.1 - - [09/Sep/2026 02:57:21] "GET /partials/chat/13 HTTP/1.1" 200 -
student-3      | 192.168.65.1 - - [09/Sep/2026 02:57:27] "PUT /api/sessions/13 HTTP/1.1" 200 -
student-3      | 192.168.65.1 - - [09/Sep/2026 02:57:27] "GET /partials/chat/13 HTTP/1.1" 200 -
student-3      | 192.168.65.1 - - [09/Sep/2026 02:57:27] "GET /partials/session-list?active=13 HTTP/1.1" 200 -
student-3      | 192.168.65.1 - - [09/Sep/2026 02:57:28] "GET /partials/chat/14 HTTP/1.1" 200 -
student-3      | 192.168.65.1 - - [09/Sep/2026 02:57:40] "DELETE /api/sessions/14 HTTP/1.1" 200 -
student-3      | 192.168.65.1 - - [09/Sep/2026 02:57:40] "GET / HTTP/1.1" 200 -
student-3      | 192.168.65.1 - - [09/Sep/2026 02:57:40] "GET /static/style.css HTTP/1.1" 304 -
student-3      | 192.168.65.1 - - [09/Sep/2026 02:57:40] "GET /partials/session-list HTTP/1.1" 200 -
```

Screenshot 4

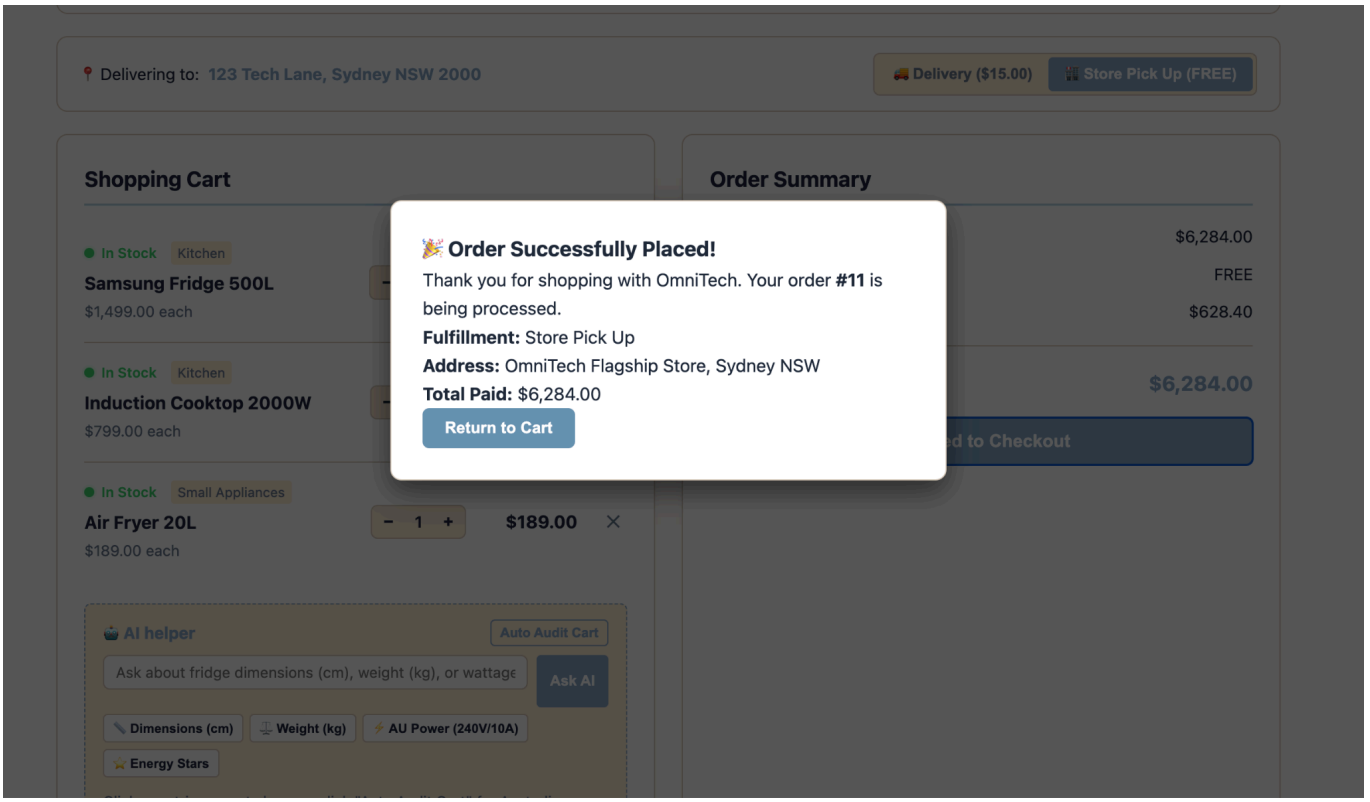
Student 4

```
PS C:\Users\tkddb\ASD\OmniTech> docker exec student-4 pytest tests/test_orders_api.py -v
============================= test session starts ==============================
platform linux -- Python 3.11.16, pytest-8.1.1, pluggy-1.6.0 -- /usr/local/bin/python3.11
cachedir: .pytest_cache
rootdir: /app
plugins: anyio-4.15.1
collecting ... collected 10 items

tests/test_orders_api.py::test_db_health PASSED                          [ 10%]
tests/test_orders_api.py::test_get_all_orders_count PASSED               [ 20%]
tests/test_orders_api.py::test_get_single_order_details PASSED           [ 30%]
tests/test_orders_api.py::test_order_not_found PASSED                    [ 40%]
tests/test_orders_api.py::test_backend_index_page PASSED                 [ 50%]
tests/test_orders_api.py::test_backend_cart_view_and_stock_indicators PASSED [ 60%]
tests/test_orders_api.py::test_cart_quantity_modification PASSED         [ 70%]
tests/test_orders_api.py::test_fulfillment_toggle PASSED                 [ 80%]
tests/test_orders_api.py::test_order_history_endpoint PASSED             [ 90%]
tests/test_orders_api.py::test_ai_helper_custom_question PASSED          [100%]

============================== 10 passed in 8.52s ==============================
PS C:\Users\tkddb\ASD\OmniTech>
```

Screenshot 1

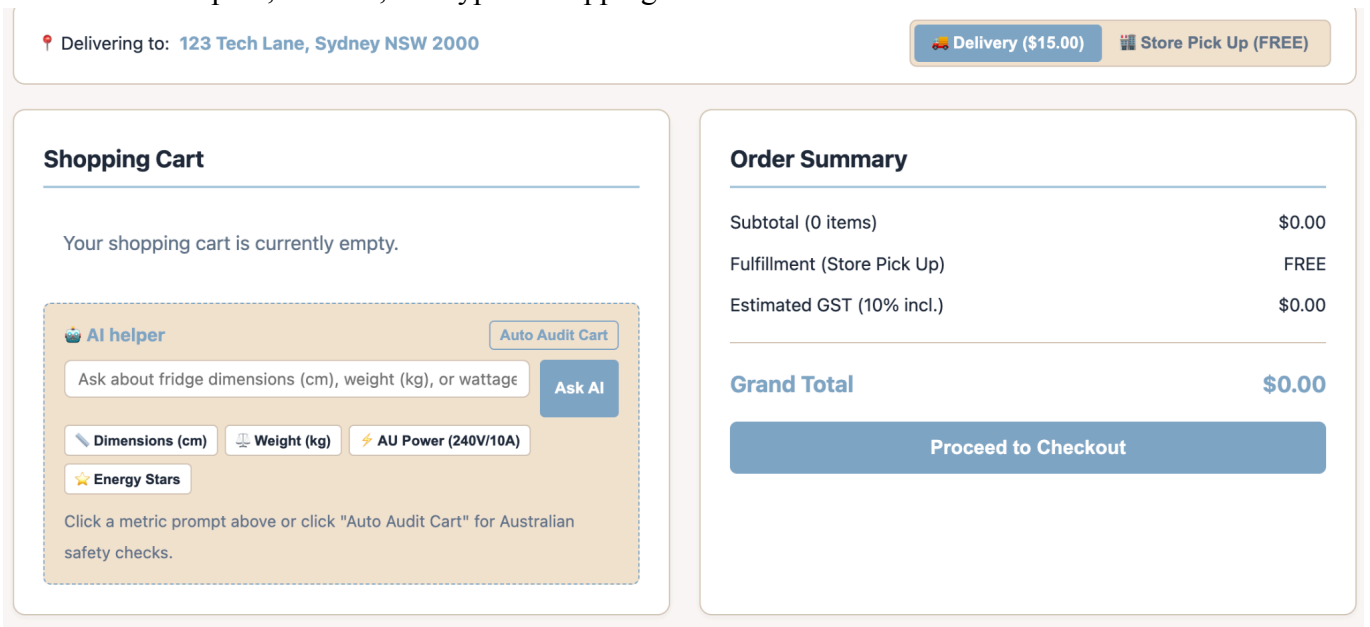


Screenshot 2 Screenshot 2 shows the AI helper answering my question where I ask, is the fridge heavy?

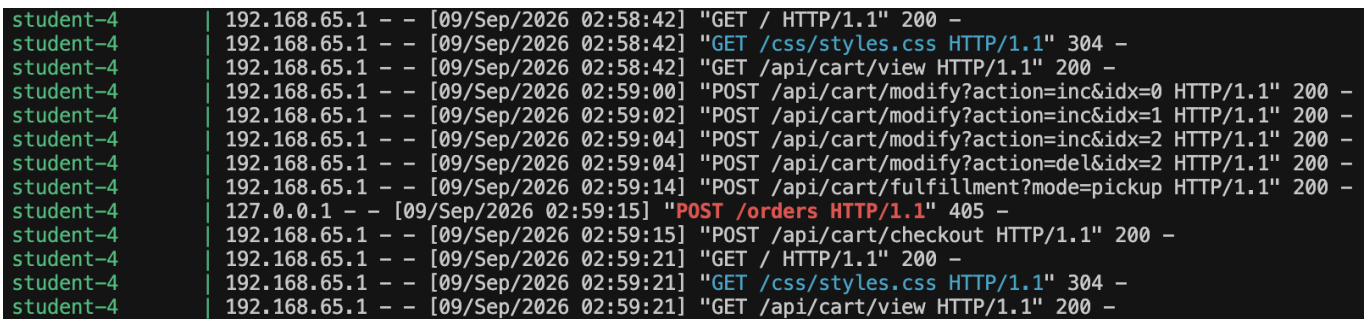


Screenshot 3

Screenshot 3 shows that when pressing proceed to checkout, the order is placed and it shows the order's details like total paid, address, and type of shipping method as well as the order id.



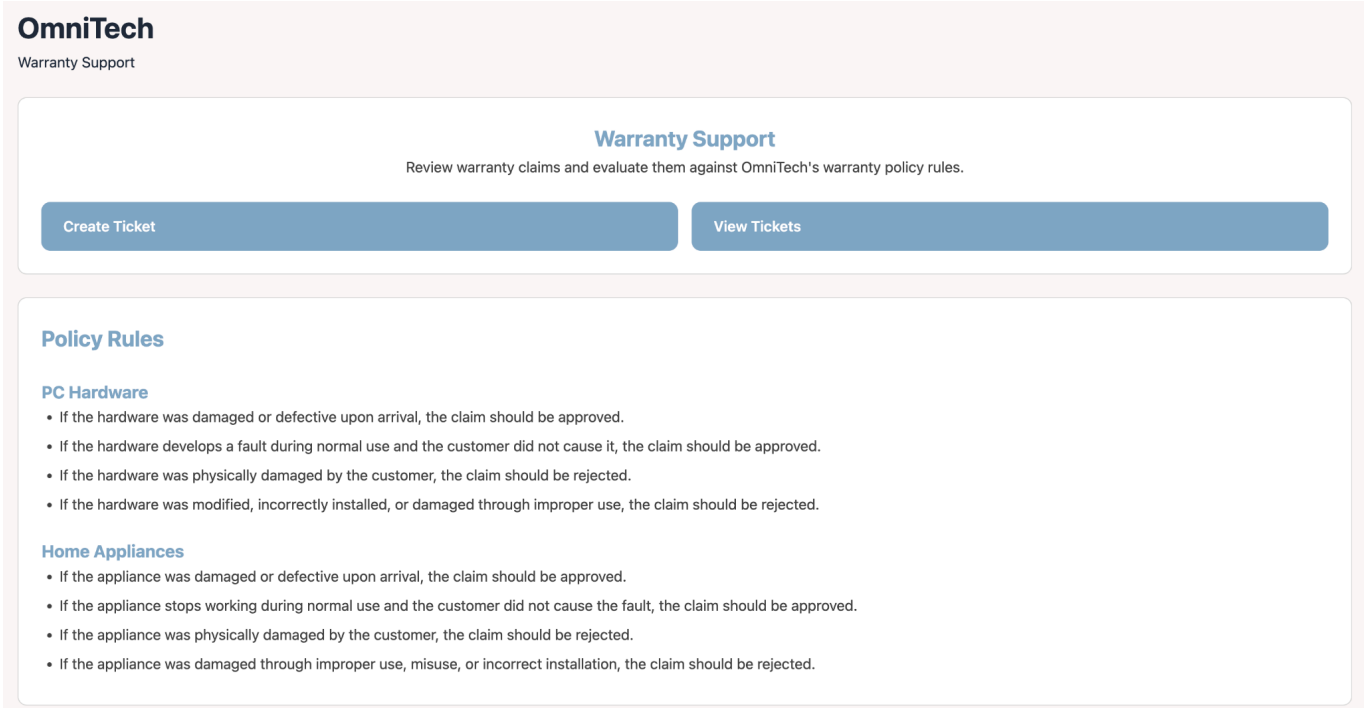
Screenshot 4



Screenshot 5

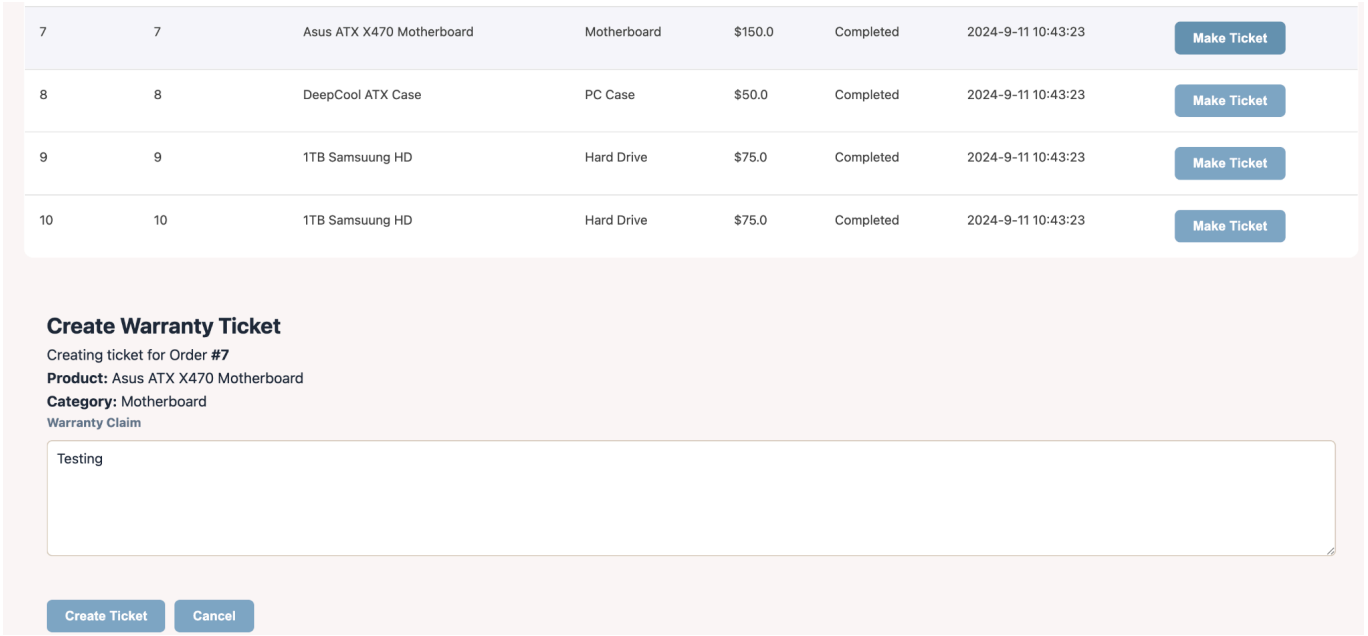
Screenshot 4 shows after ordering the cart is empty as the products in it has been assigned to an order. Screenshot 5 shows the cart modifying where I increased and decreased the quantity of the products in screenshot 3's order like the Samsung Fridge.

Student 5



Screenshot 1

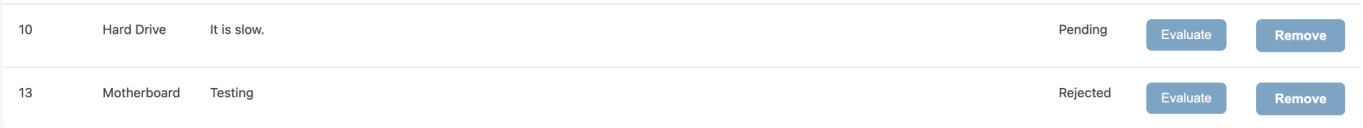
Screenshot 1 shows the home page where you can see OmniTech's policy rules



Screenshot 2

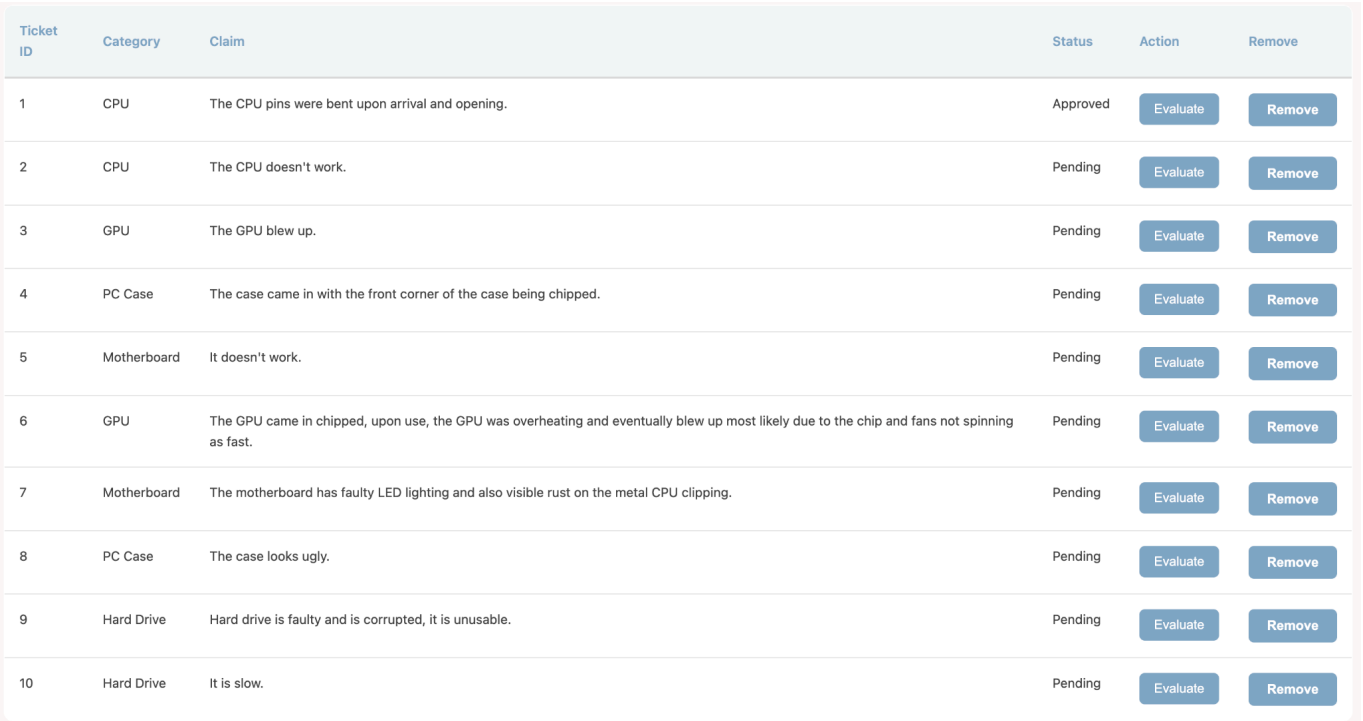
Screenshot 3

Screenshot 2 shows clicking on make ticket in the row for order id 7 where it shows the order's product name and category. You can write a claim and press create ticket or cancel. In screenshot 3, you can see the new ticket with the status as pending and when you press evaluate the AI will generate a decision with reasoning based of the claim, product category and policy rules.



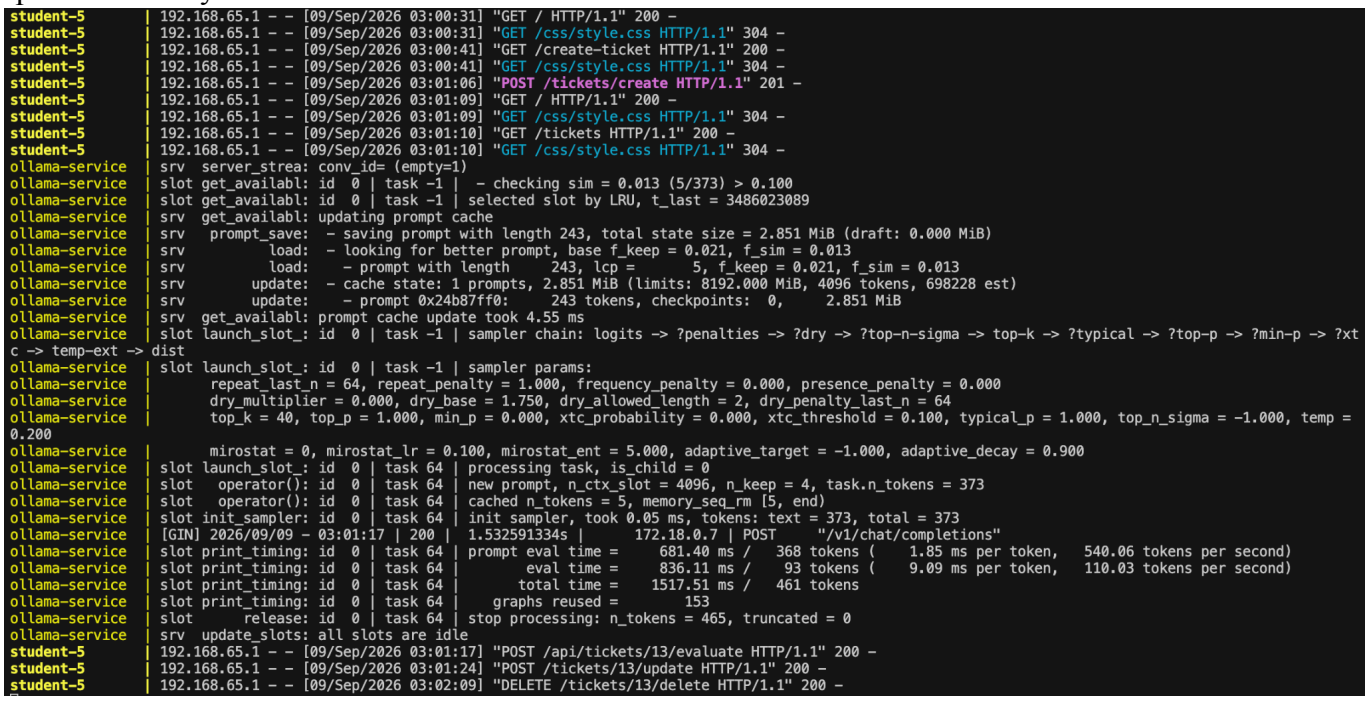
Screenshot 4

Screenshot 4 shows the updated ticket after pressing update button in screenshot 3, it updates instantly after you press it.



Screenshot 5

Screenshot 5 shows the new ticket we made is gone as a result of pressing the remove button. It also updates instantly.



Screenshot 6

20. Known Issues and Limitations

Release 0 focuses on establishing a working integrated foundation. The following limitations will be addressed in later releases.

Issue or limitation	Current impact	Planned improvement
Individual feature packaging	Each student feature currently runs as one Docker Compose container containing its frontend, backend/API logic, and local SQLite data.	Separate frontend, backend/API, and database containers will be introduced in later releases.
Cross-feature data integration	Features currently use their own seeded data. For example, the AI Product Consultant uses its own local product data rather than calling the Product Catalog API.	Implement REST API communication between feature services while preserving clear data ownership.
Shared interface styling	A shared CSS source exists, but several features use local or copied stylesheet files, so the user interface is not yet fully consistent.	Consolidate the design system and apply the shared styling consistently across all feature pages.
AI model availability	AI-Mode depends on the local Ollama container running and the **llama3.2** model being pulled before AI functions can respond.	Improve startup checks, error messages, and fallback handling for unavailable AI services.

MCP, RAG, and multi-agent features	The repository contains placeholder folders for MCP, RAG, and multi-agent services, but these services are not implemented in Release 0.	Implement MCP and RAG in Release 1, followed by multi-agent functionality in Release 2.
Cloud deployment	The Release 0 application is designed for local Docker Compose execution only.	Deploy and validate the integrated application on a cloud platform in Release 2.
End-to-end testing	Individual feature tests and workflows exist, but full cross-feature user journeys are limited because cross-feature APIs are not yet implemented.	Add integrated end-to-end testing once services exchange data through their APIs.

Student 3 Specific Issues:

ID	Issue	Severity	Notes
S3-1	The offline fallback only does keyword matching	Medium	When Ollama is down, the consultant matches on catalog text and category hints. It can't reason about price constraints or trade-offs the way the LLM can. It'll get the right category, but might not pick the "best" product within that category for the user's budget.
S3-2	How good the answers are depends on which model you've pulled	Medium	With llama3.2 the model gets it right on the first try in my testing. Smaller models like qwen2.5:0.5b often need the corrective re-prompt or end up falling back. I only allow one re-prompt to keep response times reasonable.
S3-3	The product catalog is hardcoded, not connected to Student 1's Catalog service	Low	For Release 0, I'm using a fixed 18-product catalog defined in catalog.py. I plan to integrate with Student 1's actual Catalog microservice in Release 1.
S3-4	Authentication is stubbed out	Low	user_id defaults to guest and there's no ownership check, anyone can see and edit any session. Real auth would come from a shared Users microservice.
S3-5	Destructive actions use custom inline confirmation instead of the browser's native dialog	Low	Delete, clear history, and remove recommendation all show an inline "Are you sure?" prompt instead of window.confirm(). I did this because the native dialog gets silently blocked in some embedded webviews.

21. Contribution Logs

Attendance Checkpoint

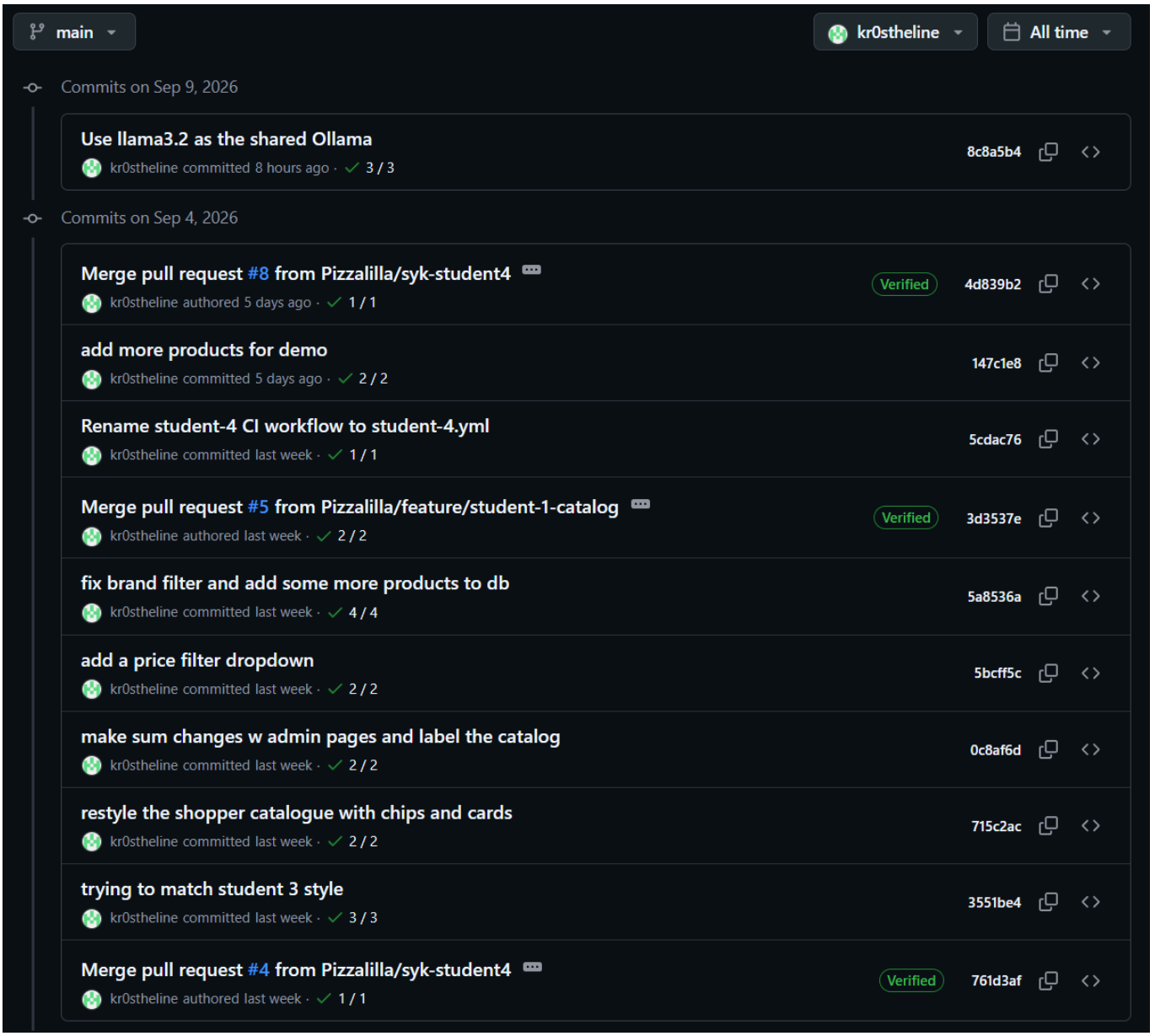
Week	Date	What we did	S1	S2	S3	S4	S5
3	Fri 7 Aug 2026	Team formed, picked our five features	✓	✓	✓	✓	✓
4	Fri 14 Aug 2026	Registration form approved, repository and Compose skeleton set up	✓	✓	✓	✓	✓
5	Fri 21 Aug 2026	Backends, databases and GitHub Actions workflows checked with the tutor	✓	✓	✓	✓	✓
6	Fri 28 Aug 2026	AI modes connected to Ollama, shared CSS theme applied	✓	✓	✓	✓	✓
7	Fri 4 Sep 2026	Integration test, showcase and video recording	✓	✓	✓	✓	✓

Contribution Table

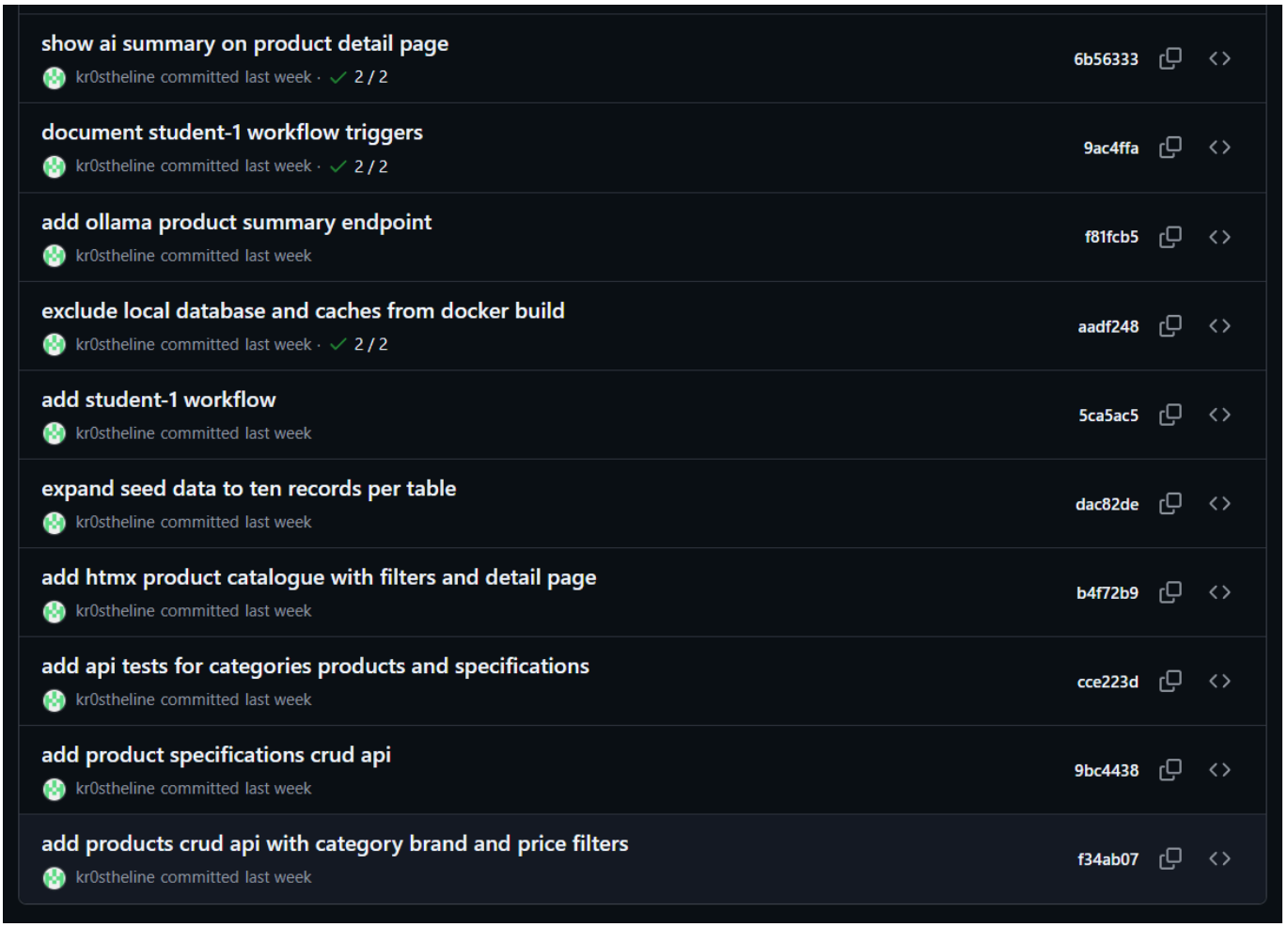
Member	Contribution
Kris Ngo	Project Overview, GitHub Actions Evidence, Docker Compose Evidence, Repository Structure, Known Issues and Limitations.
Kartikay Singh	Agile Team Project Plan, Docker Compose Architecture Diagram, Known Issues and Limitations.
Ethan Schweinsberg	Individual Feature Allocation, Overall Project Plan
Sangyun Kim	Sprint Backlog, Integrated Software Architecture Diagram, Attendance check
Brian Tran	Description of Workflow Files (TBC), Implementation Summary (TBC), Screenshots of the the Integrated Application
Individual	Non-functional and Functional requirements, feature plan, risk management plan, software architecture diagram, workflow diagram, description of workflow diagram. GitHub commits logs.

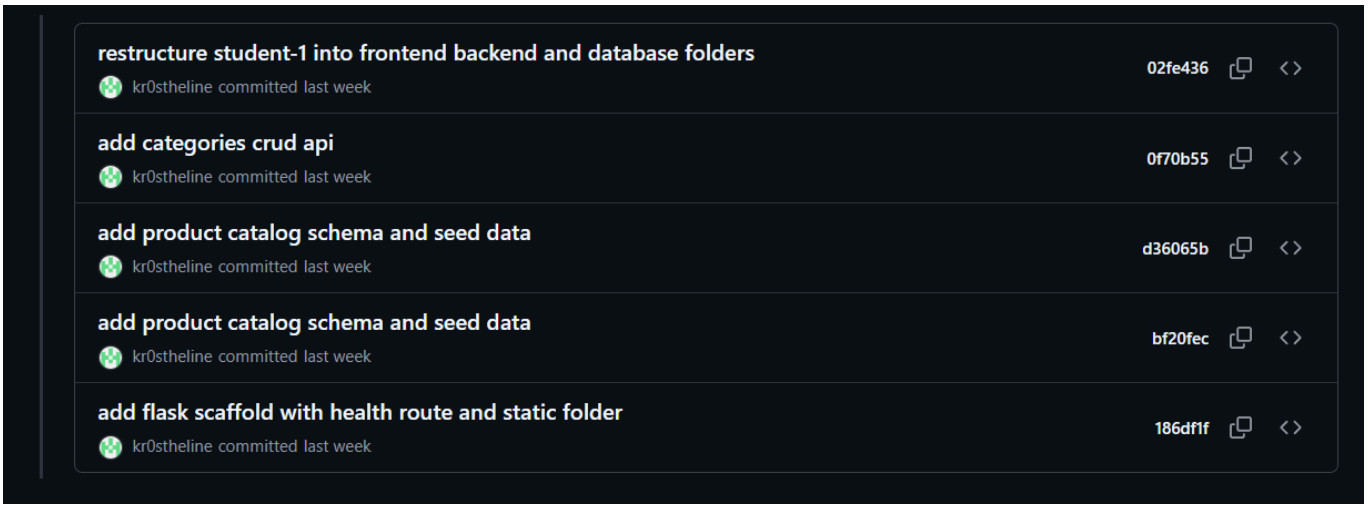
22. GitHub Commit Logs

Student 1 Commit Logs Evidence

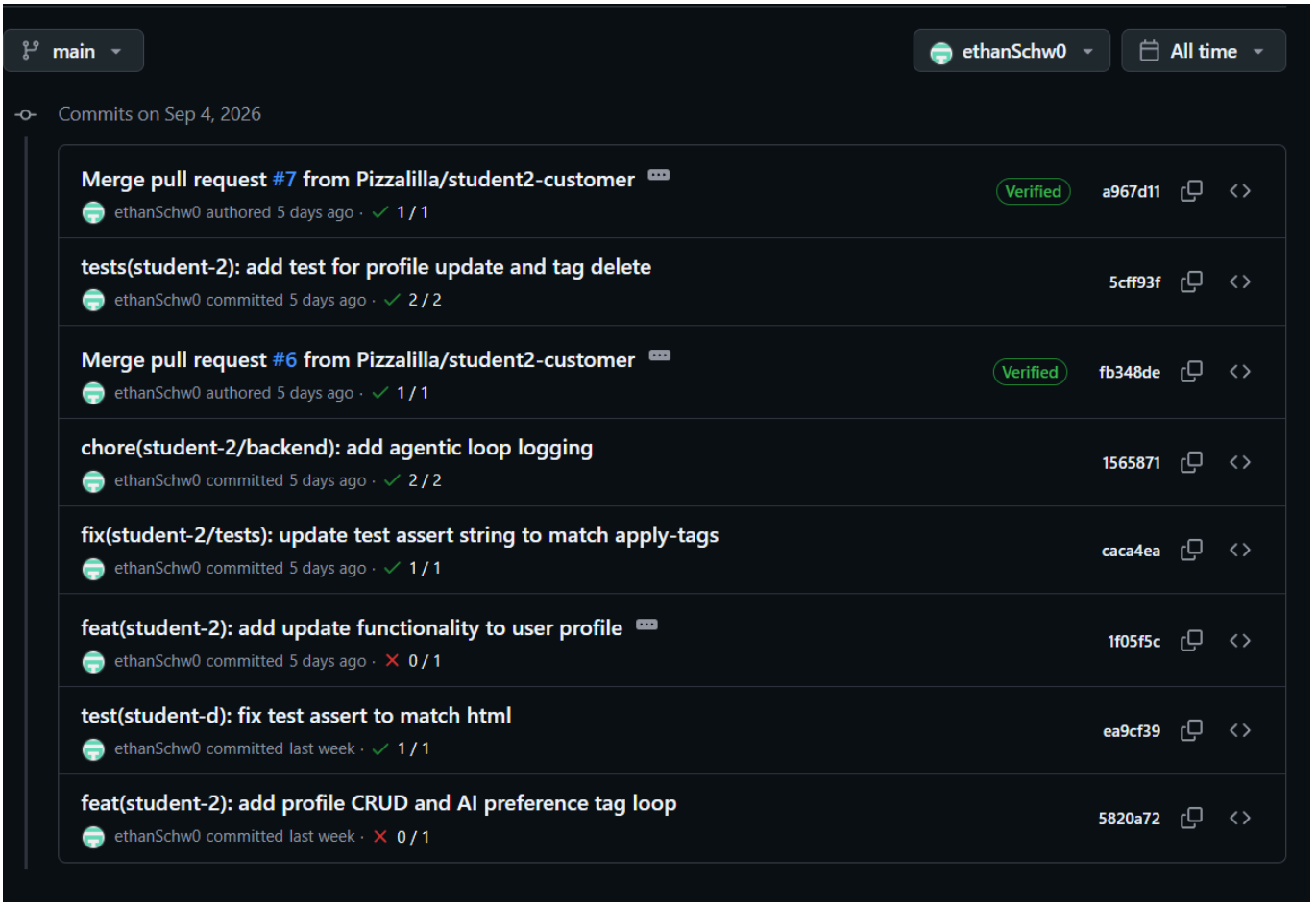



Commits on Sep 3, 2026
Merge pull request #2 from Pizzalilla/feature/student-1-catalog
kr0stheline authored last week · 7 / 7
Verified
e129c6d
label home page services with student names and features
kr0stheline committed last week · 7 / 7
fc04e24
Enhance health check response with service name
kr0stheline and Copilot authored last week · 9 / 9
Verified
ef2c1ab
Use environment variable for Flask debug mode
kr0stheline and Copilot authored last week · 9 / 9
Verified
59ed1fc
Change nav-label color to var(--text-primary)
kr0stheline and Copilot authored last week · 7 / 7
Verified
8bb461f
compile student-1 backend in shared ci workflow
kr0stheline committed last week · 7 / 7
47fc82a
label admin catalogue link as shopper view
kr0stheline committed last week · 7 / 9
9779ecf
serve unified home page on port 8080
kr0stheline committed last week · 2 / 2
d73956b
show a generating panel while the ai review runs
kr0stheline committed last week · 2 / 2
27a755c
document student-1 service pages api and prompts
kr0stheline committed last week · 2 / 2
a847af1
add specification admin page with ai review
kr0stheline committed last week · 2 / 2
618a30d
add product admin card with htmx form
kr0stheline committed last week · 2 / 2
1b8ad55
add category admin page with htmx forms
kr0stheline committed last week · 2 / 2
29e7fbc
add plan act observe adapt loop to ai summary
kr0stheline committed last week · 2 / 2
f80f27d
show ai summary on product detail page
kr0stheline committed last week · 2 / 2
6b56333

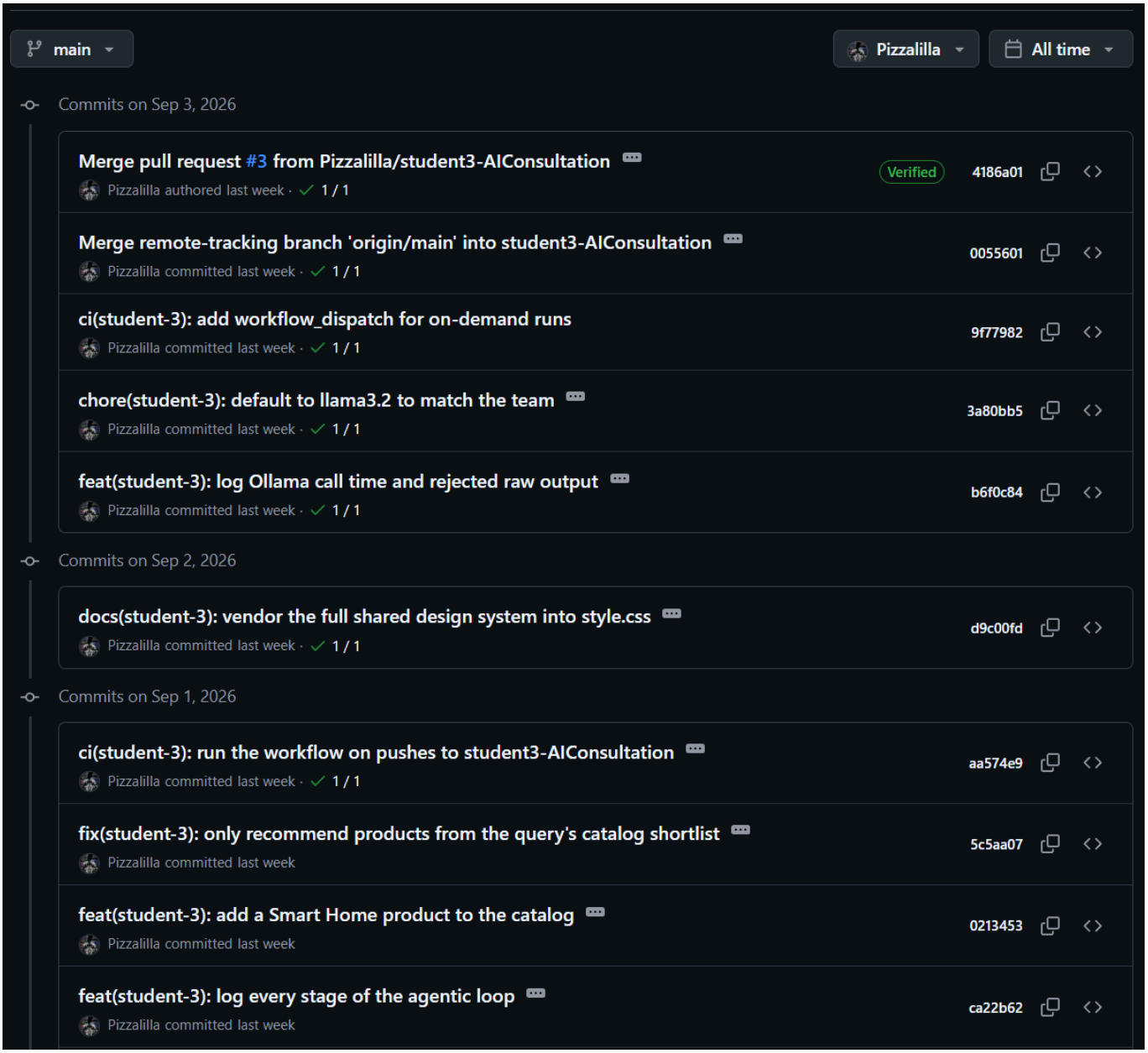

show ai summary on product detail page
kr0stheline committed last week · ✓ 2 / 2
6b56333
document student-1 workflow triggers
kr0stheline committed last week · ✓ 2 / 2
9ac4ffa
add ollama product summary endpoint
kr0stheline committed last week
f81fcb5
exclude local database and caches from docker build
kr0stheline committed last week · ✓ 2 / 2
aadf248
add student-1 workflow
kr0stheline committed last week
5ca5ac5
expand seed data to ten records per table
kr0stheline committed last week
dac82de
add htmx product catalogue with filters and detail page
kr0stheline committed last week
b4f72b9
add api tests for categories products and specifications
kr0stheline committed last week
cce223d
add product specifications crud api
kr0stheline committed last week
9bc4438
add products crud api with category brand and price filters
kr0stheline committed last week
f34ab07

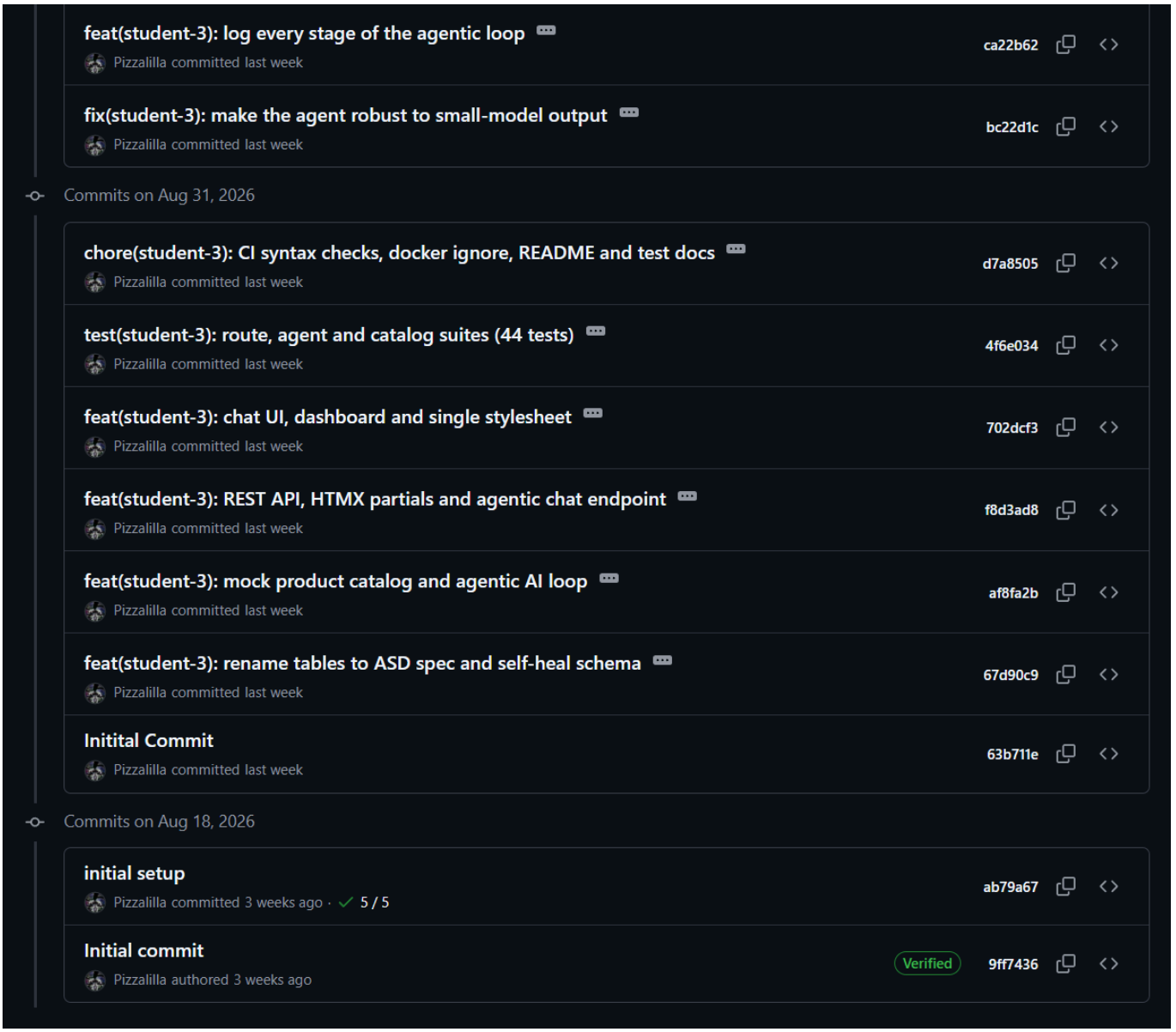

restructure student-1 into frontend backend and database folders
kr0stheline committed last week
02fe436
add categories crud api
kr0stheline committed last week
0f70b55
add product catalog schema and seed data
kr0stheline committed last week
d36065b
add product catalog schema and seed data
kr0stheline committed last week
bf20fec
add flask scaffold with health route and static folder
kr0stheline committed last week
186df1f

Student 2 Commit Logs

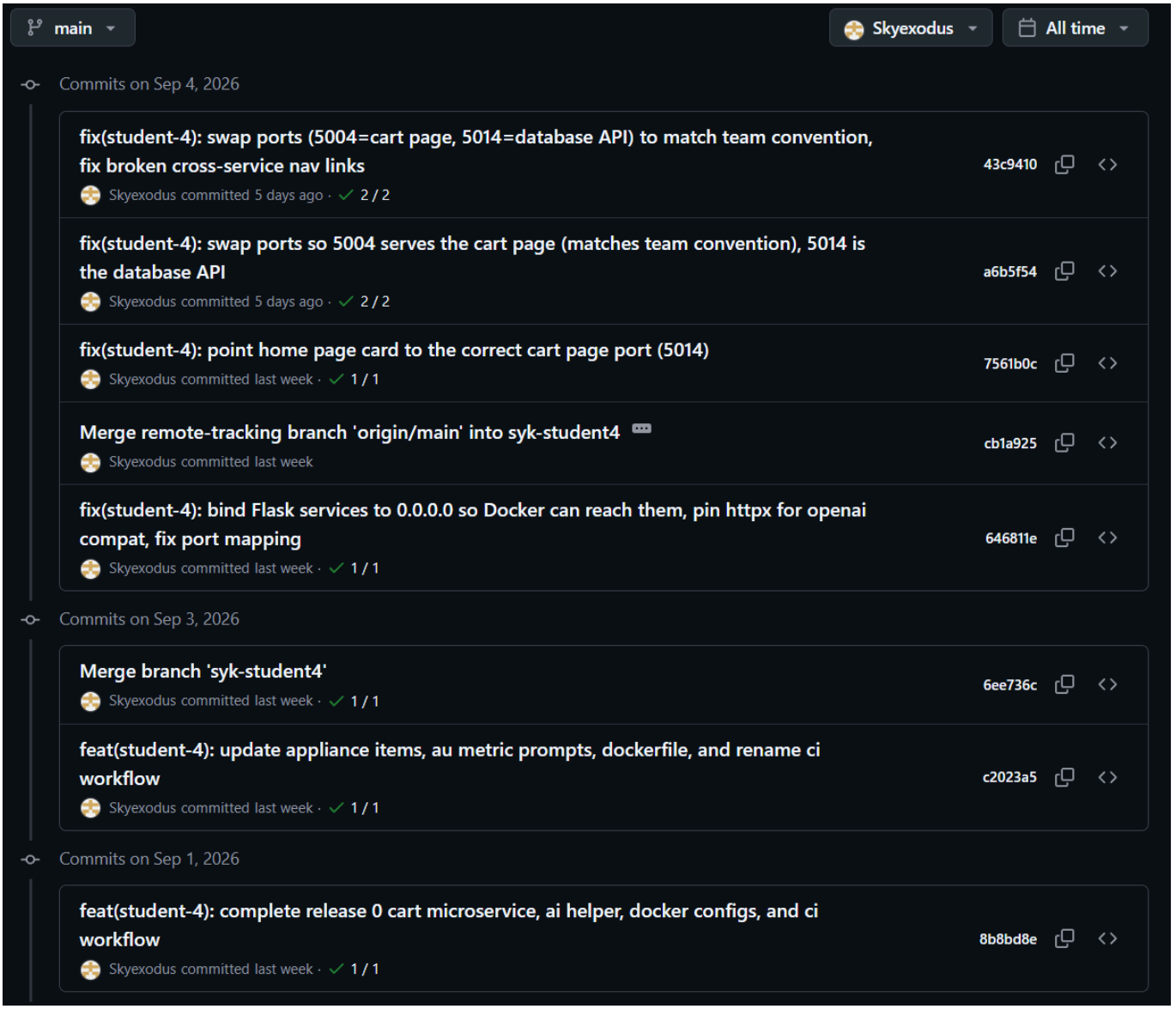


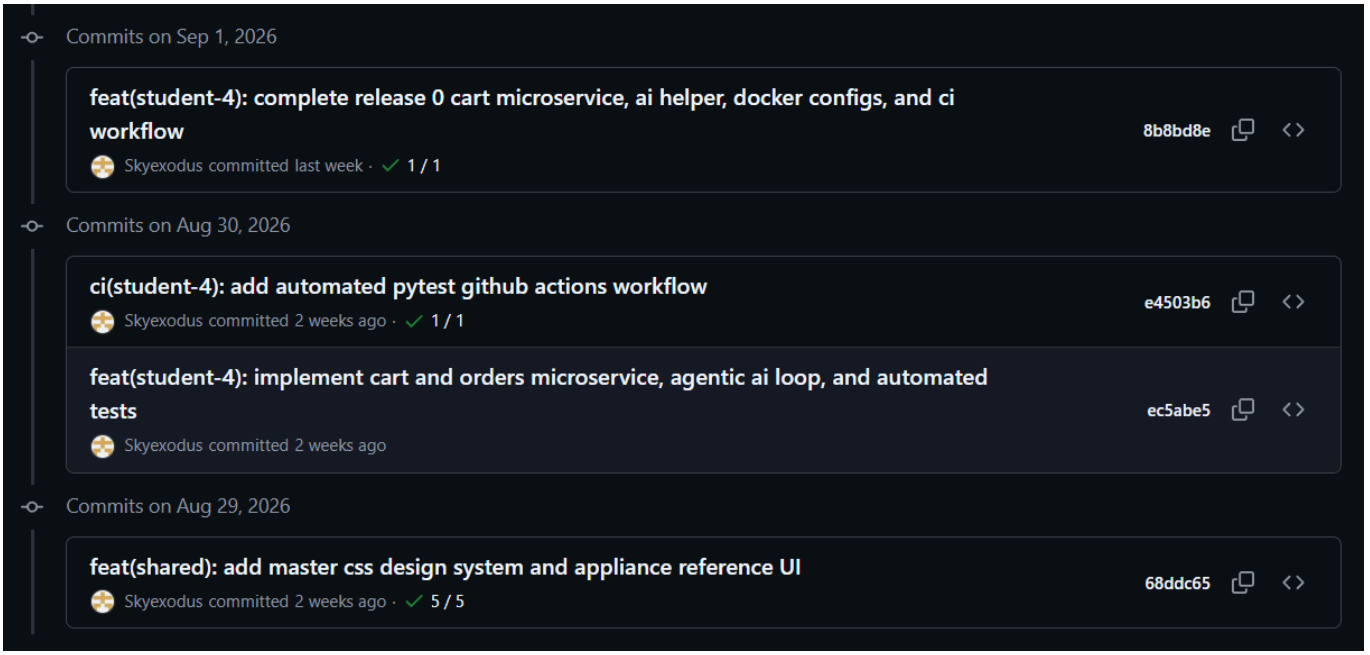
Student 3 commit logs



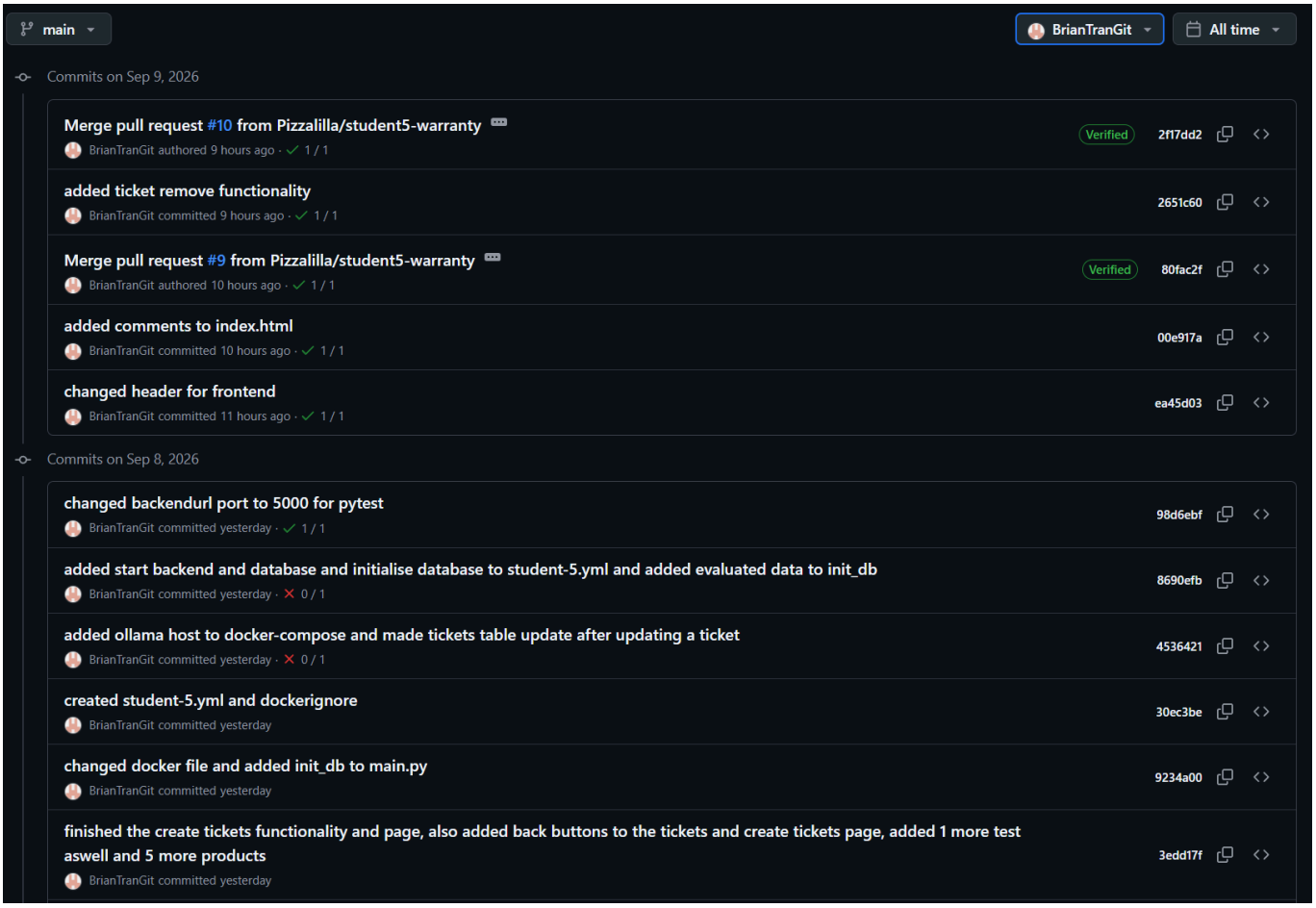

feat(student-3): log every stage of the agentic loop
Pizzalilla committed last week
ca22b62
fix(student-3): make the agent robust to small-model output
Pizzalilla committed last week
bc22d1c
Commits on Aug 31, 2026
chore(student-3): CI syntax checks, docker ignore, README and test docs
Pizzalilla committed last week
d7a8505
test(student-3): route, agent and catalog suites (44 tests)
Pizzalilla committed last week
4f6e034
feat(student-3): chat UI, dashboard and single stylesheet
Pizzalilla committed last week
702dcf3
feat(student-3): REST API, HTMX partials and agentic chat endpoint
Pizzalilla committed last week
f8d3ad8
feat(student-3): mock product catalog and agentic AI loop
Pizzalilla committed last week
af8fa2b
feat(student-3): rename tables to ASD spec and self-heal schema
Pizzalilla committed last week
67d90c9
Initital Commit
Pizzalilla committed last week
63b711e
Commits on Aug 18, 2026
initial setup
Pizzalilla committed 3 weeks ago · 5 / 5
ab79a67
Initial commit
Pizzalilla authored 3 weeks ago
Verified
9ff7436

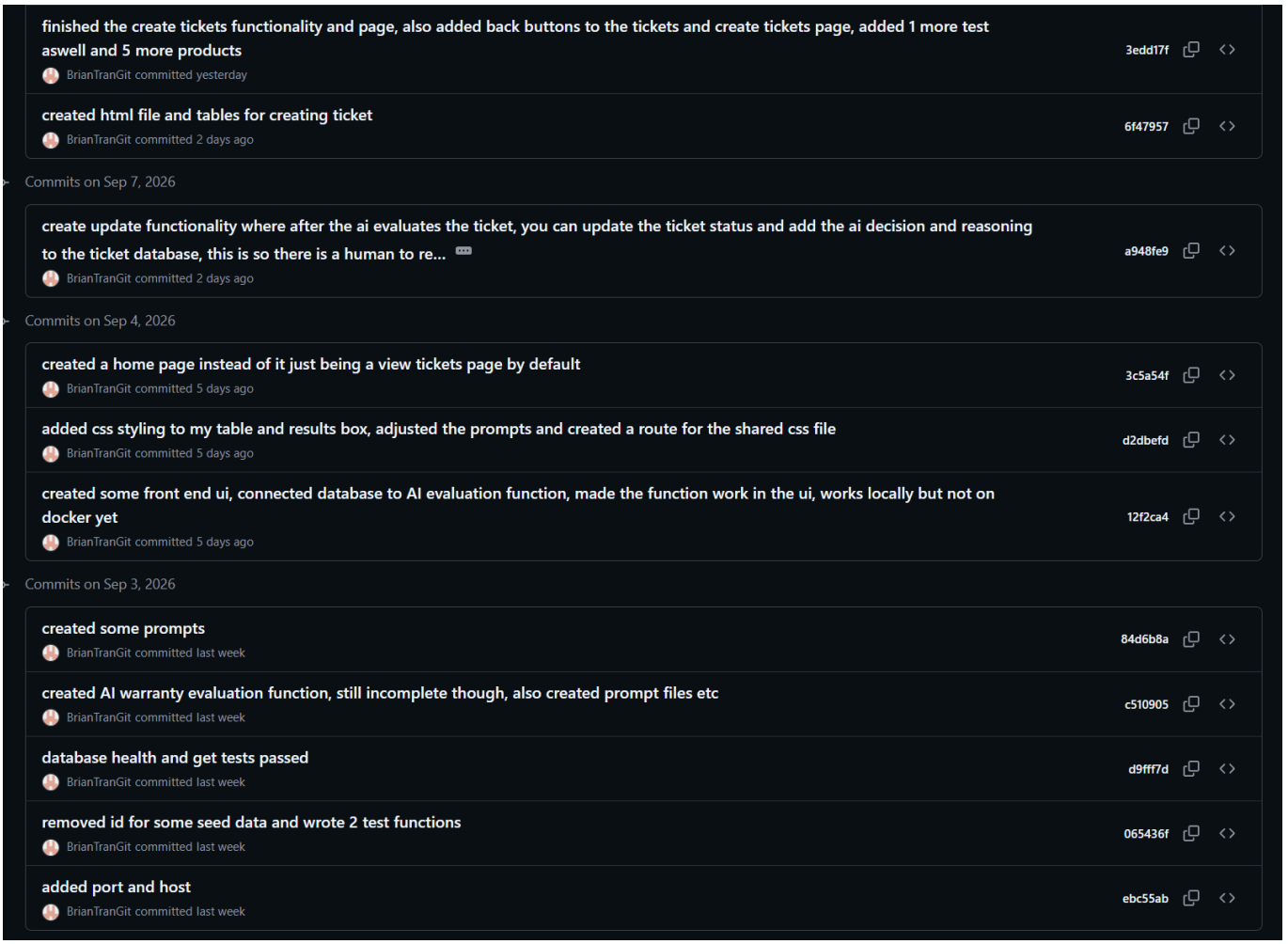
Student 4 commit logs

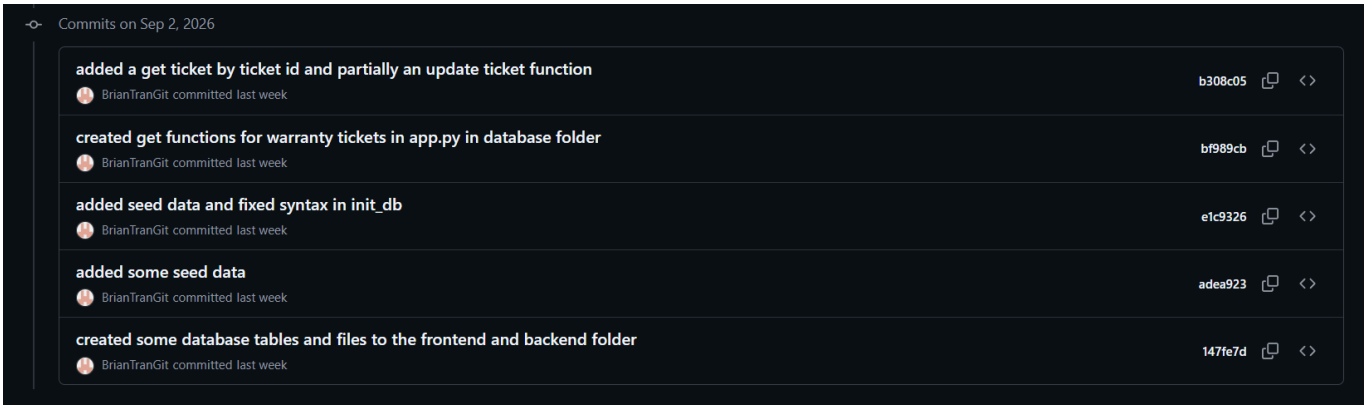




Student 5 Commit Logs







23. Video Demonstration and GitHub Repo

Link to video:
Release 0 Submission.mp4

GitHub repository: https://github.com/Pizzalilla/OmniTech